

O‘ZBEKISTON RESPUBLIKASI
RAQAMLI TEXNOLOGIYALARI VAZIRLIGI
MUHAMMAD AL-XORAZIMIY NOMIDAGI
TOSHKENT AXBOROT TEXNOLOGIYALARI UNIVERSITETI

Himoyaga ruxsat

“ATDT” kafedra mudiri

_____ Raximov N.O.

2025-yil “___” _____

Ta’lim yo‘nalishi: 60610600 – “Dasturiy injiniring”

BITIRUV MALAKAVIY ISHI

Mavzu: Ma’lumotlarni avtomatik zaxiralovchi “AvtoBackup” dasturini ishlab chiqish

Bitiruvchi _____

Mustafoqulov F.X.

Rahbar _____

Egamberdiyev N.A.

HFX
maslahatchisi _____

Shukurullayev S.

Toshkent – 2025

**MUHAMMAD AL-XORAZIMIY NOMIDAGI
TOSHKENT AXBOROT TEXNOLOGIYALARI UNIVERSITETI**

Fakultet: Dasturiy injiniringi

Kafedra: “Axborot texnologiyalarining dasturiy ta’minoti”

Yo‘nalish: 60610600 – Dasturiy injiniring

TASDIQLAYMAN

“ATDT” kafedra mudiri

_____ Raximov N.O.

2025-yil “___” _____

Talaba Mustafoqulov Firdavs Xamidullo o‘g‘li

“Ma’lumotlarni avtomatik zaxiralovchi “AvtoBackup” dasturini ishlab chiqish”
mavzusidagi bitiruv ishini topshirishga

TOPSHIRIQ

1. BMI mavzusi: Ma’lumotlarni avtomatik zaxiralovchi “AvtoBackup” dasturini ishlab chiqish

2.Universitet qarori bilan tasdiqlangan: _____ № ____-sonli

3.Ishni to‘liq bajarish uchun berilgan vaqt: ____

4.Ishning boshlang‘ich ma’lumotlari:

Mazkur bitiruv malakaviy ishda foydalanuvchi kompyuteridagi muhim ma’lumotlarni avtomatik tarzda zaxiralab boruvchi, har bir faylning versiyasini alohida saqlovchi va istalgan vaqtda tiklash imkonini beruvchi AutoBackup nomli dasturiy ta’minotni ishlab chiqish masalasi ko‘rib chiqiladi. Ish jarayonida mavjud zaxiralash tizimlari (Dropbox, Google Drive, Bacula, va boshqalar) texnik tahlil qilinadi hamda

foydalanuvchi ehtiyojlari asosida yangi, soddalashtirilgan, xavfsiz va mahalliy tizimlar bilan mos dastur ishlab chiqiladi. Loyihaning boshlang'ich bosqichida avtomatik backup tamoyillari, fayl versiyalash algoritmlari, ma'lumotlar bazasi bilan ishlash usullari, shifrlash texnologiyalari va foydalanuvchi interfeysi dizayniga oid ma'lumotlar yig'iladi. Tizimni yaratishda JavaFX, Spring Boot, PostgreSQL, Keycloak kabi zamonaviy texnologiyalar asosida dastur arxitekturasi shakllantiriladi.

5. Hisob tushuntirish matni mundarijasi:

Mazkur bitiruv ishi foydalanuvchi kompyuterida saqlanayotgan fayllarning ishonchli, avtomatik va versiyali zaxiralashini ta'minlovchi AutoBackup dasturini yaratishga qaratilgan. Loyihaning dolzarbligi bugungi kunda ma'lumotlar hajmining o'sishi, foydalanuvchi xatolari yoki texnik nosozliklar natijasida ma'lumotlar yo'qolib ketish xavfining yuqoriligi bilan izohlanadi. Shu bois, foydalanuvchining aralashuvisiz zaxiralovchi, vaqt va tizim resurslarini tejovchi, foydalanuvchining ish muhitiga moslashuvchan dastur yaratish zaruriyati mavjud.

6. Grafik materiallar ro'yxati:

Grafik materiallar qismiga AutoBackup dasturining foydalanuvchi interfeysi skrinshotlari, backup holatini ko'rsatuvchi monitoring oynasi, fayl versiyalarini ko'rish ekranlari, tiklash interfeysi, va login sahifasi kiradi. Bundan tashqari, tizim arxitekturasi, ya'ni foydalanuvchi → UI → backend → database/file system oqimini ifodalovchi diagramma, ma'lumotlar bazasi strukturasining ER-diagrammasi, backup algoritmining blok-sxemasi, foydalanuvchi va tizim o'rtasidagi Use Case diagrammalari ham kiritiladi. Har bir funksiya alohida mantiqiy oqim asosida grafik ko'rinishda beriladi.

7. Topshiriq berilgan kun: _____

Rahbar: _____
(imzo)

Topshiriqni oldi: _____
(imzo)

8. BMI ning alohida bo'limlari bo'yicha maslahatchilar:

№	Bo'lim nomi	Maslahatchi	Imzo, sana	
			Topshiriq berdi	Topshiriq oldi
1	Asosiy qism	Egamberdiyev N.A.	26.12.2024	26.12.2024
2	Hayot faoliyati xavfsizligi va ekologiya	Shukurullayev S	15.04.2025	15.04.2025

9. BMI ni bajarilishi bo'yicha kalendar grafik:

№	Malakaviy ishning bo'limlari	Bajarish muddati	Bajarilganligi haqida rahbar imzosi
1	Kirish, masalaning qo'yilishi	12.01.2025	
2	Masalaning qo'yilishi, fan sohasining tahlili	27.01.2025	
3	Tahlil qilingan usullarni tadqiq etish	18.03.2025	
4	Hayot faoliyati xavfsizligi	28.04.2025	
5	Xulosa	29.04.2025	
6	Taqdimot slaydlarini ishlab chiqish	29.04.2025	

BMI rahbari: _____

Bitiruvchi: _____

Annotatsiya

Bitiruv malakaviy ish “AutoBackup” dasturini yaratishga bag‘ishlangan. Dastur foydalanuvchi kompyuteridagi fayllarni avtomatik zaxiralab, ularning xavfsizligini ta’minlaydi. Java va JavaFX texnologiyalarida ishlab chiqilgan bo‘lib, fayl xeshlarini taqqoslash orqali o‘zgarishlarni aniqlaydi va yangilangan fayllarni belgilangan joyga nusxalaydi. Har bir foydalanuvchiga alohida shifrlash kaliti ajratilib, fayl ma’lumotlari shifrlanadi. “AutoBackup” qulay interfeysli, tezkor va ishonchli zaxiralash vositasidir.

Аннотация

Выпускная квалификационная работа посвящена разработке программы “AutoBackup” для автоматического резервного копирования файлов. Приложение на Java с использованием JavaFX сравнивает хеши файлов, определяет изменения и сохраняет обновлённые файлы в указанную папку. Для каждого пользователя создаётся отдельный ключ шифрования, обеспечивающий защиту данных. “AutoBackup” - удобный, быстрый и надёжный инструмент резервного копирования.

Annotation

The graduation qualification work “AutoBackup” focuses on developing a tool for automatic file backup. Created in Java using JavaFX, the system compares file hashes, detects changes, and copies updated files to a target folder. Each user is assigned a unique encryption key to secure file content. “AutoBackup” is a user-friendly, fast, and reliable backup solution.

Mundarija

KIRISH.....	7
I-BOB. MA'LUMOTLAR O'ZGARISHINI AVTOMATIK ZAXIRALOVCHI DASTURLAR TAHLILI	9
1.1 Operatsion Tizimlar uchun ma'lumotlarni zaxiralovchi dasturlar tahlili	9
1.2 Bulutli texnologiyalarda avtomatik zaxiralovchi xizmatlar.....	16
1.3 Avtomatik zaxiralovchi dasturlarni ishlab chiqishda foydalaniladigan texnologiyalar.....	26
1.4 I-bob bo'yicha xulosa.....	31
II-BOB. "AutoBackup" dasturini ishlab chiqish bosqichlari.....	33
2.1 Dastur loyahasini ishlab chiqish	33
2.2 Dasturning ma'lumotlar bazasini loyihalash	35
2.3 Dasturning server qismini ishlab chiqish	41
2.4 Dasturning foydalanuvchi interfeysi qismini ishlab chiqish	51
2.5 II-bob bo'yicha xulosa	59
III-BOB. HAYOT FAOLIYATI XAVFSIZLIGI VA EKOLOGIYA	61
3.1. Havo muhitining changlanganligi va uning odamga ta'siri	61
3.2. Ishlab chiqarish binolari va uskunalarga yong'in xavfsizligi talablari.....	70
XULOSA	79
FOYDALANILGAN ADABIYOTLAR VA MANBALAR.	81

KIRISH

Hozirgi kunda dunyo miqyosida raqamli texnologiyalar kundalik hayotning ajralmas qismiga aylanmoqda. Har bir tashkilot, kompaniya va hatto yakka foydalanuvchilar o'z faoliyatida katta hajmdagi ma'lumotlar bilan ishlashga majbur. Ushbu ma'lumotlar orasida korporativ hujjatlar, moliyaviy hisob-kitoblar, dizayn fayllari, shaxsiy yozuvlar yoki texnik loyihalar bo'lishi mumkin. Aynan shuning uchun, ularni muntazam ravishda zaxiralab borish tobora dolzarb masalaga aylanyapti. Har qanday kutilmagan vaziyat – xoh u texnik nosozlik, xoh inson xatosi yoki zararli dastur bo'lsin – ma'lumotlar yo'qolishiga olib kelishi mumkin. Bu esa o'z navbatida katta moddiy va ma'naviy zararlarga sabab bo'ladi.

AutoBackup dasturining ahamiyati bugungi kunda nafaqat mahalliy bozor, balki butun dunyo miqyosida ham keskin oshib bormoqda. Axborot texnologiyalari va raqamli infratuzilmalarning har tomonlama kengayishi bilan korxonalar va individual foydalanuvchilar kundalik faoliyatlarida turli xil raqamli hujjat, loyiha fayllari, shaxsiy surat-video materiallari kabi muhim ma'lumotlarga tayanadi. Shu bois, kutilmagan texnik nosozliklar, kiberhujumlar yoki oddiygina inson omili tufayli ma'lumotlar yo'qolishi xavfi tobora dolzarb masalaga aylanmoqda. Global miqyosda fayllarni vaqtda zaxiralash va avariya holatlarida ularni tezda tiklash imkonini beruvchi yechimlarga talab kundan-kunga ortmoqda. AutoBackup esa aynan shunday muammoga javob berish uchun yaratilgan: foydalanuvchi interfeysi qulay, zaxira jarayoni avtomatik nazorat ostida, hamda fayllar faqat o'zgarganida qayta yuklanadi. Bu yondashuv tarmoqli trafikni va saqlash joyi hajmini optimallashtiradi, natijada korxona infratuzilmasiga tushadigan yuk kamayadi, xarajatlar tejaladi. Versiyalash mexanizmi esa dasturiy ta'minot yangilanishlari, hujjatlar ustida bir nechta mutaxassislar bir vaqtda ishlayotgan loyihalarda avvalgi holatga qaytish zarur bo'lganda hech qanday muammosiz orqaga qaytishni ta'minlaydi. Shu tariqa

AutoBackup global raqobat sharoitida ham o'z o'rnini topib, korporativ va shaxsiy segmentlarda ham ishonchli va samarali zaxiralash vositasi sifatida paydo bo'lmoqda.

Shuningdek, global bozorga nisbatan mahalliy, ya'ni O'zbekiston sharoitidagi dolzarblik ham alohida ta'kidlanishi lozim. O'zbekistonda axborot texnologiyalarini rivojlantirishga katta e'tibor qaratilmoqda: hukumat darajasida "raqamli O'zbekiston" konsepsiyasi ilgari surilgan, ta'lim, sog'liqni saqlash va boshqa sohalarda elektron xizmatlarni joriy etish bo'yicha dasturlar amalga oshirilmoqda. Buning natijasida nafaqat davlat idoralari, balki kichik va o'rta biznes ham raqamli transformatsiya jarayonini boshlab yubordi. Lekin infratuzilmalar barqarorligi, kiberxavfsizlik va ma'lumotlarning boy berilishi muammolari hanuz hal etilishi kerak bo'lgan masalalardir. Ko'p hollarda zaxira jarayonlari manual usullarga tayanadi yoki turli xil aralashgich dasturlar yordamida tashkil etiladi, bu esa inson omili va texnik cheklovlar tufayli kamchiliklarga olib keladi.

Natijada, global tendensiyalar va mahalliy regulyator talablari nuqtai nazaridan AutoBackup dasturi bugungi kunda hamkorlikdagi ishlarda, ta'lim muassasalarida, bank va moliya sohasida, shuningdek, foydalanuvchining kundalik vazifalarini zaif bog'lagan ofis muhitida katta ahamiyatga ega. U nafaqat ishonchli zaxiralashni, balki vaqt va resurslarni tejashni, shuningdek, favqulodda vaziyatlarda o'z ma'lumotlarini tezda tiklash imkoniyatini taqdim etadi. Shu bois, AutoBackup nafaqat O'zbekistonda, balki butun dunyo bo'ylab ma'lumot xavfsizligini ta'minlash bo'yicha zamonaviy va qulay yechim sifatida o'z o'rnini mustahkamlab bormoqda.

I-BOB. MA'LUMOTLAR O'ZGARISHINI AVTOMATIK ZAXIRALOVCHI DASTURLAR TAHLILI

1.1 Operatsion Tizimlar uchun ma'lumotlarni zaxiralovchi dasturlar tahlili

Operatsion tizimlar uchun mo'ljallangan ma'lumotlarni zaxiralovchi dasturlar – bu foydalanuvchilarning qurilmalaridagi muhim bo'lgan ma'lumotlarni yo'qolishini oldini oluvchi dasturlarni aytishimiz mumkin. Bu dasturlar ma'lumotlarni avtomatik zaxiralash va kerak bo'lganda tezlik bilan tiklash imkoniyatini berishi kerak.

Ma'lumotlarni zaxiralovchi dasturlarni ham quyidagi bir nechta guruhga bo'lishimiz mumkin:

1. Foydalanuvchilar uchun desktop zaxiralovchi dasturlar.
2. Cloud (Bulut) ga asoslangan daturlar.
3. Zaxiralash usullari bo'yicha.
4. Ochiq manbali (Open source) dasturlar.
5. Platformaga bog'liq dasturlar.

Ushbu turdagi dasturlar bir birdan qo'llanuvchi sohasi va imkoniyatlari bilan bir biridan farq qiladi. Quyida bir nechta ma'lumotlarni zaxiralovchi dasturlarni ko'rib chiqamiz.

1. Acronis True Image - bu Acronis International GmbH tomonidan ishlab chiqilgan, foydalanuvchilarning shaxsiy ma'lumotlarini ishonchli saqlovchi va himoyalovchi zamonaviy zaxiralash dasturidir. Dastur foydalanuvchilarga qulaylik yaratadigan bir qator muhim imkoniyatlarni o'z ichiga oladi.

Birinchidan, diskni to'liq zaxiralash funksiyasi mavjud bo'lib, u orqali kompyuterda joylashgan barcha ma'lumotlar operatsion tizim, dasturiy ta'minotlar, sozlamalar va foydalanuvchi fayllari yagona nusxada saqlanadi. Bu esa tizimda kutilmagan muammolar yuzaga kelganida barcha ma'lumotlarni bir zumda asl holatiga tiklash imkonini beradi.

Ikkinchidan, real vaqtda (real-time) zaxiralash funksiyasi mavjud. Ya'ni, kompyuterdagi fayllarda har qanday o'zgarish yuz bersa, dastur avtomatik tarzda ushbu o'zgarishlarni zaxiraga oladi. Bu esa ayniqsa doimiy ishlovchi tizimlarda ishlaydigan foydalanuvchilar uchun juda qo'l keladi.

Uchinchidan, dastur rejalashtirilgan zaxiralash imkoniyatini ham taqdim etadi. Foydalanuvchi istalgan vaqt oralig'ida har soat, har kun yoki boshqa kerakli davrlarda zaxira olishni belgilab qo'yishi mumkin. Bu funksiyaning mavjudligi zaxiralash jarayonini butunlay avtomatlashtirishga yordam beradi.

Acronis True Image dasturi pullik asosda taklif etiladi va foydalanuvchi ehtiyojiga qarab bir nechta litsenziya variantlariga ega. 2025-yil holatiga ko'ra, rasmiy manbalar asosida quyidagi narxlar amal qiladi:

- Essential versiyasi - faqat lokal zaxiralash imkoniyatlarini o'z ichiga oladi, yillik narxi taxminan \$49.99;
- Advanced versiyasi - 500 GBgacha bulutli saqlash va kengaytirilgan funksiyalarni taqdim etadi, narxi \$89.99;
- Premium versiyasi - 1 TBgacha bulutli xotira, blokcheyn texnologiyasi asosida fayllarni tekshirish kabi imkoniyatlarni o'z ichiga oladi, yillik narxi \$124.99 ni tashkil qiladi.

2. EaseUS Todo Backup - bu 2004-yilda Xitoyda tashkil etilgan EaseUS Software kompaniyasi tomonidan ishlab chiqilgan dastur bo'lib, asosan foydalanuvchilarning shaxsiy yoki ish bilan bog'liq ma'lumotlarini ishonchli zaxiralash va tiklash imkonini yaratadi. Ushbu kompaniya o'z faoliyatini yo'qolgan fayllarni tiklash, diskni boshqarish va ma'lumotlarni xavfsiz saqlashga yo'naltirgan. Aynan EaseUS Todo Backup dasturi foydalanuvchilarga bir nechta foydali imkoniyatlarni taqdim etadi.

Birinchidan, dastur tizim va fayllarni to'liq zaxiralash funksiyasini taqdim etadi. Bu orqali butun operatsion tizim, disk bo'limlari, foydalanuvchining barcha fayllari

yagona nusxada saqlanadi. Tizimda nosozliklar paydo bo'lsa, foydalanuvchi o'z qurilmasini oldingi holatiga osongina qaytara oladi.

Ikkinchidan, avtomatik zaxiralashni rejalashtirish imkoniyati mavjud. Foydalanuvchi istalgan vaqt oralig'ida har kuni, har hafta yoki foydalanuvchining ehtiyojiga mos boshqa muddatlarda zaxira olishni belgilab qo'yishi mumkin. Bu esa foydalanuvchining aralashuvisiz zaxira jarayonining uzluksiz ishlashini ta'minlaydi.

Uchinchidan, versiyalash funksiyasi har bir fayl o'zgarganidan so'ng uning oldingi nusxasini alohida saqlab boradi. Bu esa foydalanuvchiga istalgan vaqtda kerakli faylning avvalgi holatiga qaytish imkonini beradi.

Bundan tashqari, dastur bulut texnologiyalari bilan integratsiyalashgan. Ya'ni foydalanuvchi o'z fayllarini Google Drive, OneDrive yoki Dropbox kabi mashhur bulut saqlash xizmatlariga to'g'ridan-to'g'ri zaxiraga joylashi mumkin. Bu esa ma'lumotlarga istalgan qurilmadan va joydan kirish imkonini yaratadi.

EaseUS Todo Backup foydalanuvchilarga ma'lumotlarni zaxiralash va tiklash uchun turli xil versiyalarni taklif etadi. Har bir versiya foydalanuvchining ehtiyojlariga qarab tanlanadi.

- Free versiyasi: Tizim, disk, bo'lim va fayllarni zaxiralash va tiklash va 30 kunlik 250GB bulutli saqlash imkoniyati

- Home versiyasi: Disk, bo'lim va tizimni klonlash , universal tiklash, bulutli zaxiralash (1TBgacha) va Avtomatik zaxiralash jadvali, fayl istisnolari, Outlook zaxiralash va tiklash. Narxi \$39.95

- Business versiyasi: Kengaytirilgan zaxiralash va tiklash funksiyalari, windows Server, SQL Server va Exchange qo'llab-quvvatlovi , PXE tarmoq yuklash, buyruq satri orqali zaxiralash va markazlashtirilgan boshqaruv va ko'p foydalanuvchili muhitlar uchun moslik.

Narxilari esa quyidagicha:

- Workstation: \$49.00/yil

- Server: \$199.00/yil
- Advanced server: \$299.00/yil

3. Time Machine - bu macOS operatsion tizimiga o'rnatilgan Apple kompaniyasi tomonidan ishlab chiqilgan avtomatik zaxira olish tizimi hisoblanib uni Apple o'zining MacOS tizimida ishlovchi qurilmalariga o'rnatadi. U Mac kompyuteringizdagi barcha fayllarni, tizim fayllaridan tortib hujjatlar, rasmlar va dasturlargacha avtomatik tarzda zaxiraga oladi va kerak bo'lsa, osongina tiklashga imkon beradi. Bu zaxiraga olish haftalik, kunlik va soatlik ko'rinishida bo'lishi mumkin. Zaxiralash uchun qurilma xotirasidan joy ajratishi yoki tashqi qurilmani ulashi kerak hisoblanadi. Bu tizimning katta kamchiligi har bir zaxiralash katta hajm egallashidadir.

4. Bacula - bu ochiq manbali va turli operatsion tizimlarda ishlay oladigan zaxiralash va tiklash tizimidir. U dastlab 2002-yilda ishlab chiqilgan bo'lib hozirgacha foydalanilib kelinmoqda. Ushbu tizim foydalanuvchilarga disk, server, fayl, bo'lim yoki butun tizimni zaxiraga olish va kerakli vaqtda tiklash imkoniyatini beradi.

Bacula modular arxitekturaga ega bo'lib, quyidagi asosiy qismlardan tashkil topgan: Director (asosiy boshqaruvchi), Storage Daemon (saqlash qurilmalari bilan ishlovchi modul), File Daemon (ma'lumot olinadigan qurilmalarda o'rnatiladi), va Catalog (ma'lumotlar bazasi orqali fayllar haqida axborotni yuritadi). Shuningdek, foydalanuvchi tizimni buyruq satri orqali yoki veb-interfeys orqali boshqarishi mumkin.

Bacula boshqa zaxiralovchi tizimlariga qaraganda ko'proq moslashuvchan va kengaytiriladigan xususiyatga ega. Community Edition bu bepul versiya bo'lib, uni istalgan foydalanuvchi o'z ehtiyojlariga moslab sozlashi mumkin. Enterprise Edition dan esa kompaniyalar foydalanishadi. Enterprise Edition pullik bo'lib u qo'shimcha qulayliklar va texnik xizmatni taqdim etadi. Enterprise Edition versiyaning narxlari foydalanuvchi soni, serverlar soni va texnik xizmat darajasiga qarab o'zgaradi. O'rtacha yillik litsenziya narxi \$2,000 dan \$10,000 gacha bo'lishi mumkin.

Ammo Bacula dan samarali foydalanish uchun texnik bilimga ega bo'lish talab etiladi. Chunki tizimning sozlanmalari ko'proq qo'lda, buyruqlar orqali amalga oshiriladi. Biroq, bir marta to'g'ri o'rnatilganidan so'ng u ishonchli va avtomatik ishlovchi zaxira tizimiga aylanadi.

5. Duplicati - bu bepul va ochiq manbali zaxiralash dasturi bo'lib, u foydalanuvchilarga o'z ma'lumotlarini shifrlangan va arxivlangan holatda saqlash imkonini beradi. Dastur birinchi navbatda oddiy foydalanuvchilar va kichik tashkilotlar uchun mo'ljallangan bo'lishiga qaramay u o'zining funksionalligi jihatidan ko'plab pullik tizimlarga raqobat qila oladi.

Duplicati ning asosiy afzalliklaridan biri bu - kuchli ma'lumotlar xavfsizligidir. Duplicati foydalanuvchining fayllarini AES-256 algoritmi yordamida shifrlaydi va serverda bu ma'lumotlarni ZIP formatida saqlaydi. Bu esa xavfsizlikni va saqlash joyidan tejab qolish imkonini beradi. Ma'lumotlar lokal qurilmalarda yoki bulutli xizmatlarda saqlanishi mumkin. Google Drive, OneDrive, Dropbox, Amazon S3, MEGA, FTP/SFTP va boshqa ko'plab saqlash xizmatlari bilan integratsiyalashgan. Duplicati har bir zaxiralashdan avval va keyin tekshiruvlar o'tkazadi. Dastur inkremental zaxiralash usulidan foydalanadi, ya'ni faqat oxirgi backupdan keyin o'zgargan fayllargina saqlanadi. Bu esa tizimga yuklamani kamaytiradi va saqlash joyini tejaydi.

Dastur to'liq veb-interfeys asosida boshqariladi. Foydalanuvchi brauzer orqali o'z backup rejalarini sozlashi, tiklashi yoki monitoring qilish imkoniga ega. Foydalanish jarayoni oddiy va qulay bo'lib, texnik bilimga ega bo'lmagan foydalanuvchilar ham oson foydalanishi mumkin. Duplicati hozirgi paytda Windows, Linux va macOS operatsion tizimlarini qo'llab-quvvatlaydi. U butunlay bepul tarqatiladi. Dastur ishlab chiqilganidan beri doimiy ravishda rivojlantirib borilmoqda.

Duplicati bepul, xavfsiz va funksional zaxiralash tizimi bo'lib, foydalanuvchiga o'z ma'lumotlarini bulutli yoki lokal manzilda xavfsiz va avtomatlashtirilgan tarzda

saqlash imkonini beradi. Ayniqsa, shifrlash va siqish texnologiyalari tufayli u yengil, tez va ishonchli backup vositasi sifatida ajralib turadi.

6. UrBackup - bu bepul va ochiq manbali (open-source) zaxiralash dasturi bo‘lib, mijoz-server (client-server) arxitekturasida ishlaydi. Dastur lokal tarmoq yoki internet orqali kompyuterlar va serverlardagi ma’lumotlarni markazlashtirilgan tarzda zaxiralash uchun mo‘ljallangan. Ayniqsa, kichik va o‘rta bizneslar uchun qulay va ishonchli yechim hisoblanadi.

UrBackup tizimining asosiy afzalligi shundaki, u fayl darajasidagi (file backup) va disk tasviri asosidagi (image backup) zaxiralashni qo‘llab-quvvatlaydi. Fayl darajasidagi zaxiralashda alohida hujjatlar, rasm yoki ish fayllari nusxalanadi, disk tasvirida esa butun tizim bir necha daqiqada saqlab olinadi. Bu yondashuv turli ehtiyojlarga qarab moslashadi va tezkor tiklash imkonini beradi. Dastur avtomatik zaxiralashni rejalashtirish, fayllarning versiyalarini saqlash, o‘chirilgan fayllarni tiklash va zaxira fayllarining yaxlitligini doimiy ravishda tekshirish imkoniyatlarini taqdim etadi. Bundan tashqari, UrBackup juda tez ishlaydi birinchi backupdan so‘ng, keyingi backuplar faqat o‘zgargan fayllarni nusxalaydi. Bu esa tizimga yuk tushishini kamaytiradi va disk xotirasini tejaydi. UrBackup’ning yana bir muhim jihati veb-interfeys orqali boshqarish imkoniyati. Bu orqali barcha mijoz kompyuterlari va ularning zaxira holatini masofadan turib kuzatish mumkin. Foydalanuvchi interfeysi sodda va tushunarli, shu bois texnik bilimga ega bo‘lmagan foydalanuvchilar ham uni boshqarishda qiynalishmaydi. Ushbu tizim Windows, Linux va macOS tizimlarini qo‘llab-quvvatlaydi. Server komponenti odatda Linux yoki Windows serverga o‘rnatiladi, mijoz komponentlari esa ish stansiyalarida ishlaydi.

UrBackup dasturi asosan bepul tarqatiladi. Biroq, Professional versiyasi ham mavjud bo‘lib, u ko‘proq xavfsizlik, masofaviy boshqaruv, va kengaytirilgan kuzatuv imkoniyatlarini o‘z ichiga oladi. Rasmiy ma’lumotlarga ko‘ra:

– Community (bepul) versiyasi: to‘liq funksional, ochiq manbali va kichik tarmoqlar uchun yetarli.

– Professional versiyasi: \$290/yil dan boshlanadi, bu narx ko‘proq xavfsizlik choralari va monitoring vositalarini o‘z ichiga oladi.

UrBackup bu o‘z infratuzilmasini nazorat ostida saqlamoqchi bo‘lgan foydalanuvchilar va tashkilotlar uchun mo‘ljallangan samarali zaxiralash tizimi hisoblanadi. U qulay interfeys, yuqori tezlik va kengaytirilgan sozlash imkoniyatlari bilan ajralib turadi. Ayniqsa, tarmoq orqali bir nechta qurilmalardan bir vaqtda zaxira olishda foydali hisoblanadi.

Dastur nomi	Platforma	Foydalanish turi	Versiyalash	Shifrlash	Bulutga zaxiralash	Interfeys turi
Acronis True Image	Windows, macOS	Tijorat (pullik)	Ha	AES-256	Ha	Grafikli (GUI)
EaseUS Todo Backup	Windows	Freemium (ba’zi pullik)	Ha	Ha	Ha (ba’zi versiyalarda)	Grafikli (GUI)
Time Machine	faqat macOS	Bepul (macOSda default)	Ha	Ha	Yoq (faqat lokal)	Grafikli (macOS)
Bacula	Linux, Windows	Ochiq manbali (kompleks)	Ha	Ha	Moslashtirilsa bo‘ladi	Buyruq qatori (CLI)
Duplicati	Windows, macOS	Ochiq manbali	Ha	AES-256	Ha (Dropbox, GDrive va b.)	Veb-interfeys
UrBackup	Windows, Linux	Ochiq manbali	Ha	Ha	Yoq (lokal va LAN)	Veb-interfeys

1.1.1 – jadval. Zaxiralash dasturlari taqqoslash jadvali

1.2 Bulutli texnologiyalarda avtomatik zaxiralovchi xizmatlar

Bulutli texnologiyalarga asoslangan avtomatik zaxiralash (backup) xizmatlari zamonaviy IT infratuzilmaning ajralmas qismiga aylangan. Ular foydalanuvchilarga o'z ma'lumotlarini xavfsiz saqlash, istalgan vaqtda qayta tiklash va qurilmadan mustaqil ravishda ulanish imkonini beradi.

Bulutli avtomatik zaxiralash - bu foydalanuvchi fayllari yoki tizim ma'lumotlarini internet orqali masofaviy serverga (bulutga) avtomatik ravishda nusxalab borish jarayonidir. Bu xizmatlar foydalanuvchi ishtirokisiz ma'lum vaqt oraliq'ida yoki fayl o'zgarishiga asoslangan holda ishlaydi [1].



1.2.1-rasm. Cloud zaxiralash

Quyida bulutli zaxiralash xizmatlari haqida aytib o'tilgan:

Google Drive va Google Workspace. Ushbu zaxiralash xizmatlari bugungi raqamli dunyoda foydalanuvchilarning ma'lumotlarini xavfsiz saqlash, istalgan vaqtda ularga kirish imkoniyatini yaratish va avtomatlashtirilgan muhitda ishlash imkonini beruvchi eng ko'p targalgan xizmatlardan hisoblanadi. Ushbu texnologiyalar Google kompaniyasi tomonidan taqdim etilgan. Ayniqsa o'quv yurtlari, davlat tashkilotlari hamda xususiy kompaniyalar uchun keng ko'lamli imkoniyatlarni taklif etadi. Fayllarni bulutda saqlash nafaqat qo'shimcha qulaylik yaratish bilan birga ularni yo'qotish

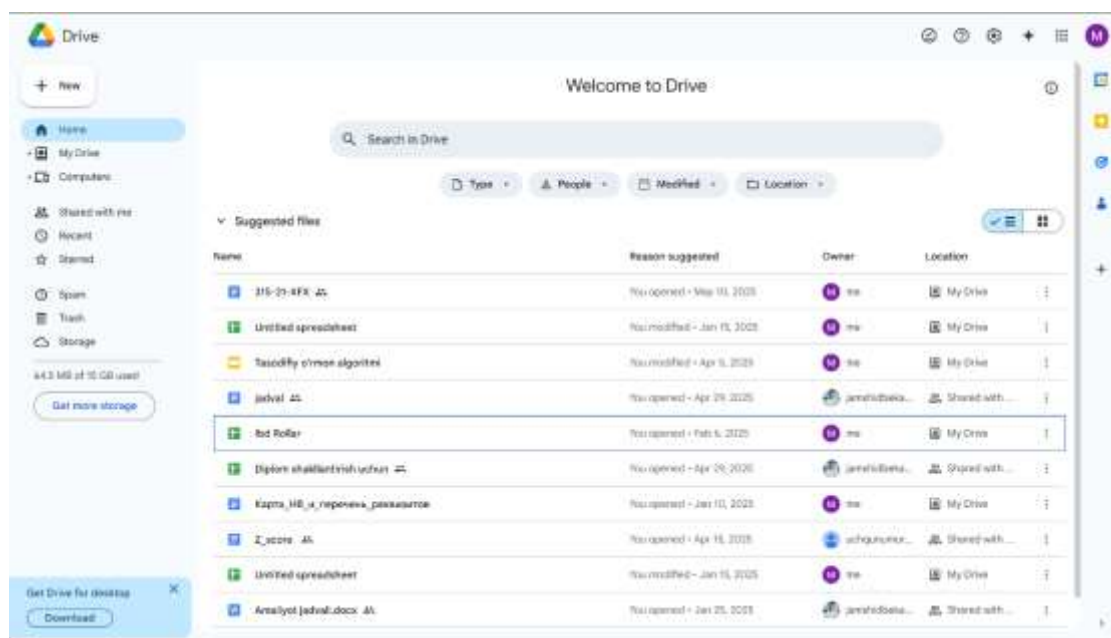
xavfini kamaytiradi. Google Drive yordamida foydalanuvchi ma'lumotlarini bulutga yuklaydi. Bu jarayonni "Backup and Sync" nomli dastur boshqaradi. Oddiy foydalanuvchi uchun bu juda qulay hisoblanadi. Ya'ni bir marta sozlab qo'yilgach, qolgan barcha jarayonlar orqa fonda amalga oshadi. Bu xizmat aslida juda kuchli xavfsizlik asoslariga ega. Zaxiralangan fayllar Google'ning kuchli serverlarida AES-256 algoritmda shifrlanadi. Shuningdek, ikkita autentifikatsiya bosqichi orqali foydalanuvchining akkaunti himoyalanaadi. Shu tufayli, hatto qurilma yo'qolgan taqdirda ham ma'lumotlar xavfsiz qoladi. Bu ayniqsa universitetlar yoki ofislar uchun muhim. Har qanday texnik nosozlikda o'quv materiallari yoki biznes hujjatlari tiklanishi mumkin.

Google Workspace bundan ham kengroq imkoniyatlarga ega. U Google Drive ni ham o'z ichiga oladi, lekin unga Gmail, Calendar, Docs, Sheets, Slides kabi xizmatlar ham qo'shilgan. Ayniqsa ta'lim muassasalari uchun Workspace for Education versiyasi mavjud bo'lib, bu orqali o'qituvchilar va talabalar birgalikda ishlashi, vazifalarni onlayn bajarishi, natijalarni baholashi mumkin bo'ladi. Shu bilan birga, barcha ishlar avtomatik tarzda zaxiralab boriladi. Google Workspace foydalanuvchilarga "Vault" nomli maxsus vositani ham taqdim etadi. Bu vosita yordamida elektron pochta xabarlarini ko'rish mumkin. Bundan tashqari fayllar va boshqa ma'lumotlar tarixini ko'rib chiqish, kerak bo'lsa, ularni tiklash yoki saqlab qolish mumkin. Ko'plab davlat idoralari aynan ushbu vositadan foydalanadi. Chunki u axborotlarning nazoratini ta'minlaydi va qonunchilikka muvofiq arxivlash imkonini beradi. Google Drive foydalanuvchining kompyuteridagi belgilangan papkalarni real vaqtda kuzatadi. Har qanday o'zgarish aniqlansa ushbu o'zgarishlar darhol bulutdagi fayl bilan sinxronlashtiriladi. Bu sinxronizatsiya internet yordamida amalga oshadi. Bu esa foydalanuvchiga istalgan joyda ishlash imkonini beradi. Qiziqarli jihatlaridan yana biri fayllarning versiyalanishi. Agar foydalanuvchi hujjatni xato bilan o'zgartirsa yoki noto'g'ri saqlab qo'yishi mumkin. Bunda Drive orqali hujjatning oldingi versiyasini

tiklab olish mumkin. Bu funktsiya ko‘plab foydalanuvchilarni keraksiz stressdan xalos qiladi va muhim hujjatlarning yo‘qolishini oldini oladi.

Google Drive afzalliklaridan yana biri integratsiya hisoblanadi. U ko‘plab boshqa xizmatlar bilan ulangan. Masalan, Zoom, Slack, Trello va boshqa ofis platformalari bilan birga ishlay oladi. Bu foydalanuvchining umumiy ish unumdorligini oshiradi. Shu bilan birga zaxira ma’lumotlar sun’iy intellekt yordamida tahlil qilinadi. Masalan, zarur fayllar avtomatik tarzda tavsiya etilishi yoki faylning ruxsatlari xavfsizlik sababli o‘zgartirilishi mumkin. Ma’lumotlarni Google Drive orqali zaxiralash nafaqat individual foydalanuvchilar uchun, balki yirik tashkilotlar uchun ham katta ahamiyatga ega. Ma’lumotlar soni kundan-kunga oshib borayotgan bir vaqtda, ularni xavfsiz va tartibli saqlash eng muhim ehtiyojlardan biridir. Ayniqsa, bugungi kunda raqamli tahdidlar kuchaygan bir paytda, bulutda shifrlangan va avtomatik backup tizimlariga ega bo‘lish zaruriy hisoblanadi.

Google Drive va Google Workspace backup xizmatlari foydalanuvchiga qulay interfeys, yuqori darajadagi xavfsizlik, avtomatlashtirilgan jarayonlar va kengaytirilgan imkoniyatlar orqali ishonchli va zamonaviy ma’lumot saqlash tizimini taqdim etadi.



1.2.2-rasm. Google Drive ishchi oynasining ko‘rinishi

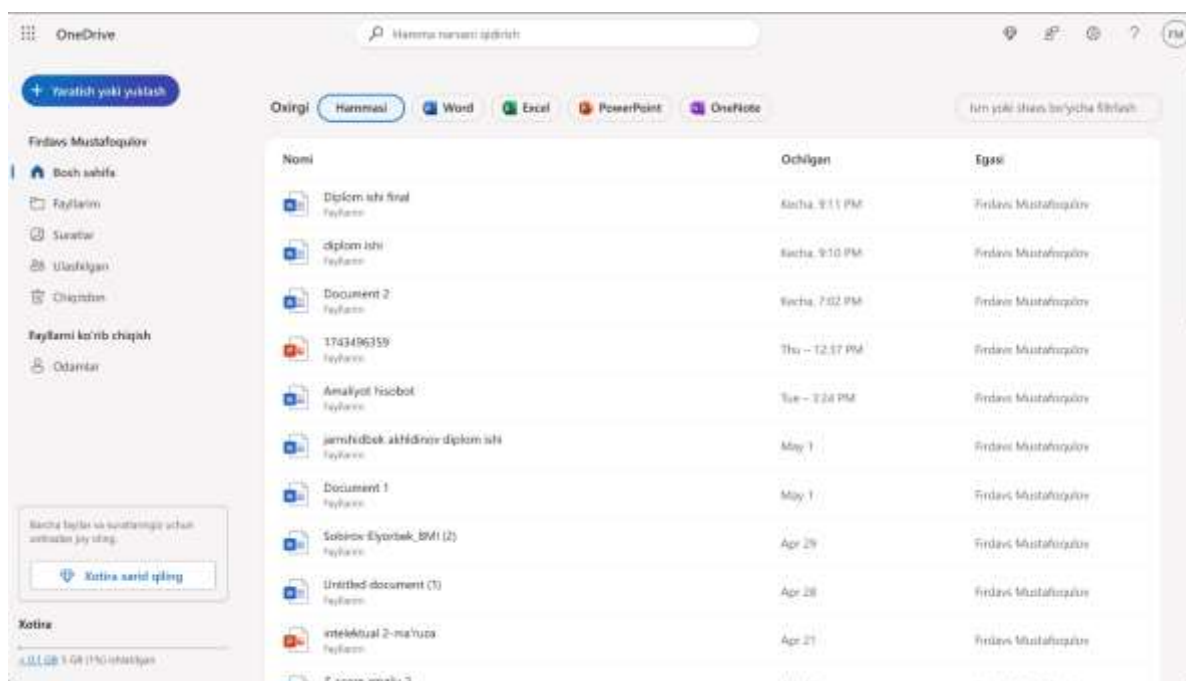
Microsoft OneDrive - bu Microsoft tomonidan ishlab chiqilgan bulutli saqlash va avtomatik zaxiralash xizmatidir. U foydalanuvchilarga hujjatlar, rasmlar, videolar, va boshqa turdagi fayllarni masofaviy serverda xavfsiz saqlash imkonini beradi. OneDrive, ayniqsa, Windows foydalanuvchilari uchun chuqur integratsiyalashganligi sababli, kundalik ish jarayonida o'zini eng qulay va foydali vosita sifatida namoyon qilmoqda. OneDrive'ning asosiy afzalliklaridan biri avtomatik zaxiralash hisoblanadi. Foydalanuvchi o'z kompyuteridagi "Documents", "Desktop" va "Pictures" kabi asosiy papkalarni OneDrive bilan bog'lab qo'yadi. Shundan so'ng, ushbu papkalarga joylangan barcha fayllar internetga ulangan zahoti avtomatik ravishda Microsoft serverlariga yuklab boriladi. Bu esa foydalanuvchini fayllarni qo'lda zaxiralash majburiyatidan xalos qiladi va ish unumdorligini oshiradi. Xizmatda mavjud bo'lgan ersiyalash (version history) imkoniyati foydalanuvchilarga hujjatlarning oldingi holatini tiklab olishga yordam beradi. Masalan, Word yoki Excel hujjatida noto'g'ri o'zgarish kiritilgan bo'lsa, foydalanuvchi OneDrive orqali hujjatning avvalgi versiyasini ko'rib chiqishi va tiklashi mumkin. Bu ayniqsa ta'lim tizimida, loyiha ustida ishlayotgan talabalar va o'qituvchilar uchun juda foydali.

OneDrive, shuningdek, xavfsizlik borasida ham ilg'or texnologiyalarga asoslanadi. Microsoft foydalanuvchi ma'lumotlarini AES 256-bit shifrlash bilan himoya qiladi. Bundan tashqari, ikki bosqichli autentifikatsiya (2FA) va "Personal Vault" funksiyasi orqali maxfiy hujjatlar uchun qo'shimcha himoya choralari mavjud. Personal Vault bu maxsus xavfsizlik bilan himoyalangan papka bo'lib, unga kirish uchun har safar qo'shimcha autentifikatsiya talab etiladi.

Microsoft OneDrive'ning yana bir muhim jihati u Office 365 (hozirda Microsoft 365) to'plami bilan to'liq integratsiyalashgan. Bu integratsiya natijasida foydalanuvchilar Word, Excel, PowerPoint va OneNote hujjatlarini OneDrive'da real vaqt rejimida saqlay olishadi. Mobil qurilmalar uchun OneDrive ilovasi ham mavjud. U Android va iOS platformalarida ishlaydi. Shu sababli, foydalanuvchi har doim o'z

ma'lumotlariga istalgan joyda kirish imkoniyatiga ega bo'ladi. Korxona va tashkilotlar uchun OneDrive for Business versiyasi ham mavjud. Bu versiyada administratorlar fayllarga kirish huquqlarini nazorat qila olishadi. Xavfsizlik siyosatlarini belgilashadi va xodimlar o'rtasida ma'lumotlar almashinuvini tizimli tashkil qilishlari mumkin. Ayniqsa, katta hajmdagi ma'lumotlar bilan ishlaydigan kompaniyalar uchun bu imkoniyatlar qo'l keladi.

OneDrive foydalanuvchilarga 5 GB bepul saqlash joyini taklif etadi. Agar ko'proq joy kerak bo'lsa, 100 GB, 1 TB va 6 TB gacha pullik rejalar mavjud. Ayniqsa Microsoft 365 obunasi bilan birga 1 TB joy va Office dasturlari to'plami birga keladi, bu esa narx-navo va qulaylik nuqtai nazaridan foydalanuvchi uchun foydali yechim hisoblanadi. Texnik tomondan, OneDrive fayllarni bulutda saqlash bilan bir qatorda, ular bilan oflayn rejimda ham ishlash imkonini beradi. Ya'ni, foydalanuvchi internetga ulanmagan paytda fayl ustida ishlashi mumkin, keyinroq tarmoqqa ulanganida esa barcha o'zgarishlar avtomatik sinxronlanadi.



1.2.3-rasm. OneDrive ishchi oynasi

Dropbox - bu zamonaviy bulutli saqlash va avtomatik zaxiralash tizimlarining eng mashhurlaridan biri bo'lib, u foydalanuvchilarga o'z fayllarini xavfsiz tarzda masofaviy serverlarda saqlash, sinxronlash va boshqalar bilan oson ulashish imkonini beradi. Dastlab 2007-yilda tashkil etilgan ushbu xizmat ko'p o'tmay millionlab foydalanuvchilar e'tiborini qozondi va hozirda ham biznes, ta'lim, shaxsiy foydalanish kabi sohalarda keng qo'llanilmoqda. Dropbox'ning eng asosiy afzalliklaridan biri bu sodda interfeys va kuchli sinxronizatsiya imkoniyatidir. Foydalanuvchi o'z kompyuterida maxsus Dropbox papkasini yaratadi. Ushbu papkaga joylangan fayllar avtomatik tarzda bulutga yuklanadi va istalgan boshqa qurilmada ayni bir hisob orqali avtomatik ko'rinadi. Ya'ni, foydalanuvchi ofisda ish boshlagan faylini kechqurun uydagi kompyuterdan davom ettirishi mumkin. Dropbox orqali nafaqat fayllarni saqlash, balki ularni real vaqt rejimida o'zgartirish va boshqalar bilan ulashish ham mumkin. Har bir fayl uchun alohida havola (link) hosil qilish va uni elektron pochta, messenjer yoki ijtimoiy tarmoqlar orqali jo'natish imkoni mavjud. Bu, ayniqsa, guruhda ishlovchi talabalar, tadqiqotchilar yoki jamoaviy loyiha ustida ish olib borayotgan xodimlar uchun katta qulaylik yaratadi.

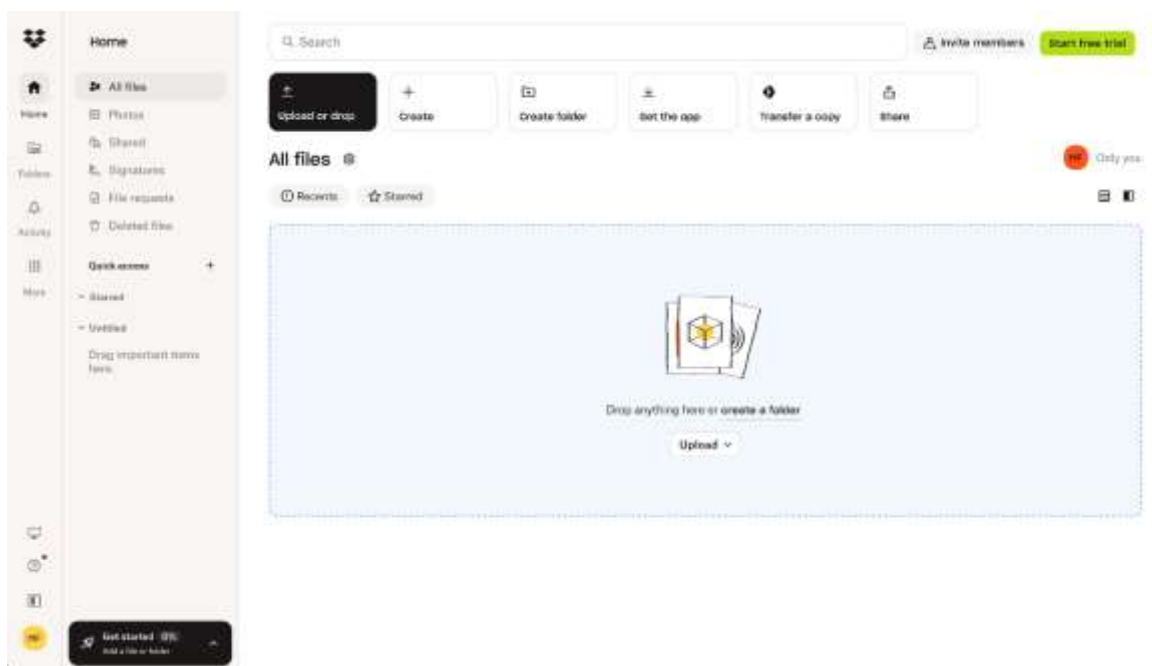
Dropbox xizmatida versiyalash imkoniyati mavjud bo'lib, bu foydalanuvchiga faylning avvalgi holatlarini tiklash imkonini beradi. Oddiy bepul versiyada bu imkoniyat 30 kungacha amal qiladi, pullik tariflarda esa 180 kungacha saqlanadi. Bu foydalanuvchi tasodifan o'chirib yuborgan yoki noto'g'ri o'zgartirgan fayllarni osongina tiklab olishiga yordam beradi.

Xavfsizlik masalasi Dropbox'da yuqori darajada e'tiborga olingan. Har bir fayl serverga yuborilishidan oldin TLS (Transport Layer Security) protokoli orqali shifrlanadi, saqlanish jarayonida esa AES-256 bit shifrlash standarti qo'llaniladi. Bundan tashqari, ikki bosqichli autentifikatsiya yordamida akkauntlar qo'shimcha darajada himoyalanaadi. Korxonalar uchun esa qo'shimcha auditoriya jurnallari, kirish siyosatlari va ma'lumotlarni geografik hududda joylashtirish imkoniyatlari mavjud.

Dropbox'ning biznes versiyasi Dropbox Business tashkilotlar uchun mo'ljallangan bo'lib, jamoa a'zolari orasidagi hujjat aylanishini yengillashtiradi. Admin panel orqali foydalanuvchilarning huquqlarini boshqarish, umumiy papkalar yaratish, fayllarga kirish tarixini kuzatish va xavfsizlik siyosatlarini joriy etish mumkin. Ayniqsa, davlat idoralari yoki ta'lim muassasalari uchun bu imkoniyatlar juda dolzarb hisoblanadi. Mobil qurilmalar uchun mo'ljallangan Dropbox ilovasi yordamida Android va iOS foydalanuvchilari telefon kamerasidan tushirilgan suratlar va videolarni avtomatik zaxiralashlari mumkin. Ilova orqali fayllarni ko'rish, tahrirlash, havola yuborish va shaxsiy ma'lumotlar bilan ishlash ancha qulay va tezkor. Bu, ayniqsa, harakatchanlik talab qilinadigan kasblarda yoki onlayn ta'limda foydali vositadir. Dropbox dasturiy ta'minotchilarga ham keng imkoniyatlar yaratadi. Ochilgan API'lar yordamida ko'plab uchinchi tomon ilovalari Dropbox bilan integratsiyalashgan. Masalan, Adobe, Slack, Zoom, Trello kabi mashhur xizmatlar bevosita Dropbox bilan bog'lanib, fayllarni bevosita bulutdan chaqirish va ularga real vaqt rejimida o'zgartirish kiritishga imkon beradi.

Dropbox bepul versiyasida 2 GBgacha saqlash joyi taqdim etadi. Pullik rejalar esa foydalanuvchining ehtiyojiga qarab 2 TB, 3 TB va undan ortiq saqlash hajmini taklif qiladi. Narxlari boshqa xizmatlarga nisbatan nisbatan yuqoriroq bo'lishi mumkin, biroq xizmat sifati va ishonchliligi buning o'ziga yarasha kompensatsiyasi hisoblanadi.

Dropbox bu ishonchli, sodda, xavfsiz va platformalararo moslashuvchan bulutli zaxiralash tizimi bo'lib, nafaqat oddiy foydalanuvchilarga, balki tashkilotlar, o'quv muassasalari va professional jamoalar uchun ham a'lo darajadagi tanlov hisoblanadi. Ayniqsa, diplom ishi doirasida zamonaviy zaxiralash texnologiyalarini tahlil qilishda Dropbox kabi xizmatlar alohida o'rganishga loyiqdir.



1.2.4-rasm. Dropbox ishchi oynasi

iCloud - bu Apple kompaniyasi tomonidan ishlab chiqilgan bulutli saqlash va avtomatik zaxiralash xizmati bo'lib, u Apple ekosistimiga kiruvchi barcha qurilmalar uchun markaziy, ishonchli va qulay yechim hisoblanadi. iPhone, iPad, Mac, Apple Watch va hatto Windows kompyuterlari ham ushbu xizmatdan foydalanish imkoniyatiga ega. Apple zaxiralash tizimini ekosistema darajasiga ola chiqqan kompaniya hisoblanadi. iCloud foydalanuvchining barcha muhim ma'lumotlarini – rasmlar, kontaktlar, hujjatlar, ilovalar sozlamalari va hatto qurilma sozlamalarini avtomatik tarzda zaxiralab boradi. iCloud xizmatining asosiy afzalligi – uning chuqur integratsiyalashganligi. iPhone yoki iPad foydalanuvchisi yangi qurilmani ishga tushirganda, avvalgi qurilmadagi barcha ma'lumotlar iCloud yordamida bir necha daqiqada tiklanadi. Bu foydalanuvchini ilovalarni qayta o'rnatish, kontaktlarni qo'lda kiritish yoki sozlamalarni eslab qolish zaruratidan butunlay xalos qiladi. Shuningdek, foydalanuvchi bir qurilmada boshlagan ishini boshqasida davom ettira oladi – bularning barchasi iCloud hisobiga.

iCloud xizmatida mavjud bo'lgan eng foydali funksiyalardan biri bu iCloud Backup. Bu vosita foydalanuvchining telefonini Wi-Fi tarmog'iga ulanganda va

quvvatlagichga ulab qo'yilganda avtomatik tarzda butun qurilmaning zaxira nusxasini yaratadi. Unda ilovalar, foto va video materiallar, sozlamalar, xabarlar, va Apple Wallet'dagi ma'lumotlar saqlanadi. Shunday qilib, foydalanuvchi qurilmasini yo'qotgan yoki almashtirgan taqdirda ham iCloud orqali uni to'liq holatda tiklash mumkin. iCloud Photos funksiyasi foydalanuvchining barcha suratlarini va videolarini avtomatik tarzda bulutga yuklaydi. Bu nafaqat qurilmadagi saqlash joyini bo'shatadi, balki rasmlarni boshqa qurilmalarda ham ko'rish va tahrirlash imkoniyatini yaratadi. Foydalanuvchi Mac kompyuterida tahrir qilgan surat iPhone'da ham darhol yangilanadi bu esa ishni soddalashtiradi va sinxronlikni ta'minlaydi. Xavfsizlik masalasida Apple juda qat'iy siyosat yuritadi. iCloud'ga yuklangan barcha ma'lumotlar AES-128 bit yoki undan yuqori darajadagi shifrlash bilan himoyalaniadi. Ba'zi hollarda esa, foydalanuvchi iCloud Keychain yoki Apple Health ma'lumotlarini faollashtirgan bo'lsa, end-to-end (uch-to-uch) shifrlash qo'llaniladi, ya'ni ushbu ma'lumotlarga Apple ham kira olmaydi. Bundan tashqari, ikki bosqichli autentifikatsiya (Two-Factor Authentication) orqali akkaunt qo'shimcha himoya ostida bo'ladi.

Apple foydalanuvchilarga 5 GBgacha bepul saqlash joyini taqdim etadi. Biroq, ko'pchilik foydalanuvchilar uchun bu hajm yetarli bo'lmagani sababli Apple pullik obunalarni taklif qiladi: 50 GB, 200 GB yoki 2 TBgacha bo'lgan variantlar mavjud. Ayniqsa, oilaviy rejim (Family Sharing) yordamida 2 TB hajmni bir nechta a'zolar bilan baham ko'rish imkoniyati katta afzalliklardan biridir. iCloud nafaqat shaxsiy foydalanish uchun, balki ta'lim sohasida ham foydali bo'lishi mumkin. Masalan, iPad'lardan foydalanuvchi o'qituvchilar dars materiallarini, testlarni yoki loyiha fayllarini iCloud Drive orqali o'quvchilar bilan osongina baham ko'rishadi. Har bir o'quvchi iCloud'dagi umumiy papkaga kirib, kerakli materialni ko'chirib olish yoki bevosita tahrir qilish imkoniga ega bo'ladi. Bu raqamli ta'lim jarayonlarini ancha soddalashtiradi.

iCloud Drive esa foydalanuvchilarga turli formatdagi fayllarni saqlash, ularni boshqa ilovalar bilan integratsiya qilish, va havola orqali boshqalar bilan bo‘lishish imkoniyatini beradi. Masalan, Pages, Numbers, Keynote hujjatlari iCloud orqali bir vaqtning o‘zida bir necha foydalanuvchi tomonidan tahrir qilinishi mumkin. Bu ayniqsa jamoaviy loyihalarda, tadqiqot ishlarida yoki talabalarning guruh ishlari davomida juda qulay. iCloud xizmatining yana bir foydali funksiyasi bu “Find My” deb nomlangan imkoniyatdir. Ushbu xizmat qurilma yo‘qolgan taqdirda uni topish, masofadan bloklash yoki ichidagi ma’lumotlarni o‘chirish imkonini beradi. Bu esa ma’lumotlarni yo‘qotish xavfini sezilarli darajada kamaytiradi va foydalanuvchiga ruhiy xotirjamlik bag‘ishlaydi.

Xulosa qilib aytganda, iCloud - bu zamonaviy raqamli hayot uchun zarur vosita bo‘lib, nafaqat fayllarni saqlaydi, balki foydalanuvchi tajribasini to‘liq boshqarish, sinxronlashtirish va himoya qilish imkonini beradi. iCloudning chuqur integratsiyasi, xavfsizlik darajasi, avtomatik backup tizimi va boshqa ko‘plab qulayliklari uni zamonaviy bulutli xizmatlar orasida eng ishonchli va dolzarb yechimlardan biriga aylantiradi.



1.2.5-rasm. iCloud asosiy oynasi

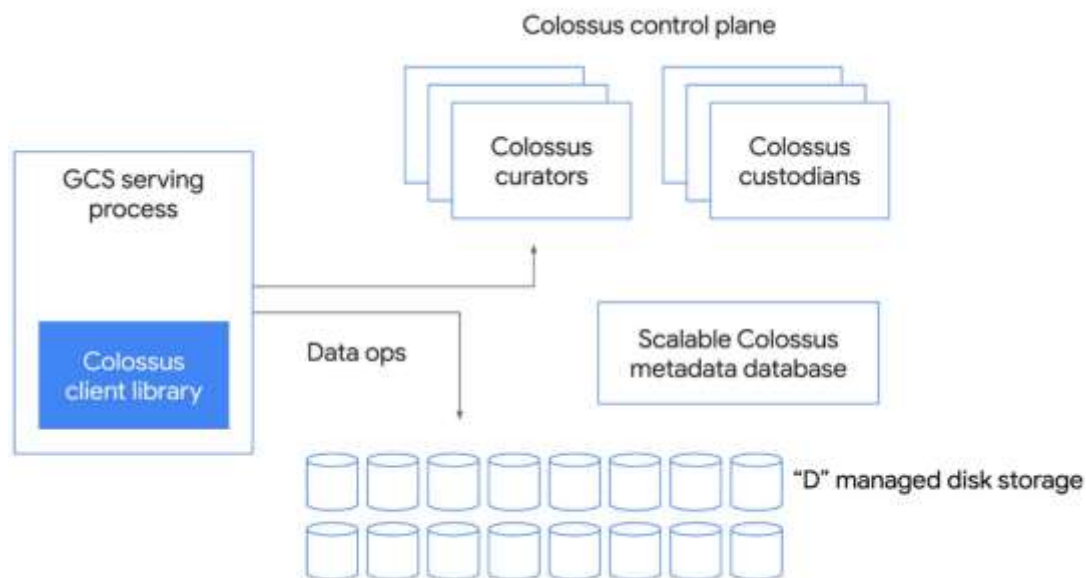
1.3 Avtomatik zaxiralovchi dasturlarni ishlab chiqishda foydalaniladigan texnologiyalar.

Bugungi kunda har qanday sohada, xoh bu ta'lim, biznes yoki sog'liqni saqlash bo'lsin, raqamli ma'lumotlar juda katta ahamiyatga ega. Ma'lumotlar bilan ishlayotgan har bir tashkilot ularni saqlash, himoyalash va kerak bo'lganda tiklash imkoniyatiga ega bo'lishi kerak. Aynan shu maqsadda avtomatik zaxiralovchi dasturlar ishlab chiqiladi. Ushbu dasturlar foydalanuvchining aralashuvisiz, ma'lumotlarni doimiy ravishda zaxiralab boradi. Ularni yaratishda esa zamonaviy texnologiyalar muhim rol o'ynaydi. Masalan, hammamizga tanish bo'lgan Google Drive xizmatini olaylik.

Google Drive bu oddiy fayl saqlash vositasi emas. U zamonaviy texnologiyalarning uzviy uyg'unligi orqali ishlaydigan, murakkab arxitekturaga ega yirik tizimdir. Bu tizimning yuragi sifatida Google Cloud Platform (GCP) xizmat qiladi. Aynan GCP bazasida joylashgan hisoblash va saqlash serverlari millionlab foydalanuvchi fayllarini real vaqt rejimida qabul qiladi, saqlaydi, sinxronlashtiradi va xavfsizligini ta'minlaydi.

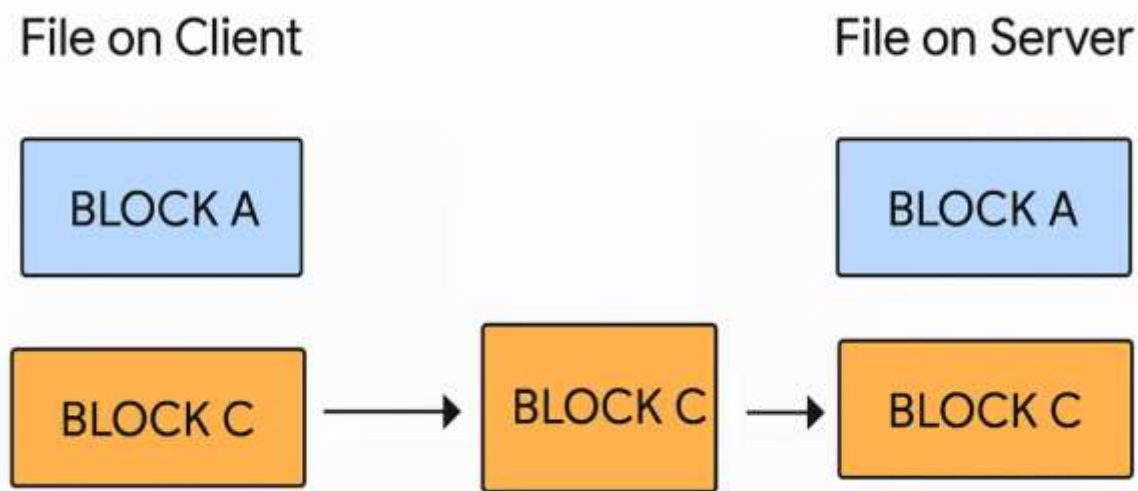
Google Cloud Platform (GCP) – bu Google kompaniyasi tomonidan yaratilgan va dunyo bo'yicha keng qo'llanilayotgan bulutli hisoblash xizmatlari to'plamidir. GCP ning asosiy ustunliklaridan biri – bu yuqori darajadagi ishonchlilik, xavfsizlik va masshtablilikdir. Google o'zining kuchli server infratuzilmasi va ma'lumotlar markazlari orqali foydalanuvchilarga global darajada tezkor xizmatlar taqdim etadi.

Saqlash masalasiga kelganda, Google Drive o'zining maxsus fayl tizimi Colossusdan foydalanadi. Bu tizim oddiy fayllarni millionlab kichik bloklarga bo'lib, ularni geografik jihatdan turli joylarda joylashgan serverlar bo'ylab tarqatadi. Bu yondashuv ma'lumotlarni tezkor yetkazish va nosozliklarga bardoshli bo'lishiga xizmat qiladi.



1.3.1-rasm. Colossus tizimining ichki arxitekturası

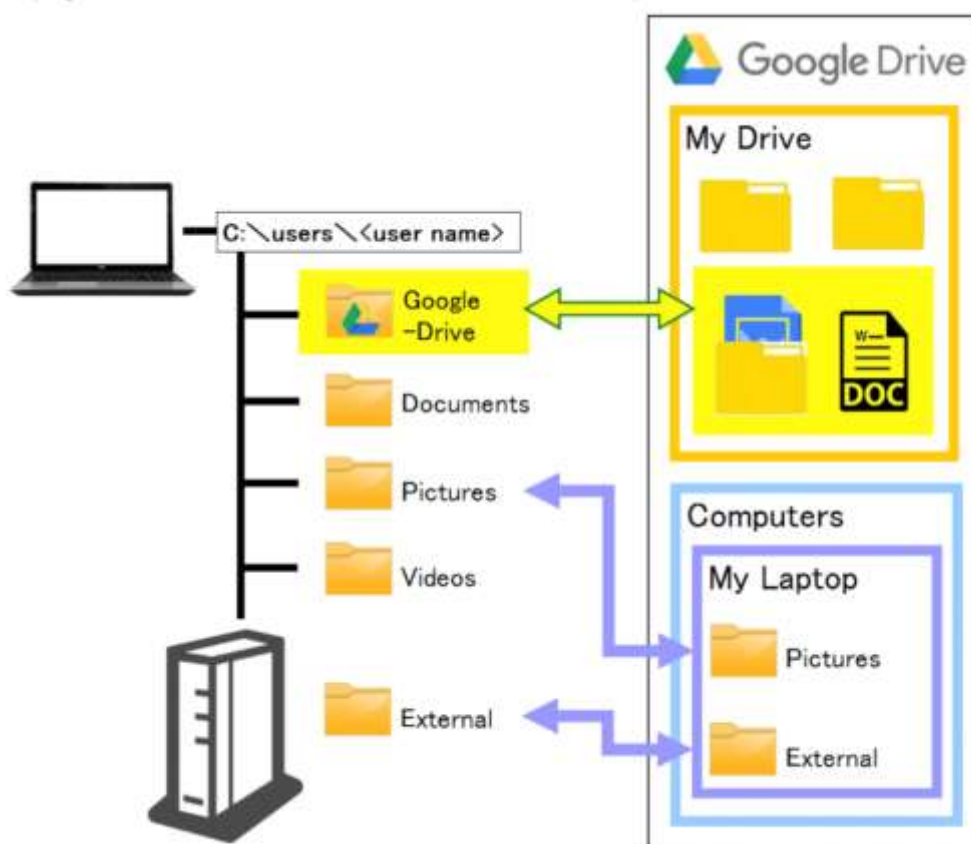
Google Drive ning sinxronizatsiya tizimida ajoyib mexanizm ishlaydi. Siz kompyuteringizda yoki telefonda faylga o'zgartirish kiritsangiz, u darhol Google serverlariga uzatiladi va boshqa qurilmalarda yangilanadi. Buning uchun delta sync deb ataladigan optimallashtirilgan texnologiyadan foydalaniladi. Bu vosita faqat o'zgargan qismini zaxiraga oladi, butun faylni emas. Bu esa internet trafigini kamaytiradi va sinxronlash tezligini oshiradi.



1.3.2-rasm. Delta Sync ishlash tartibi

Google xavfsizlik borasida ham ancha oldinda. Har bir fayl end-to-end shifrlash asosida kodlanadi. Fayl avval foydalanuvchi qurilmasida shifrlanadi, so'ng serverga uzatiladi va u yerda yana qo'shimcha xavfsizlik qatlamlari bilan himoyalanadi. Bu yondashuv ma'lumotlarga ruxsatsiz kirishning oldini oladi, hattoki Google muhandislari ham foydalanuvchi fayliga kira olmaydi.

Yana bir diqqatga sazovor jihat - Google Drive'dagi AI (sun'iy intellekt) va ML (mashinali o'rganish) texnologiyalari. Bu texnologiyalar foydalanuvchi odatlarini kuzatib, eng ko'p foydalaniladigan fayllarni tezroq topishga yordam beradi. Masalan, siz biror faylni har dushanba kuni ochsangiz, tizim uni oldindan keshlab qo'yadi va siz uchun birinchi navbatda chiqaradi. Yoki siz faylingiz ichida nimadir izlayotganingizda, Drive siz o'ylayotgan matnni sun'iy intellekt orqali tahmin qiladi va izlanmalarni yanada aniq va tez qiladi.



1.3.3-rasm. Google Drive sinxronizatsiyasi ko'rinishi

Shuningdek Google Drive'ning eng katta afzalliklaridan biri Google Docs integratsiyasi hisoblanadi. Fayllarni Drive ichida to'g'ridan-to'g'ri ochish, tahrirlash va boshqa bilan real vaqtda hamkorlik qilish imkoniyati bu xizmatni raqobatchilardan ajratib turadi. Siz hujjatni yuklab olib, qayta yuklashga hojat qoldirmaysiz barcha jarayonlar Drive interfeysi orqali amalga oshadi [2].

iCloud – bu oddiy fayl saqlovchi vosita emas, u Apple ekotizimining yuragi bo'lib, har bir foydalanuvchining qurilmasini kuzatmasdan, sun'iy intellekt darajasida boshqaradigan zaxiralash tizimidir. Apple bu xizmat orqali foydalanuvchining har bir suratini, faylini, chatini va sozlamasini fon rejimida, sezilmas darajada muhofaza qiladi. Bularning ortida esa juda chuqur muvofiqlashtirilgan texnologik arxitektura yotadi.

iCloud o'zining asosiy infratuzilmasini Apple Cloud Infrastructure nomi ostida yuritadi, lekin bu butunlay o'zining serverlariga asoslanmagan. Aslida Apple ma'lumotlarni parchalab, bir qismini Amazon, bir qismini Google Cloud, yana bir qismini Microsoft Azure kabi yirik provayderlarga tarqatadi. Ammo bularning hech biri to'liq faylga ega bo'lmaydi, chunki har bir parcha shifrlangan va alohida kalitlarda yashiringan bo'ladi. Bu esa xavfsizlikni nafaqat texnik, balki strategik darajada ta'minlaydi. Har safar foydalanuvchi iPhone'dagi faylni yangilasa yoki biror rasmga tushsa, u to'liq emas, balki faqat o'zgargan qismi zaxiraga olinadi. Bu incremental backup texnologiyasi deb yuritiladi va aynan mana shu yechim tufayli iCloud zaxiralari tez, qulay va internetni tejovchi ko'rinishda ishlaydi.

Ma'lumotlar esa end-to-end shifrlash asosida himoyalanaadi. Bu shuni anglatadiki, ularni shifrlash jarayoni foydalanuvchining qurilmasida boshlanadi va faqat uning ruxsati bilan ochiladi, hatto Apple muhandislari ham bunga kira olmaydi. iCloud da nafaqat saqlash, balki sinxronizatsiya ham sun'iy intellekt orqali boshqariladi. Foydalanuvchi odatlarini o'rgangan tizim aynan kerakli fayllarni oldindan yuklab tayyorlab qo'yadi, kuchli internet bo'lmasa, yuklashni kechiktiradi, foydalanuvchining qurilma holatiga qarab zaxiralash jadvalini moslashtiradi. Bularning

barchasini tizim fon rejimida, hatto ekranning o'zi yoniq bo'lmagan paytda ham bajaradi. Sinxronizatsiya faqat yangi fayllar uchun emas, balki mavjud fayllarning versiyalari uchun ham qo'llaniladi. Foydalanuvchi istalgan paytda faylning avvalgi ko'rinishiga qaytishi mumkin, bu esa iCloud Drive'da versiyalash va tiklash texnologiyasi orqali amalga oshiriladi [17].

Yana bir muhim jihat data deduplication. Bu orqali tizim bir xil fayllarni ikki marta saqlamaydi. Faqat bitta nusxasini ushlab turadi. Lekin kerakli joylarda uni ko'rsataveradi. Bu ham joyni tejaydi, ham ishlash tezligini oshiradi. Apple shuningdek, CloudKit texnologiyasi orqali ishlab chiquvchilarga o'z ilovalarini iCloud bilan bog'lash imkonini beradi. Masalan, Notes ilovasida yozgan matningiz avtomatik ravishda iCloud da saqlanadi, uni ochish, o'zgartirish, boshqa qurilmada davom ettirish uchun hech qanday maxsus harakatga hojat yo'q. Hamma narsa avtomatik. Foydalanuvchining xotirjamligini ta'minlash uchun esa iCloud Backup funksiyasi mavjud bo'lib, telefon tunda quvvatga ulanganida va Wi-Fi mavjud bo'lsa, butun tizim sozlamalari, ilovalar, chatlar va fayllarni fon rejimida to'liq zaxiraga oladi. Bu zaxira foydalanuvchi yangilangan qurilmada oldingi telefonini tiklash imkonini beradi. Hatto ekrandagi ilovalarning joylashuvi ham o'z holicha qaytadi.

iCloud+ obunachilari esa Private Relay nomli xizmat orqali internetda shaxsiylikni yanada kuchaytirgan holda harakat qilishadi. Bu texnologiya foydalanuvchi IP-manzilini, tashrif buyurgan saytlari va joylashuvini yashiradi – va buni Apple ning o'zi ham kuzata olmaydi. Barcha bu texnologiyalar uyg'unlikda ishlaydi, foydalanuvchining e'tiborini tortmagan holda hayotining eng muhim ma'lumotlarini ko'zga ko'rinmas, ammo ishonchli tarzda asraydi. iCloud – bu texnologiyalarning tinimsiz uyg'unligidan yaralgan sokin lekin kuchli yordamchi, siz esa uni sezmagan holda kun bo'yi unga tayanib yashaysiz.

1.4 I-bob bo'yicha xulosa

Bugungi kunda foydalanuvchilar, tashkilotlar va korxonalar tobora ko'proq raqamli ma'lumotlarga tayanib ish yuritmoqdalar. Ushbu axborot resurslari odatda qimmatli hujjatlar, loyiha fayllari, foydalanuvchi profillari, moliyaviy hisob-kitoblar, media fayllar yoki dasturiy kodlardan iborat bo'ladi. Ana shunday muhim ma'lumotlarning tasodifan yo'qolib ketishi texnik nosozliklar, inson xatolari, zararli dasturlar yoki tashqi omillar natijasida ulkan yo'qotishlarga olib kelishi mumkin. Aynan shu sababli, avtomatik zaxiralash tizimlariga ehtiyoj bugungi kun texnologik rivojlanish sharoitida nihoyatda dolzarb bo'lib bormoqda.

Tahlil qilingan tizimlar (Dropbox, Google Drive, Bacula, Duplicati, Acronis True Image, UrBackup va boshqalar) har xil infratuzilmalar, foydalanuvchi darajalari va ehtiyojlarga mos turli xildagi yechimlarni taqdim etadi. Tijorat dasturlari ko'proq intuitiv interfeyslar, bulutli saqlash imkoniyatlari va kengaytirilgan xizmatlarni taklif qilsa, ochiq manba vositalar foydalanuvchiga moslashtirilgan konfiguratsiya, nazorat va bepul foydalanish imkonini beradi. Har bir tizim o'zining afzallik va cheklovlariga ega bo'lsa-da, ularning barchasida kuzatiladigan asosiy umumiy xususiyatlar quyidagilardir: avtomatik nusxa olish, versiyalash, xavfsiz uzatish (shifrlash), ma'lumotlarni tiklash va interfeys orqali nazorat qilish.

Bundan tashqari, amalga oshirilgan tahlil shuni ko'rsatdiki, ko'plab mavjud zaxiralash tizimlari foydalanuvchidan muayyan texnik bilim va sozlash jarayonlarini talab qiladi. Bu holat oddiy foydalanuvchilar uchun ma'lum darajada noqulaylik tug'diradi. Shu nuqtayi nazardan qaralganda, interfeysi qulay, sozlashsiz avtomatik ishlaydigan va fon rejimida faoliyat yurituvchi zaxiralash vositalariga talab ortib bormoqda. Aynan shu yondashuv AutoBackup dasturini loyihalashda asosiy mezonlardan biri sifatida belgilandi.

Ko'plab tizimlar faqat ma'lumotni zaxiralashga qaratilgan bo'lib, versiyalash yoki tiklash imkoniyatlari yetarlicha quvvatli emas. AutoBackup esa, foydalanuvchi

ma'lumotining o'zgarish tarixini yuritadi, har bir versiyani alohida saqlaydi va ularni kerakli holatda istalgan vaqtda tiklash imkonini beradi. Bu yondashuv nafaqat ma'lumot xavfsizligini, balki foydalanuvchining ish samaradorligini ham oshiradi.

Ma'lumotlar o'zgarishini avtomatik tarzda zaxiralovchi tizimlar nafaqat shaxsiy kompyuter foydalanuvchilari uchun, balki kichik va o'rta biznes subyektlari, ta'lim muassasalari va IT sohasida faoliyat yurituvchi mutaxassislar uchun ham zaruriy vositaga aylanmoqda. Ushbu dasturiy echimlar inson xatosi, kutilmagan nosozliklar yoki zararli dasturiy tahdidlar ta'sirini kamaytirish va ma'lumotlar bilan ishlashda barqarorlikni ta'minlashda muhim rol o'ynaydi. Shunday qilib, tahlil asosida yaratilgan AutoBackup dasturi amaliy jihatdan foydali, texnik tomondan ishonchli va zamonaviy backup tizimlariga mos keluvchi yechim sifatida shakllantirildi.

Tizimlar orasida mavjud farqlar, ularning texnik yechimlari va foydalanuvchi tajribasiga ta'siri AutoBackup kabi yangi loyihani yaratishda muhim tayanch bo'ldi. Ayniqsa, ma'lumotlar o'zgarishini hash algoritmlari orqali aniqlash, faqat o'zgargan fayllarni zaxiralash, faylning avvalgi versiyalarini saqlab borish, va foydalanuvchi aralashuviga muhtoj bo'lmagan rejimda ishlash kabi yondashuvlar real ehtiyojlarga javob beradigan muhim funksiyalardir.

Xulosa sifatida aytish mumkinki, ma'lumotlar o'zgarishini avtomatik zaxiralovchi tizimlar zamonaviy axborot texnologiyalari sohasida ishonch, xavfsizlik va uzluksizlikni ta'minlovchi asosiy komponentga aylangan. Ushbu tizimlar foydalanuvchiga ko'proq vaqtni tejash, ortiqcha xavotirni kamaytirish, ma'lumotlar ustidan doimiy nazorat o'rnatish imkonini beradi. Shu bois, ularning texnologik tahlili asosida ishlab chiqilgan AutoBackup dasturi nafaqat dolzarb muammoning amaliy yechimi sifatida, balki foydalanuvchi ehtiyojiga mos, zamonaviy, ishonchli va kengaytiriladigan dasturiy mahsulot sifatida o'zining texnik ustunliklariga ega bo'lishi mumkin.

II-BOB. “AutoBackup” dasturini ishlab chiqish bosqichlari

2.1 Dastur loyihasini ishlab chiqish

AutoBackup dastur loyihasini ishlab chiqish jarayonida dastlabki va eng muhim bosqichlardan biri bu talablarni aniqlash (requirements analysis) bosqichi hisoblanadi. Aynan shu bosqichda kelajakda ishlab chiqiladigan dastur qanday muammolarni hal qilishi, foydalanuvchiga nima uchun kerakligi va u qanday shart-sharoitlarda ishlashi lozimligi chuqur o'rganiladi. Har qanday dastur muvaffaqiyatli ishlashi uchun, eng avvalo, uni yaratishga undagan asosiy muammoni to'g'ri tushunish zarur.

Shunday qilib, talablarni aniqlash bosqichida birinchi e'tibor qaratilgan jihat fayllarni avtomatik zaxiralash funksiyasi bo'ldi. Ya'ni foydalanuvchi muhim fayllarini har safar qo'lda boshqa joyga nusxalashga majbur bo'lmasligi kerak. AutoBackup foydalanuvchi tomonidan tanlangan fayllarni kuzatib boradi va ular o'zgargan zahoti avtomatik tarzda serverga yuboradi. Bu orqali kodlar yoki boshqa muhim hujjatlar tasodifan o'chib ketishdan saqlanadi.

Asosiy talablardan yana biri fayllarni versiyalash tizimi bo'ldi. Foydalanuvchi bir xil faylni bir necha marta o'zgartirishi mumkin. Bunda oldingi versiyalar ham saqlanib qolishi kerak. Foydalanuvchi xatolikga yo'l qo'ysa foydalanuvchi ma'lumotni uni osongina tiklash imkonini beradi. Bu bosqichda foydalanuvchining ish tajribasi, ya'ni interfeysdan qanday foydalanishi, ro'yxatdan o'tish va avtorizatsiya jarayonlari qanday bo'lishi, qayta tiklash funksiyasi qayerda joylashishi va tizimdagi xavfsizlik choralari ham aniq belgilab chiqildi. Ayniqsa, ma'lumotlarni shifrlash, login orqali tizimga kirish va serverda saqlanayotgan fayllarning maxfiyligi kabi talablar asosiy e'tiborda bo'ldi.

Shunday qilib, talablarni aniqlash bosqichi faqat texnik shartlar emas, balki real hayotdagi ehtiyoj va tajribaga asoslangan holda shakllantirildi. Bu esa AutoBackup dasturini foydalanuvchining amaliy muammosini hal qilishga qaratilgan, haqiqiy yechimga aylantirishga xizmat qildi. Shu hodisa ya'ni kodlarimning Git ga

yuklanmagan holda yo‘qolib ketgani meni nafaqat bu dasturga chuqur e’tibor berishga, balki uning imkoniyatlarini kengaytirishga, xatoliklar uchun real himoya qatlamlari qo‘shishga undadi.

AutoBackup dastur loyihasini ishlab chiqish jarayonida amalga oshirilgan jarayonlardan yana biri bu texnik tahlil hisoblanadi. Ushbu bosqichda mavjud bo‘lgan zaxiralash tizimlari o‘rganildi. Ushbu tizimlarning afzalliklari va kamchiliklari tahlil qilinib, loyiha uchun texnologik yechimlar va arxitekturasi aniqlashtirildi. Texnik tahlilda asosan uch jihatga e’tibor berildi. Bular funksional imkoniyatlar, xavfsizlik va barqarorlikdir.

Avvalo Dropbox, Google Drive, iCloud va OneDrive kabi mashhur zaxiralash xizmatlarining texnologik tahlili amalga oshirildi. Ushbu xizmatlar foydalanuvchiga fayllarni onlayn tarzda saqlash, avtomatik sinxronizatsiya qilish, versiyalarni nazorat qilish imkonini beradi. Foydalanuvchilar kerakli holatda faylni ushbu versiya holatiga qaytarish imkoniyatiga ega bo‘lishadi. Ushbu tizimlarning foydalanuvchi interfeysi soddaligi va bulutli arxitektura orqali yuqori darajada xavfsizlikni ta’minlashi AutoBackup uchun ham asosiy hususiyatlaridan biri bo‘ldi. Ayniqsa, Dropbox ning fayl o‘zgarishlarini real vaqt rejimida kuzatishi iCloud ning fayl versiyalarini optimallashtirishi va Google Drive ning shifrlash orqali himoyalangan uzatish mexanizmi tahlil qilinib ularning foydali tamoyillari AutoBackup ga moslashtirildi.

Texnik tomondan qaralganda, AutoBackup arxitekturasi JavaFX va Spring Boot texnologiyalariga tayangan. JavaFX foydalanuvchi interfeysini chiroyli va oson foydalaniladigan qilish uchun tanlandi. Spring Boot esa backend tizimini barqaror, kengaytiriladigan va mikroxizmatlarga tayyor holatda yaratishga imkon berdi. Ushbu tanlov ayniqsa autentifikatsiya jarayonida o‘zini oqladi Keycloak orqali foydalanuvchi login va parollari himoyalaniishi tokenlar orqali backend bilan xavfsiz aloqa o‘rnatilishga yordam berdi.

Texnik tahlilda yana bir muhim jihat sifatida ma'lumotlar bazasini keltirsak bo'ladi. PostgreSQL kuchli, ishonchli va katta hajmdagi ma'lumotlar bilan ishlay oladigan ochiq manbali bazalardan biri hisoblanadi. Shu sabablar PostgreSQL ni AutoBackup dasturida qo'llashga sabablar bo'ldi.

Shuningdek, dastur doimiy monitoring asosida ishlashi, ya'ni tanlangan fayl yoki papkalardagi o'zgarishlarni belgilangan intervalda kuzatib borishi kerak edi. Buning uchun maxsus Thread mexanizmlari yordamida fon jarayoni qo'llanildi. Bu yondashuv "voqeaga asoslangan" (event-based) emas, balki "vaqtga asoslangan" (time-based) holda ishlaydi. Dastur belgilangan vaqtda faylni hash algoritm orqali tekshiradi va avvalgi versiya bilan solishtiradi. Agar o'zgarish aniqlansa, zaxira olinadi. Bu orqali tarmoqqa ortiqcha yuklama va serverda xotirani isrof bo'lishi oldi olinadi.

Texnik tahlil davomida yana bir e'tibor qaratish kerak bo'lgan jihat bu shifrlash mexanizmi bo'ldi. Zamonaviy zaxiralash tizimlari AES-256 kabi algoritmlardan foydalanib, foydalanuvchining fayllarini serverga yuborishdan oldin shifrlaydi. AutoBackup loyihasida ham foydalanuvchi fayllari jo'natilishidan oldin shifrlanadi, shunda ma'lumotlar hatto serverda saqlanayotgan bo'lsa ham maxfiylikicha qoladi.

Xulosa qilganda, AutoBackup loyihasining texnik tahlil bosqichi mavjud zaxiralash tizimlarining eng kuchli jihatlarini o'rganish, ularni mahalliy ehtiyojlarga moslashtirish va foydalanuvchiga xavfsiz, avtomatik va ishonchli backup tizimini taqdim etishga yo'naltirildi. Bu esa loyiha sifatini oshirib, uni real hayotda foydali tizimga aylantirishga xizmat qildi.

2.2 Dasturning ma'lumotlar bazasini loyihalash

AutoBackup dasturining ma'lumotlar bazasini loyihalash jarayonida talablarni to'plash va tahlil qilish bosqichi juda muhim rol o'ynadi. Chunki ushbu bosqichda foydalanuvchilarning ehtiyojlari, tizimning funksional imkoniyatlari va ma'lumotlar bilan ishlash logikasi chuqur o'rganildi. Dastur zaxiralash jarayonida foydalanuvchi fayllarining versiyalarini saqlashi, har bir foydalanuvchining o'ziga xos fayl tuzilmasi

bo'lishi, shuningdek, xavfsizlik va tezkorlik talablariga javob berishi lozim edi. Shu sababli, ma'lumotlar bazasini to'g'ri tuzish va uni optimal ishlashga moslash loyiha muvaffaqiyatining asosiy omillaridan biri bo'lib xizmat qildi.

Talablarni to'plash jarayonida eng avvalo, foydalanuvchi kim bo'lishi, u qanday ma'lumotlar bilan ishlashi va zaxiralash jarayonida qanday turdagi ma'lumotlar bazaga yozilishi kerakligi aniqlashtirildi. Masalan, foydalanuvchi hisobiga oid ma'lumotlar: foydalanuvchi nomi, parol (yoki token orqali autentifikatsiya), roli va akkaunt holati kabi atributlar kiritilishi zarur deb topildi. Bundan tashqari, har bir foydalanuvchi o'zining fayllarini serverga yuboradi demak, bu fayllar haqida ham maxsus tuzilmalar kerak bo'ladi: fayl nomi, hajmi, saqlangan sana, versiya raqami, va joylashuvi kabi atributlar ajratilishi kerak edi.

Tahlil natijasida aniqlangan eng muhim obyektlardan biri fayl versiyasi bo'ldi. AutoBackup tizimi har safar bir faylda o'zgarish aniqlansa, uni yangidan saqlaydi, lekin avvalgi nusxalarini o'chirib yubormaydi. Shu bois, file_versions nomli maxsus jadval yaratilishi zarur deb belgilandi. Unda faylga tegishli har bir versiyaning identifikatori, sha256 hash qiymati, faylga bog'liq bo'lgan zaxiralash vaqti, hamda serverdagi saqlanish yo'li ko'rsatiladi. Bu esa foydalanuvchiga istalgan vaqtda kerakli versiyani tiklash imkonini beradi.

Shuningdek, papkalar bilan ishlash zarurati ham tahlil qilindi. Foydalanuvchi fayllarni faqat alohida fayl sifatida emas, balki butun papkalarni tanlab zaxiraga olish imkoniga ega bo'lishi kerak. Buning uchun folder nomli jadval tashkil qilinib, har bir papkaning identifikatori, ota papka bilan bog'lanishi (ya'ni recursive tuzilma), va egasi ko'rsatiladigan struktura ishlab chiqildi. Bu bilan foydalanuvchi fayllarini topish, tartibga solish va qayta tiklash jarayonlari sezilarli darajada soddalashtirildi [3].

Ma'lumotlar bazasini loyihalashda yana bir muhim masala ma'lumotlar xavfsizligi va foydalanuvchiga tegishli bo'lgan fayllar bilan boshqa foydalanuvchilarning aralashib ketmasligi edi. Shu sababli, barcha asosiy jadvallar

(foydalanuvchilar, fayllar, versiyalar) o‘zaro referenslar bilan bog‘landi va har bir yozuvning kimga tegishli ekanini aniqlovchi user_id atributi orqali bog‘liq bo‘lishi ta’minlandi. Bu orqali har bir foydalanuvchining faqat o‘ziga tegishli ma’lumotlar bilan ishlashi kafolatlandi. Tahlil davomida ishlash tezligini oshirish uchun kerakli ustunlarga indekslar qo‘shilishi, fayl versiyalarini aniqlashda hash solishtirish optimallashtirilishi, va haddan ortiq fayl yuklamasini oldini olish uchun fayl hajmiga cheklov qo‘yilishi kabi texnik talablarga ham alohida e’tibor qaratildi [4].

Umuman olganda, AutoBackup dasturi uchun ma’lumotlar bazasini loyihalash bu faqat texnik struktura emas, balki foydalanuvchi ehtiyojlariga moslashtirilgan, real hayotdagi muammolarni hal qilishga qaratilgan muhim bosqich bo‘ldi. Talablarni chuqur tahlil qilish natijasida tuzilgan ma’lumotlar bazasi, dasturdagi barcha zaxiralash, versiyalash va tiklash jarayonlarining asosiy tayanchi sifatida xizmat qilmoqda.

Dasturdagi jadvallarni ko‘rib chiqamiz:

Field nomi	Ma’lumot turi	Izoh
id	Long	Foydalanuvchi ID (primary key, avtomatik yaratiladi)
username	String	Unikal foydalanuvchi nomi (bo‘sh bo‘lishi mumkin emas)
email	String	Unikal elektron pochta manzili (bo‘sh bo‘lishi mumkin emas)
password	String	Foydalanuvchining paroli (bo‘sh bo‘lishi mumkin emas)

createdAt	LocalDateTime	Hisob yaratilgan sana va vaqt
lastLoginAt	LocalDateTime	So‘nggi tizimga kirish vaqti

2.2.1-jadval “AvtoBackup” dasturining User jadvali.

Field nomi	Ma’lumot turi	Izoh
id	Long	Zaxira faylning yagona identifikatori (avtomatik yaratiladi)
user	User (obyekt)	Fayl egasini ifodalaydi, User jadvalidagi id ga tashqi kalit
fileName	String	Fayl nomi
filePath	String	Kompyuterdagi fayl manzili (ko‘rsatilgan yo‘l)
backupFrequency	Integer	Zaxiralash chastotasi (daqikalarda)
versionCount	Integer	Faylning saqlangan versiyalar soni (default: 0)
lastBackupTime	LocalDateTime	So‘nggi marta zaxira olingan vaqt
status	String	Holati: ACTIVE, PAUSED, STOPPED (default: ACTIVE)
createdAt	LocalDateTime	Fayl zaxiralash yozuvi yaratilgan vaqt

updatedAt	LocalDateTime	Oxirgi marta yangilangan vaqt
nextBackupTime	LocalDateTime	Keyingi rejalashtirilgan zaxiralash vaqti
realFilePath	String	Faylning haqiqiy joylashuvi (real manzil – OS darajasida)
fileHash	String	Faylning hash qiymati (o‘zgarishlarni aniqlash uchun)

2.2.2-jadval “AvtoBackup” dasturining BackupFile jadvali.

BackupFile jadvalda foydalanuvchilarning fayllari haqidagi ma’lumotlar va User jadvaliga reference saqlaydi.

Field nomi	Ma’lumot turi	Izoh
id	Long	Har bir versiyaga unikal identifikator (avtomatik yaratiladi)
backupFile	BackupFile	Qaysi faylga tegishli ekanligini bildiradi (tashqi kalit, file_id)
versionNumber	Integer	Faylning versiyasini bildiradi
fileUrl	String	Faylning URL manzili
fileSize	Long	Fayl hajmi baytlarda
creationTime	LocalDateTime	Ushbu versiya yaratilgan sana va vaqt

2.2.3-jadval “AvtoBackup” dasturining BackupVersion jadvali.

BackupVersion jadvalda foydalanuvchilarning fayllarning versiyalari haqidagi ma'lumotlar saqlanadi.

Field nomi	Ma'lumot turi	Izoh
id	Long	Unikal identifikator (avtomatik yaratiladi)
userId	Long	User jadvalidagi foydalanuvchining ID raqami (tashqi kalit)
fileId	Long	BackupFile jadvalidagi faylning ID raqami (tashqi kalit)

2.2.4-jadval "AvtoBackup" dasturining UserFiles jadvali.

UserFiles jadvalida foydalanuvchilar va fayllarni o'zaro bog'lovchi reference lar saqlanadi.

Field nomi	Ma'lumot turi	Izoh
id	Long	Unikal identifikator (avtomatik yaratiladi)
userId	Long	User jadvalidagi foydalanuvchining ID raqami (tashqi kalit)
token	Varchar(255)	Parolni tiklash uchun yaratilgan maxfiy token
code	Varchar(255)	Email orqali yuboriladigan tasdiqlovchi kod

email	Varchar(255)	Foydalanuvchining ro'yxatdan o'tgan email manzili
expiry	Timestamp(6)	Tokenning amal qilish muddati
expiry_date	Timestamp(6)	Parolni tiklash tokenining tugash sanasi

2.2.5-jadval “AvtoBackup” dasturining PasswordResetTokens jadvali.

Jadval foydalanuvchilarning parolni tiklash (reset password) jarayonida ishlatiladigan ma'lumotlarni o'zida saqlaydi [5].

2.3 Dasturning server qismini ishlab chiqish

Hozirgi kunda dasturiy tizimlarning samarali ishlashi ko'p jihatdan ularning server qismining to'g'ri tashkil etilganiga bog'liq. Server tomoni foydalanuvchi arizalaridan (klientlardan) keladigan so'rovlarni qabul qilish, ularga ishlov berish, ma'lumotlar bazasi bilan muloqot qilish, xavfsizlikni ta'minlash va boshqa ko'plab muhim funksiyalarni bajaradi. Shu sababli, server qismini ishlab chiqishda zamonaviy, ishonchli va kengaytiriladigan texnologiyalarni tanlash juda muhim hisoblanadi.

Server tomonida ishlatiladigan texnologiyalar bir necha asosiy toifalarga ajratiladi: backend dasturlash tillari va freymvorklar, ma'lumotlar bazasi boshqaruv tizimlari (MBBT), xavfsizlikni ta'minlovchi vositalar, API interfeyslarni yaratish vositalari, foydalanuvchi autentifikatsiyasi va avtorizatsiyasi uchun tizimlar, shuningdek server infratuzilmasi va konteynerlash texnologiyalari.

1. Backend dasturlash tillari va freymvorklar

Server dasturlashida eng keng tarqalgan tillar Java, Python, JavaScript (Node.js), C#, va Go hisoblanadi. Har birining o'ziga xos ustunliklari mavjud. Masalan, Java kuchli ob'ektga yo'naltirilgan dasturlash modeli va barqarorligi sababli katta va kompleks tizimlarda keng qo'llaniladi. Spring Boot kabi freymvorklar Java asosida

mikroservis arxitekturasini soddalashtirib, RESTful API larni tez va ishonchli shakllantirish imkonini beradi. Python esa Django yoki Flask kabi freymvorklar orqali tez ishlab chiqish sikli va o‘qilishi oson kodlar bilan mashhur. Node.js esa bir vaqtning o‘zida ko‘plab so‘rovlarni boshqarish uchun asinxron arxitektura orqali samaradorlikni oshiradi.

2. Ma’lumotlar bazasi boshqaruv tizimlari (MBBT)

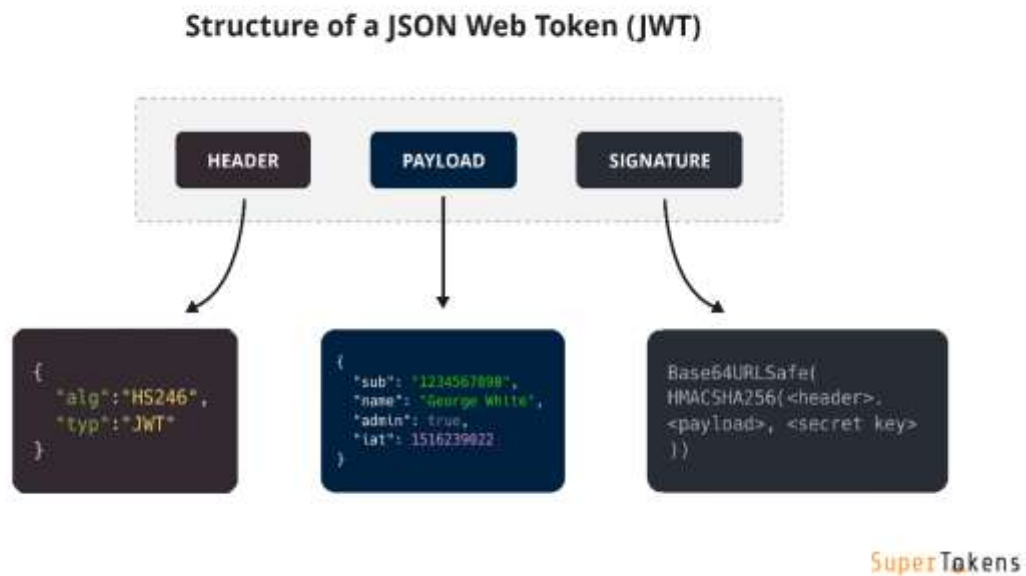
Server qismida asosiy funksiyalardan biri ma’lumotlarni saqlash va ularga tezkor kirishni ta’minlash. Buning uchun PostgreSQL, MySQL, MongoDB, Oracle, yoki Redis kabi tizimlardan foydalaniladi. Relatsion MBBTlar (masalan, PostgreSQL va MySQL) an’anaviy strukturalashtirilgan ma’lumotlar bilan ishlash uchun mos bo‘lsa, MongoDB kabi NoSQL yechimlari dinamik va moslashuvchan struktura talab qiladigan tizimlarda qo‘llaniladi.

3. Xavfsizlik texnologiyalari

Ma’lumotlar xavfsizligini ta’minlash har qanday server dasturining ajralmas bo‘lagi. HTTPS protokoli orqali shifrlangan ma’lumot uzatish, JWT (JSON Web Token) asosida tokenli autentifikatsiya, OAuth2 va OpenID Connect protokollari orqali foydalanuvchini tekshirish amaliyotda keng tarqalgan. Bundan tashqari, parollarni bcrypt, argon2 yoki PBKDF2 kabi algoritmlar bilan xashlab saqlash usuli server xavfsizligini oshiradi.

JWT ning tarkibi asosiy 3 qismdan iborat. Header (sarlavha), Payload va Signature dan tashkil topgan.

1. Header - Token qanday algoritm bilan imzolanganini bildiradi (masalan, HMAC yoki RSA).
2. Foydalanuvchiga oid ma’lumotlar va tokenning mazmunini o‘z ichiga oladi. Bu qismda claims bo‘ladi.
3. Token buzilmaganini tekshirish uchun ishlatiladi [6].



2.3.2-jadval. JWT token tarkibiy tuzilishi.

4. API interfeyslarini yaratish

Foydalanuvchi bilan server oʻrtasidagi aloqa REST yoki GraphQL API lari orqali amalga oshiriladi. RESTful API lar odatda HTTP soʻrovlariga asoslangan boʻlib, CRUD (Create, Read, Update, Delete) operatsiyalarini bajarishda keng qoʻllaniladi. GraphQL esa murakkab soʻrovlar va dinamik javoblar uchun samarali echim hisoblanadi. Swagger va Postman kabi vositalar bu API larni sinovdan oʻtkazish va hujjatlashtirishda muhim rol oʻynaydi.

5. Avtorizatsiya va autentifikatsiya

Server tizimlarida foydalanuvchilarni aniqlash va ularning huquqlarini nazorat qilish uchun Keycloak, Auth0, Firebase Authentication, yoki oʻzlashtirilgan tokenli tizimlardan foydalaniladi. Ushbu vositalar yordamida foydalanuvchilarni roʻyxatdan oʻtkazish, kirish, huquqlarni boshqarish va sessiyalarni kuzatish osonlashadi.

6. Kesh va ishlashni tezlashtirish

Serverlar ish faoliyatini optimallashtirish uchun kesh mexanizmlari keng qoʻllaniladi. Redis, Memcached yoki Ehcache kabi texnologiyalar tez-tez soʻraladigan

ma'lumotlarni xotirada saqlash orqali javob tezligini oshiradi. Shuningdek, fayllarni CDN (Content Delivery Network) orqali tarqatish vaqti sezilarli darajada kamayadi. AutoBackup dasturining backend tuzilmasi va funksional imkoniyatlari, foydalanuvchining ma'lumotlarini avtomatik tarzda zaxiralash, ularni versiyalar asosida saqlash va istalgan vaqtda tiklash imkoniyatini beradi. Ushbu tizimning server tomoni ya'ni backend qismi Spring Boot texnologiyasi asosida yaratilgan bo'lib, u yuqori samaradorlik, modulga asoslanganlik va kengaytirilish imkoniyatlari bilan ajralib turadi.

Backend tizimining eng muhim vazifasi - foydalanuvchi tomonidan yuboriladigan fayllarni qabul qilish, ularni serverda xavfsiz saqlash, versiyalarini nazorat qilish, va kerak bo'lsa foydalanuvchiga tiklab berish imkonini yaratishdan iborat. Bu jarayon oddiy ko'ringani bilan, ichki mexanizmlari ancha murakkab hisoblanadi. Masalan, foydalanuvchi AutoBackup ilovasida faylni tanlaganidan so'ng, ushbu faylga oid metadata (faylning kerakli ma'lumotlari) RESTful API orqali backendga yuboriladi. Tizim bu metadata orqali faylning serverda mavjudligini tekshiradi. Agar bu fayl yangilik kiritilgan holatda bo'lsa, uni yangi versiya sifatida qabul qiladi, aks holda fayl yuklanmasdan turib "avvaldan mavjud" deb belgilab qo'yiladi [7].

Backendda ishlovchi versiyalash har bir yuborilgan fayl uchun SHA-256 hash qiymati hisoblab chiqiladi va avvalgi versiyalar bilan solishtiriladi. Agar hash qiymati farq qilsa, bu faylga o'zgartirish kiritilgan deb baholanadi va tizim avtomatik tarzda yangi versiyani yaratadi. Versiyalar har bir fayl uchun alohida identifikator orqali bog'langan bo'lib, ularning saqlanish yo'li, yaratilgan vaqti va hajmi kabi ma'lumotlar PostgreSQL ma'lumotlar bazasida saqlanadi. Shu tarzda, foydalanuvchi istalgan vaqtda eski versiyalarni ko'rib chiqishi, yuklab olishi yoki dastlabki holatga qaytishi mumkin bo'ladi.

Fayllarni o‘zgarganini aniqlashda fayllarning hajlariini taqqoslash orqali amalga oshirishimiz mumkin edi. Lekin bu aniq bo‘lmasigi mumkin. Sababi agar bizda bir faylning hajmi ma’lum bo‘lsa va uni ichidagi ma’lumotni o‘zgartiranimzida bu o‘zgarishdan so‘ng hajmi avvalgi fayl hajmi bilan bir xil bo‘lib qolishi mumkin. Shuning uchun ham bu vaziyatda hash funksiyani ishlatish to‘g‘ri bo‘ladi. Hash funksiya bir xil ma’lumot uchun doim bir xil hash qiymati chiqarib beradi.

Xesh funksiyasi qanday ishlaydi



2.3.3-jadval. Hesh funksiyasi ishlash tartibi

```

import java.io.InputStream;
import java.security.MessageDigest;

public class HashingFiles {
    public static void main(String[] args) throws Exception {
        File file = new File("src/test.txt");
        System.out.println("Hashlangan holatidagi ko'rinishi -> " + calculateSHA256(file));
    }

    public static String calculateSHA256(File file) throws Exception {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        try (InputStream fis = new FileInputStream(file)) {
            byte[] byteArray = new byte[1024];
            int bytesCount;
            while ((bytesCount = fis.read(byteArray)) != -1) {
                digest.update(byteArray, 0, bytesCount);
            }
        }
        byte[] hash = digest.digest();
        return bytesToHex(hash);
    }

    private static String bytesToHex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }
}
  
```

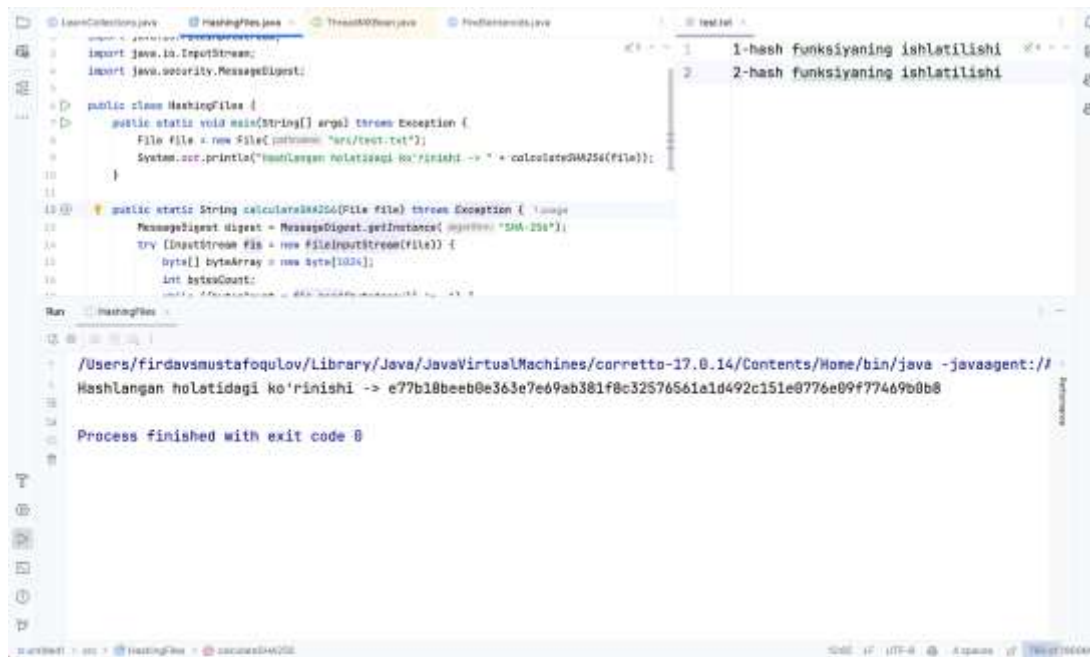
```

/Users/firdavsmustafuqulov/Library/Java/JavaVirtualMachines/corretto-17.0.14/Contents/Home/bin/java -java:
Hashlangan holatidagi ko'rinishi -> 9f04af3d2be0175e9350ea7f78d1c9132cb4d60f0410b85e4ed17433f3b38d8c

Process finished with exit code 0
  
```

2.3.4-rasm. Faylning 1- holatdagi hesh qiymati

Ushbu rasmda ko‘rinadiki fayllarni heshlashda SHA-256 algoritmidan foydalanilmoqda. Sababi SHA-256 ancha ishonchli bo‘lib ushbu algoritmdan generatsiya qilingan hesh qiymati noyob hisoblanadi.



```
import java.io.InputStream;
import java.io.OutputStream;
import java.security.MessageDigest;

public class HashingFiles {
    public static void main(String[] args) throws Exception {
        File file = new File("src/test.txt");
        System.out.println("Hashlangan holatidagi ko'rinishi -> " + calculateSHA256(file));
    }

    public static String calculateSHA256(File file) throws Exception {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        try (InputStream fis = new FileInputStream(file)) {
            byte[] byteArray = new byte[1024];
            int bytesCount;
            while ((bytesCount = fis.read(byteArray)) != -1) {
                digest.update(byteArray, 0, bytesCount);
            }
        }
        byte[] hash = digest.digest();
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }
}
```

Run HashingFiles

```
/Users/firdavsmustafoulov/Library/Java/JavaVirtualMachines/corretto-17.0.14/Contents/Home/bin/java -javaagent:./
Hashlangan holatidagi ko'rinishi -> e77b18beeb0e363e7e69ab381f8c32576561a1d492c151e8776e09f77469b0b8

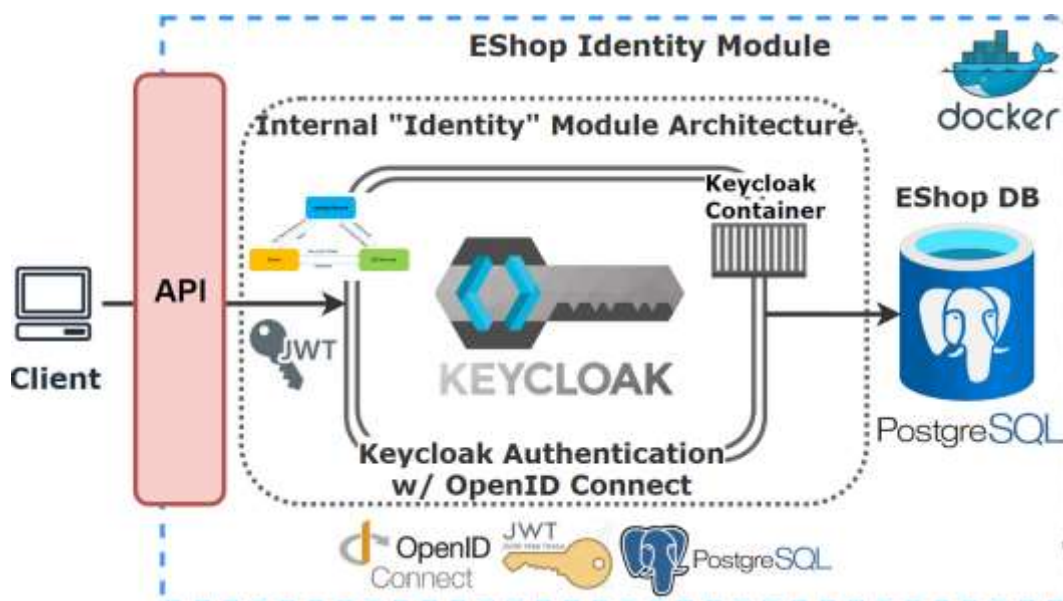
Process finished with exit code 0
```

2.3.5-rasm. Faylning 2- holatdagi hesh qiymati

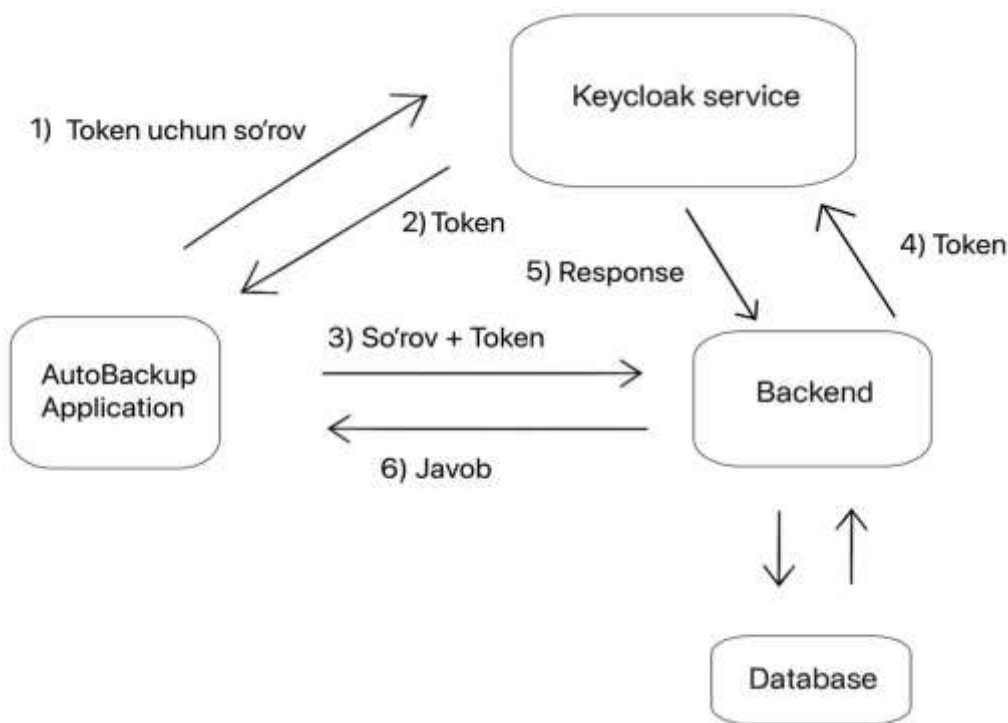
AutoBackup backend tizimida xavfsizlik masalasi ham ko‘rib chiqilgan. Tizimga kirish faqat autentifikatsiyadan o‘tgan foydalanuvchilar orqali amalga oshadi. Bu jarayon Keycloak orqali boshqariladi va OAuth2 protokoliga asoslangan. Foydalanuvchi login va parolini ilovaning desktop application qismiga kiritganidan so‘ng, Keycloak serveri unga maxsus token qaytaradi. Shu token barcha keyingi so‘rovlar bilan birga yuboriladi va backend faqat amal qilayotgan token asosida foydalanuvchiga ruxsat beradi. Bu esa tizimga ruxsatsiz kirishlarning oldini oladi va butun arxitekturaning xavfsizligini ta‘minlaydi.

Keycloakning tanlanishini asosiy sabablaridan biri bu dasturning kengayishini osonlik bilan amalga oshirish uchun. Ya’ni dasturning havfsizlik (security) qismini alohida servis ko‘rinishida chiqarishimiz bizga mikroservis arxitekturasida dastur uchun qo‘shimcha yangi servislarni osonlik bilan qo‘shish imkonini beradi. Bu Keycloak barcha servislarga token generatsiya qilib berishi va shu generatsiya qilingan

tokenlarni tokenni olgan servis boshqa servislarga so'rov jo'natish uchun ishlatadi. Bu servislar ham token haqiqiyligini tekshirish uchun Keycloak ga murojat qiladi. Keycloak esa o'zini tokenini taniydi va servisa bu token ishinchli ekanligi haqida ma'lumot beradi.



2.3.6-rasm. Keycloak ning ishlash tartibi.



2.3.7-rasm. AutoBackup dasturi dasturi ishlash jarayoni tartibi

AutoBackup ilovasi ishga tushgach, foydalanuvchi dasturga o'z login va parolini kiritadi. Dastur birinchi navbatda Keycloak xizmatiga murojaat qiladi va foydalanuvchining kiritgan ma'lumotlari asosida autentifikatsiya qilish uchun token so'raydi. Bu bosqichda foydalanuvchining shaxsini aniqlash maqsadida so'rov yuboriladi. Keycloak serveri foydalanuvchi ma'lumotlarini tekshirganidan so'ng, agar ma'lumotlar to'g'ri bo'lsa, foydalanuvchiga tegishli bo'lgan maxsus JWT tokenni qaytaradi. AutoBackup ilovasi ushbu olgan tokenni o'zining keyingi so'rovlariga qo'shadi va backendga yuboradi. Backend bu tokenni qabul qilib, birinchi navbatda uning haqiqiyligini aniqlash uchun uni Keycloak xizmatiga qayta yuboradi. Keycloak tokenning yaroqliligini tasdiqlab, backendga javob qaytaradi. Agar token haqiqiy va amal qilayotgan bo'lsa, backend foydalanuvchining so'rovini bajaradi, ya'ni zarur ma'lumotlarni bazadan olib, qayta ishlaydi va natijani AutoBackup ilovasiga jo'natadi. Bu jarayon orqali foydalanuvchi tomonidan yuborilgan har bir so'rov tekshirilgan va himoyalangan holda bajariladi. Bu yondashuv dastur xavfsizligini oshirib, ma'lumotlarga faqat ruxsat berilgan foydalanuvchilar kirishini ta'minlaydi. Shuningdek, bu yechim foydalanuvchi secciyalarini boshqarish va auditi qilishda ham katta afzallik beradi [8].

Shuningdek, backend server fayllarni saqlashda oddiy kataloglar tuzilmasidan foydalanadi, lekin bu kataloglar ichki tizim bo'yicha strukturaviy ravishda foydalanuvchi ID lari va fayl identifikatorlari asosida tashkil etilgan. Bunday yondashuv nafaqat ma'lumotlarni tartibli saqlash, balki ularni tezkor topish va tiklash imkoniyatlarini ham soddalashtiradi. Har bir yuklangan fayl backend server tomonidan qabul qilinadi, tekshiriladi va so'ng maxsus yo'nalgan katalogga saqlanadi. Shu bilan birga, uning haqida barcha tafsilotlar ma'lumotlar bazasida aks ettiriladi. Ma'lumotlar bazasida fayllar alohida ma'lumotlar jadvalidagi BackupFile jadvalidagi ID siga mos ravishdagi nomlangan jildga saqlanadi. Misol uchun agar yuklagan faylimizga ma'lumotlar bazasidan 43 raqamli id berilsa uploads nomli faylda 43 raqamli jild

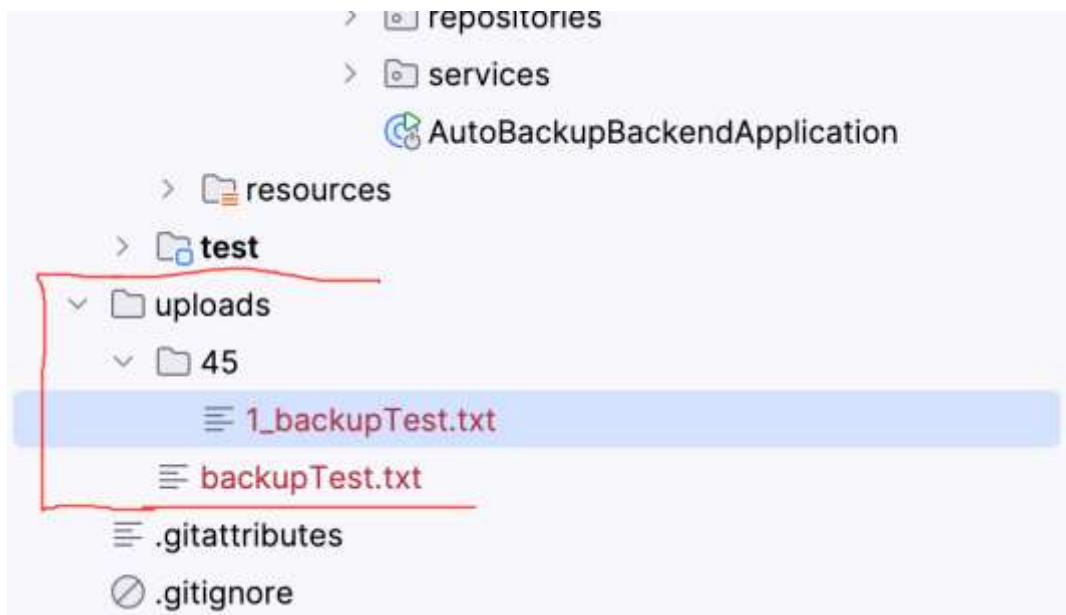
yaratadi va uning ichida saqlay boshlaydi. Bu kabi guruhlash orqali bazadan faylning ID sini oliboq aniq fayl manziliga dastur tezda murojat qilishi mumkin [9].



id	backup_frequency	created_at	file_name	file_path	last_backup_time	status	updated_at
48		2025-06-16 17:55:56.452788	backupTest.txt	uploads/BackupTest.txt	2025-06-16 17:56:42.392173	ACTIVE	2025-06-16 17:56:42

2.3.8-rasm. Fayl haqidagi ma'lumotning MB dagi jadvalda ko'rinishi

Bu rasmdan ko'rishimiz mumkinki fayllarni dastur ma'lumotlar bazasining jadvallarida saqlamaydi. U yerda faqat faylning serverda saqlangan o'rni saqlanadi holos. Bunga sabab jadvallardagi bir katakka saqlanishi mumkin bo'lgan ma'lumotlarning hajmi cheklanganligidir. Ma'lumotlar bazasi, xususan PostgreSQL yoki boshqa RDBMS lar katta hajmli binar fayllarni (masalan, PDF, ZIP, video, rasm fayllar) saqlashda samarali yechim bo'la olmaydi. Jadvalga fayllar yuklanishi bilan uning ishlashi sekinlashadi, zahira olish va tiklash jarayonlari esa ko'p vaqt oladi. Bu yerda fayl tizimlaridan foydalanish to'g'ri bo'lar edi. Zamonaviy fayl tizimlari (NTFS, ext4, ZFS, va boshqalar) aynan katta hajmdagi fayllar ustida amallar bajarish uchun mo'ljallangan. Ular ishga tushirishda kechikishlarsiz katta fayllarni o'qib yuklay oladi. Fayllarni bazada saqlash esa uni ortiqcha yuklama bilan band qiladi. Bu esa har bir jadvaldagi intekslarga (indexing) ga katta tasir ko'rsatadi. Jadvaldagi ID ustuni indekslangan bo'ladi. Har safar yangi ma'lumot qo'shilganda jadvalda ushbu ustun qayta indekslanadi. Og'ir ma'lumot qo'shilishi uni samaradorligini kamaytiradi. Fayl tizimlarini yana bir ustunlik jihati bu ma'lumotlarni oqim (stream) ko'rinishda bera olishidir. Fayllar fayl tizimida saqlanganda ularni xizmatlar orqali to'g'ridan-to'g'ri serverdan oqim (stream) shaklida berish mumkin. Bu esa katta fayllarni yuklab olish yoki ko'rsatishda tarmoqli o'tkazuvchanlikni tejaydi va tez ishlaydi. Baza orqali fayl yuklansa, avval to'liq o'qiladi, so'ng transfer qilinadi, bu esa ko'proq xotira va vaqt talab qiladi [10].



2.3.9-rasm. Zaxiralanadigan fayllarning serverda joylashish ko‘rinishi

Dasturda shuningdek rejalashtirilgan zaxiralash funksiyasi ham mavjud. Bu jarayon backendda belgilangan schedule asosida ishlovchi xizmatlar orqali amalga oshadi va belgilangan vaqtda foydalanuvchining tanlangan fayllari avtomatik tarzda tahlil qilinadi va kerakli bo‘lsa serverga qayta yuboriladi.

Backend tizimi REST API usulida qilingan, ya’ni foydalanuvchi va dastur o‘rtasida aniq ajratilgan interfeys mavjud bo‘lib, har bir amaliyot (fayl yuklash, faylni ko‘rish, versiyalarni olish, tiklash, o‘chirish) uchun maxsus endpointlar mavjud. Bu endpointlar orqali tizimga kiruvchi barcha ma’lumotlar nazorat qilinadi, kerakli tekshiruvlardan o‘tkaziladi va loglar orqali qayd etib boriladi. Log tizimi yordamida tizimda yuz bergan har qanday harakatlar (muvaffaqiyatli yoki xatolik bilan yakunlangan) tahlil qilinadi va kelajakdagi optimallashtirish ishlari uchun asos bo‘lib xizmat qiladi.

Yuqoridagilarning barchasi AutoBackup dasturining backend qismi faqat texnik jihatdan emas, balki foydalanuvchi ehtiyojlari, ma’lumotlar xavfsizligi, resurslardan oqilona foydalanish va tizim barqarorligini ta’minlash nuqtai nazaridan qisman funksional yechim bo‘lishini ko‘rsatadi. Ushbu backend arxitekturasida tizimni

kengaytirish, boshqa xizmatlar bilan integratsiyalash kabi jarayonlarda ham asosiy poydevor sifatida xizmat qilishi mumkin. Bu esa AutoBackup dasturining kelajakda, xavfsiz va foydalanuvchi uchun qulay zaxiralash vositasi sifatida shakllanishiga zamin yaratadi [11].

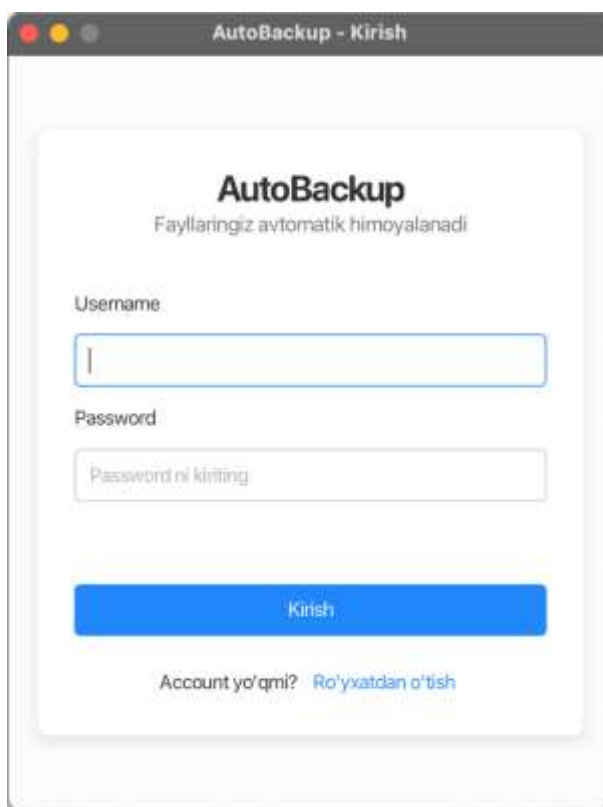
2.4 Dasturning foydalanuvchi interfeysi qismini ishlab chiqish

AutoBackup dasturining foydalanuvchi interfeysi (UI) qismini ishlab chiqish bu tizimdan samarali foydalanish imkonini beruvchi muhim bosqichlardan biri hisoblanadi. Har qanday dasturda foydalanuvchi bilan tizim o'rtasidagi birinchi aloqa bu aynan interfeys orqali amalga oshadi. Shuning uchun AutoBackup dasturini ishlab chiqishda UI qismini ishlab chiqish foydalanuvchi tajribasi va interfeys soddaligi, muhim ahamiyatga ega.

Dastur interfeysi JavaFX texnologiyasi asosida yaratildi. JavaFX bu Java dasturlash tilida GUI (grafik foydalanuvchi interfeysi) yaratish uchun mo'ljallangan kuchli vosita bo'lib, u zamonaviy dizayn va tashqi resurslar bilan integratsiyani qo'llab-quvvatlaydi. Ushbu texnologiya yordamida AutoBackup dasturida yengil, ammo funksional interfeys yaratildi. Interfeys qismini ishlab chiqishda birinchi navbatda foydalanuvchining dasturni ishlatishdagi asosiy yo'nalishlari aniqlab olindi. Dasturda foydalanuvchi quyidagi asosiy amallarni bajarishi kerak: tizimga kirish (login), fayl yoki papkani tanlash, zaxiralash parametrlarini belgilash, zaxiralash holatini kuzatish, va kerak bo'lsa fayllarni tiklash. Har bir amal uchun alohida sahifa (view) va ular o'rtasidagi o'tishlar (scene transition) loyihalandi. Bu orqali foydalanuvchining harakati ketma-ket va mantiqiy bo'lib qoladi.

Bu rasmda AutoBackup dasturining kirish oynasi tasvirlangan. Ushbu oynada foydalanuvchi o'z login ma'lumotlari foydalanuvchi nomi (username) va parol (password) ni kiritib, tizimga kirishi mumkin. Kirish tugmasi bosilgach, foydalanuvchi avtorizatsiya qilinadi va dastur asosiy oynasiga o'tadi. Agar foydalanuvchida hali hisob bo'lmasa, "Ro'yxatdan o'tish" havolasi orqali yangi akkaunt yaratish imkoniyati

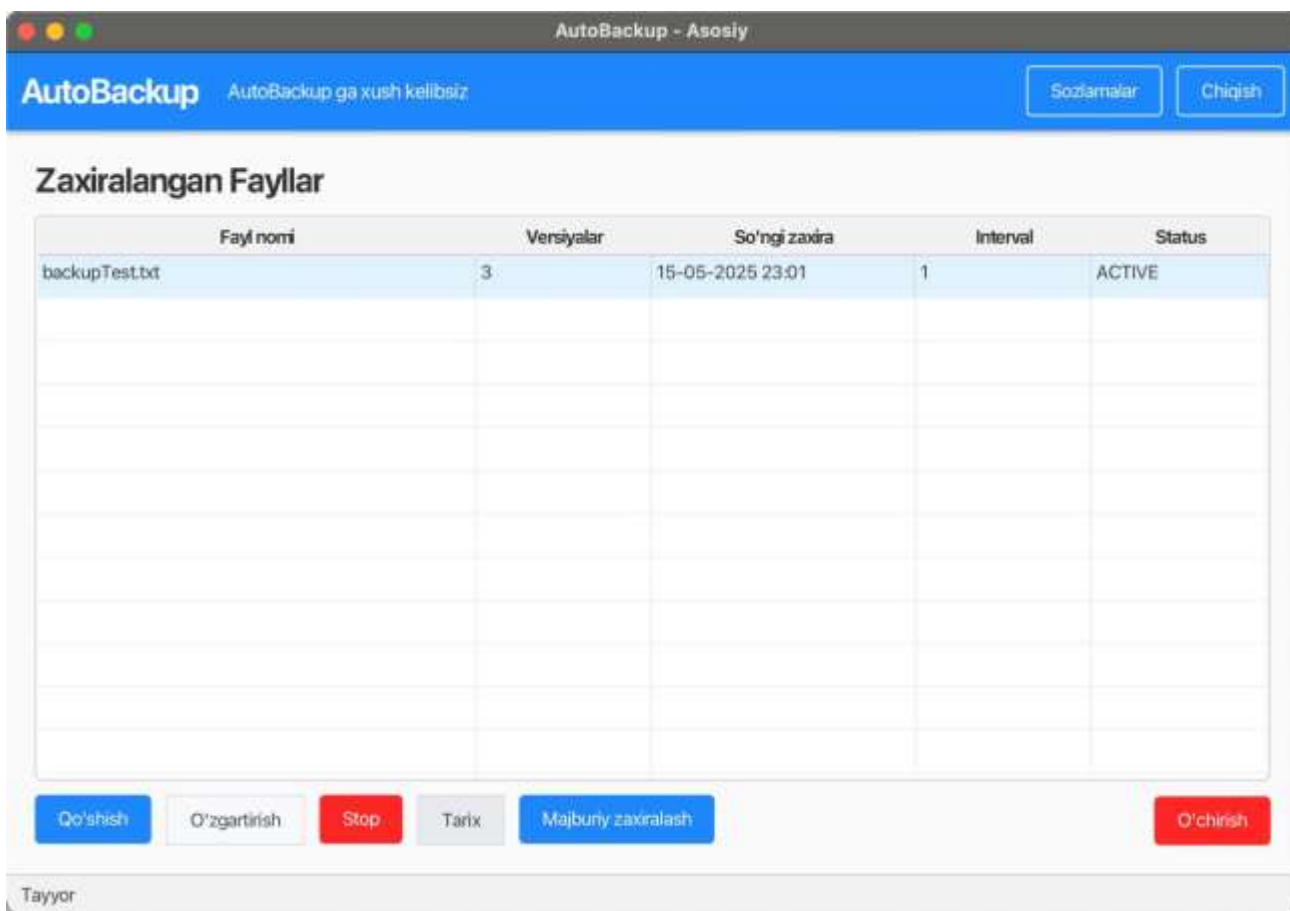
mavjud. Bu oyna oddiy va tushunarli ko‘rinishga ega bo‘lib, dasturdan foydalanishni boshlash uchun qulay kirish nuqtasini ta’minlaydi.



2.4.1-rasm. Foydalanuvchi interfeysining login oynasi

Asosiy sahifada esa oydalanuvchi interfeysining asosiy boshqaruv paneli bo‘lib, u foydalanuvchiga o‘z fayllari ustidan to‘liq nazorat olib borish imkonini beradi. Bu oynada foydalanuvchi hozirgacha zaxiralashga qo‘shgan barcha fayllarni ko‘rishi, ularning so‘nggi zaxiralangan vaqtini, necha versiyasi mavjudligini va zaxiralash jarayonining holatini yaqqol kuzatishi mumkin.

Ushbu oynaning eng muhim afzalligi shundaki, foydalanuvchi texnik jihatdan murakkab harakatlar qilmasdan, o‘z ma’lumotlarini to‘liq nazorat qilishi, avtomatik zaxiralash jarayonini boshqarishi va barcha jarayonni oddiy, tushunarli va qulay interfeys orqali olib borishi mumkin. Shu sababli bu oyna AutoBackup dasturining funksional va foydalanuvchi uchun qulayligini yaqqol ifodalab beruvchi asosiy element hisoblanadi.



2.4.2-rasm. Foydalanuvchi interfeysining asosiy oynasi

AutoBackup dasturining asosiy oynasi foydalanuvchiga zaxiraga olingan fayllar ustidan to'liq nazoratni taqdim etadi. Ilova ishga tushirilgach, foydalanuvchi barcha faol fayllar ro'yxatini ko'rishi mumkin. Ushbu ro'yxatda fayl nomi, versiyalar soni, so'nggi zaxira sanasi va vaqti, zaxiralash oraliqlari (interval) hamda status kabi muhim ustunlar mavjud.

Masalan, yuqoridagi rasmda backupTest.txt nomli fayl 1-marta zaxiraga olinganini ko'rish mumkin. So'nggi zaxiralash 14-05-2025 17:56da amalga oshirilgan bo'lib, zaxiralash oralig'i 1 ga teng, ya'ni belgilangan intervallar asosida doimiy ravishda nazorat qilinmoqda. Faylning statusi ACTIVE bo'lib, bu uning hozirda faol zaxiralash rejimida ekanini bildiradi. Ilova foydalanuvchiga bir qator amallarni bajarish imkonini ham beradi. Pastki qismdagi tugmalar orqali foydalanuvchi yangi fayl qo'shishi (Qo'shish), mavjud fayl parametrlarini o'zgartirishi (O'zgartirish), zaxiralash

jarayonini vaqtincha to'xtatishi (Stop), ilgari olingan zaxiralar tarixini ko'rishi (Tarix), zudlik bilan majburiy zaxiralashni boshlashi (Majburiy zaxiralash) yoki kerakli faylni ro'yxatdan butunlay o'chirish (O'chirish) imkoniga ega bo'ladi. Shuningdek, interfeysning yuqori qismida Sozlamalar va Chiqish tugmalari mavjud bo'lib, foydalanuvchi bu orqali dastur sozlamalarini o'zgartirishi yoki tizimdan chiqib ketishi mumkin. Bu turdagi vizual interfeys foydalanuvchiga qulaylik yaratadi, tizimda nima sodir bo'layotganini real vaqt rejimida kuzatish va kerakli harakatlarni tezkor amalga oshirish imkonini beradi. AutoBackup dasturining bu grafik interfeysi oddiy, intuitiv va samarali bo'lib, foydalanuvchining tajribasini sezilarli darajada yaxshilaydi. Bu interfeys komponenti fon rejimida ishlayotgan dasturning holatini nazorat qilishda juda foydali hisoblanadi. Foydalanuvchi dasturni sistem trayga minimallashtirib qo'ygan taqdirda ham, zaxiralash davom etadi va trayda bildirishnoma orqali foydalanuvchi ogohlantiriladi [12].

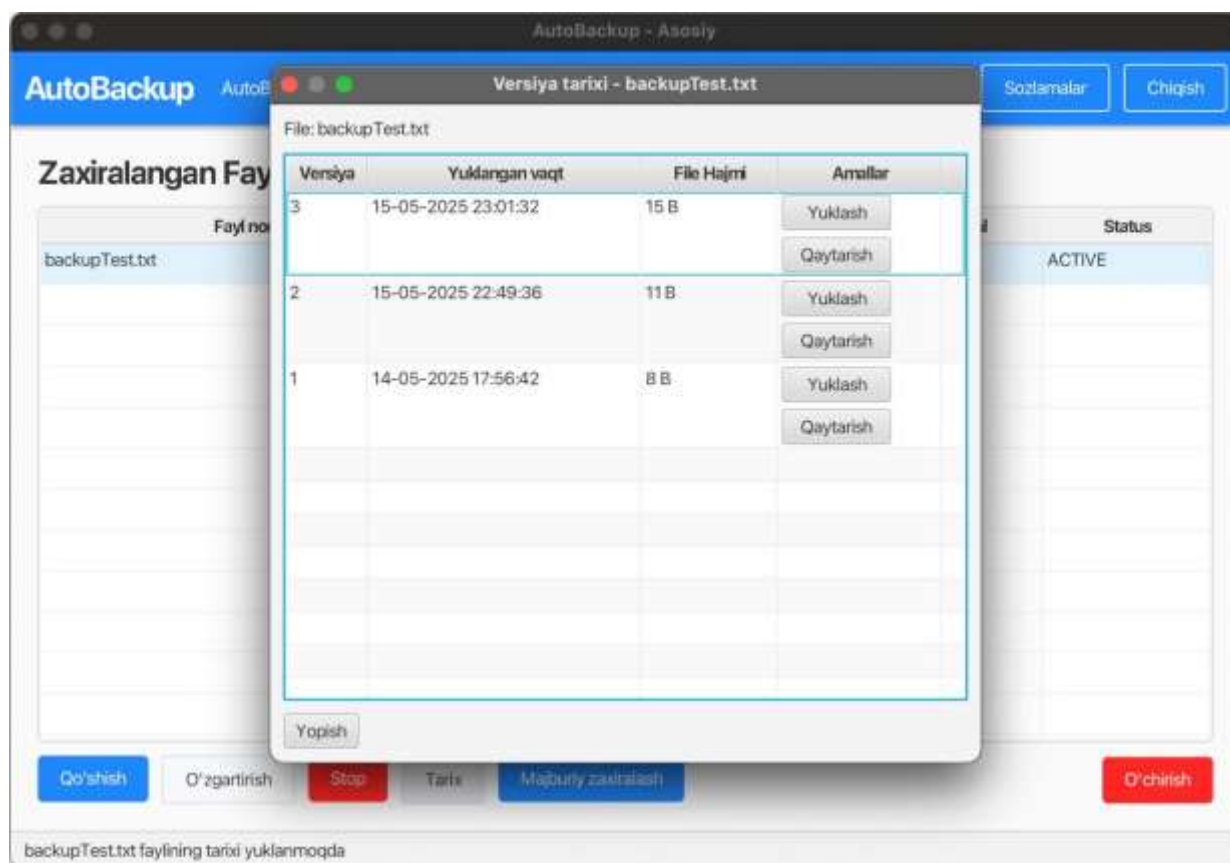
Ushbu ko'rinishda AutoBackup dasturining fayl versiyalari bo'yicha tarixni ko'rsatadigan oynasi tasvirlangan. Bu oyna foydalanuvchi tanlagan faylga tegishli barcha saqlangan versiyalar ro'yxatini ko'rish imkonini beradi. Har bir versiya uchun uning yaratilgan sanasi, hajmi va unga tegishli amallar aniq va lo'nda ko'rinishda berilgan.

Har bir qatorda versiya raqami, zaxiralanish vaqti, fayl hajmi va ikkita tugma joylashgan biri "Yuklash", ikkinchisi esa "Qaytarish" funksiyasi uchun. "Yuklash" tugmasi orqali foydalanuvchi tanlangan versiyani kompyuteriga yuklab olishi mumkin, "Qaytarish" esa o'sha versiyani dastlabki holatga tiklash imkonini beradi. Bu imkoniyat, ayniqsa, foydalanuvchi faylga noto'g'ri o'zgartirish kiritgan yoki uni xatolik bilan saqlab qo'ygan holatlarda foydali bo'ladi.

Interfeys soddaligi bilan ajralib turadi ortiqcha elementlar yo'q, barcha zaruriy ma'lumotlar bir qarashda tushunarli bo'ladi. Bu foydalanuvchiga vaqtni tejash, kerakli versiyani tezda topish va amallarni kechikmasdan bajarish imkonini beradi. Pastki

qismdagi “Yopish” tugmasi yordamida ushbu oyna yopilib, foydalanuvchi asosiy sahifaga qaytadi.

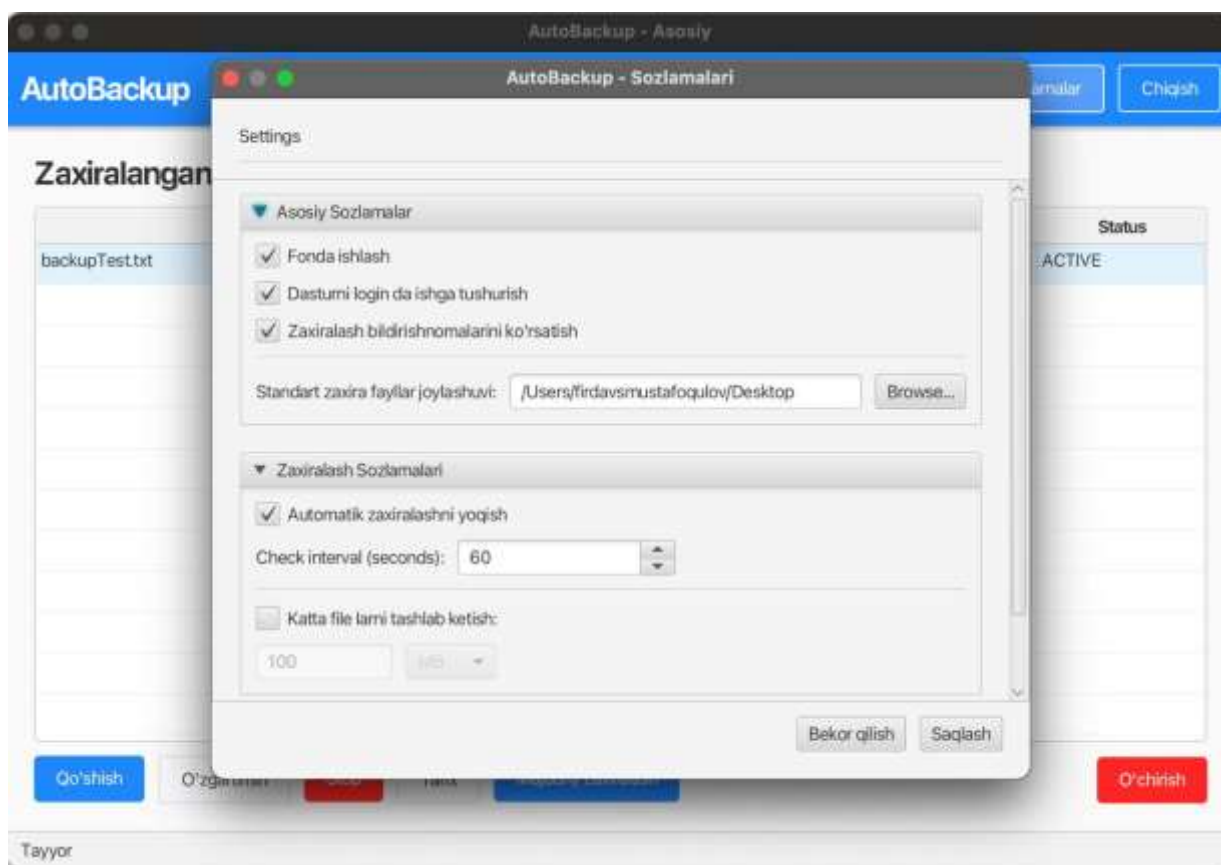
AutoBackup dasturining ushbu funksiyasi orqali foydalanuvchi o‘z ma’lumotlarini faqat saqlabgina qolmay, balki ularni nazorat qila oladi. Versiyalar bo‘yicha ishlashning qo‘shilishi bu dasturga ma’lumotlar xavfsizligini ta’minlovchi oddiy vositadan ko‘ra ko‘proq funksional dasturiy yechim sifatida qarash imkonini beradi. Texnik murakkabliklar foydalanuvchidan yashirilgan, interfeys esa ochiq va amaliy yondashuvga ega. Shu orqali har qanday foydalanuvchi texnik tajribasidan qat’i nazar o‘z fayllarini samarali boshqara oladi.



2.4.3-rasm. Faylni versiyaga qaytarish oynasi

AutoBackup dasturining “Sozlamalar” oynasi foydalanuvchiga zaxiralash jarayonini o‘ziga mos ravishda boshqarish va moslashtirish imkonini beruvchi muhim interfeys hisoblanadi. Ushbu oynada foydalanuvchi dastur ish faoliyatining asosiy parametrlarini tanlashi, zaxiralash intervalini belgilashi hamda fayllarni saqlash

manzilini o'zgartirishi mumkin. Oyna ikki asosiy qismga bo'lingan: umumiy (yoki asosiy) sozlamalar va zaxiralashga oid parametrlar.



2.4.4-rasm. Sozlamalar oynasi

Asosiy sozlamalar blokida bir nechta qulay funksiyalar mavjud. Birinchi navbatda, “Fonda ishlash” opsiyasi tanlangan bo’lsa, dastur fon rejimida, ya’ni foydalanuvchining faol aralashuvisiz, avtomatik tarzda ishlashda davom etadi. Bu esa foydalanuvchi ilovani yopib yuborgan taqdirda ham, dastur o’z ishini to’xtatmasligini anglatadi. Shuningdek, “Dasturni login da ishga tushurish” funksiyasi yordamida foydalanuvchi operatsion tizimga kirganda AutoBackup avtomatik ravishda ishga tushadi. Bu yondashuv foydalanuvchidan har safar dasturni qo’lda ishga tushirishni talab qilmaydi, bu esa qulaylik yaratadi. Navbatdagi parametr, “Zaxiralash bildirishnomalarini ko’rsatish” bo’lib, bu orqali foydalanuvchi har bir muvaffaqiyatli yoki muammo bilan yakunlangan zaxiralash jarayoni haqida tizimdan bildirishnoma olish imkoniyatiga ega bo’ladi [16].

Keyingi bo‘lim “Zaxiralash Sozlamalari” bo‘lib, bu yerda zaxiralashning avtomatikligini sozlash imkoniyati taqdim etilgan. “Avtomatik zaxiralashni yoqish” belgisi faol bo‘lsa, dastur tanlangan vaqt oralig‘ida fayllarning o‘zgarishini tekshiradi va ularni avtomatik tarzda zaxiraga oladi. Bu intervallarni foydalanuvchi soniyalar orqali aniq belgilashi mumkin. Masalan, rasmda bu interval 60 soniyaga teng, ya’ni har bir daqiqada fayllardagi o‘zgarishlar tekshiriladi. Agar o‘zgarish aniqlansa, fayl yangilangan holatda zaxiraga olinadi. Shuningdek, katta hajmli fayllarni e’tibordan chetda qoldirish uchun “Katta fayllarni tashlab ketish” opsiyasi ham mavjud bo‘lib, uni faollashtirgan holda foydalanuvchi maksimal fayl hajmini MB da belgilashi mumkin. Bu imkoniyat kam resursli tizimlarda zaxiralash jarayonining barqaror ishlashini ta’minlashda foydalidir.

Sozlamalar oynasi foydalanuvchining tasarrufidagi barcha parametrlarni o‘zlashtirgandan so‘ng, o‘zgarishlarni “Saqlash” tugmasi orqali qo‘llash mumkin. Agar foydalanuvchi o‘zgarishlardan voz kechmoqchi bo‘lsa, “Bekor qilish” tugmasi orqali avvalgi holatga qaytish imkoniyati mavjud. Ushbu funksiyalar orqali dastur yengil va moslashtiriladigan shaklda foydalanuvchiga xizmat qiladi. Shu yo‘l bilan AutoBackup tizimi nafaqat avtomatik zaxiralashni ta’minlaydi, balki har bir foydalanuvchining ehtiyojiga moslashadi.



2.4.5-rasm. Fayling shifrlangan holati

Yuqoridagi rasmda ko‘rsatilgan fayl – 1_backupTest.txt – AutoBackup dasturining ish faoliyati davomida shifrlangan holda saqlangan fayl namunasi

hisoblanadi. Fayl tarkibidagi belgilar odatiy matn ko‘rinishida emas, balki kriptografik algoritm asosida kodlangan va inson tomonidan o‘qib bo‘lmaydigan, maxsus formatda aks etgan. Bu holat faylni saqlashdan oldin shifrlash (encryption) jarayoni muvaffaqiyatli bajarilganligini ko‘rsatadi. Mazkur shifrlash funksiyasi foydalanuvchining ma’lumotlarini serverga yuborilishidan oldin himoya qilish uchun xizmat qiladi. Dasturda bu jarayon odatda AES (Advanced Encryption Standard) algoritmi asosida amalga oshiriladi va foydalanuvchining maxfiy paroli yoki undan hosil qilingan SecretKey yordamida har bir fayl alohida shifrlanadi. Shunday yondashuv orqali tizimda saqlanayotgan fayllar, hattoki serverda bo‘lsa ham, faqat tegishli foydalanuvchi tomonidan ochilishi mumkin bo‘ladi.

Shifrlangan fayl ochilganda yuqoridagi kabi simvollar to‘plami ko‘rinadi ya’ni bu raw binary data (xom ikkilik ma’lumot) bo‘lib, shifrsiz holatda qayta ishlanmaguncha real mazmunni o‘zida aks ettirmaydi. Tizim tomonidan ushbu faylni tiklash yoki ochish faqat to‘g‘ri kalit (decryption key) orqali amalga oshiriladi. Bu yondashuv AutoBackup dasturining axborot xavfsizligi darajasini oshiradi va foydalanuvchi ma’lumotlarini uchinchi tomon aralashuvidan himoyalashga xizmat qiladi. Xususan, agar fayllar serverdan o‘g‘irlangan taqdirda ham, ularning mazmuni shifrlangan holatda qoladi va hech qanday foyda bermaydi.

Shunday qilib, ko‘rsatilgan fayl holati AutoBackup tomonidan amalga oshirilgan klient tomonli shifrlash mexanizmining samaradorligini amalda tasdiqlovchi ko‘rinishdir.

Loyihani amalga oshirish dastlabki tahlil, funksional talablarni aniqlash va texnologik tanlov bosqichidan boshlandi. Bu bosqichda foydalanuvchi ehtiyojlari, mavjud muammolar, mavjud backup tizimlarining afzallik va kamchiliklari o‘rganilib, dastur uchun optimal arxitektura loyihalashtirildi [13].

2.5 II-bob bo'yicha xulosa

Dasturiy mahsulotni muvaffaqiyatli yaratish uchun ishlab chiqish jarayonini to'g'ri bosqichlarga ajratish va har bir bosqichda aniq maqsadlarga erishish zarur. "AutoBackup" dasturini ishlab chiqish jarayoni aynan mana shu metodik yondashuvga asoslanib amalga oshirildi. Jarayon quyidagi asosiy bosqichlar orqali amalga oshirildi: muammoni aniqlash, talablarni to'plash va tahlil qilish, texnologiyalarni tanlash, arxitektura ishlab chiqish, kod yozish, testlash, interfeys dizaynini ishlab chiqish va yakuniy optimallashtirish.

Ishning birinchi bosqichida muammoni aniqlash va talablarni shakllantirish foydalanuvchining real ehtiyoji asos qilib olindi. Bu ehtiyoj shaxsiy kompyuterda saqlanayotgan muhim ma'lumotlarning tasodifan yo'qolib ketishiga qarshi ishonchli zaxiralash tizimini ishlab chiqishni talab qildi. Muallifning shaxsiy tajribasida dastur kodlari Git'ga yuklanmagan holda yo'qolganligi ushbu dasturni yaratish g'oyasini shakllantirdi. Keyingi bosqichda tahlil va texnologiyalarni tanlash mavjud zaxiralash tizimlari o'rganildi, ularning kuchli va zaif tomonlari tahlil qilindi. Bu tahlil natijasida AutoBackup uchun mos, samarali va kengaytiriladigan texnologiyalar Spring Boot, JavaFX, PostgreSQL, Keycloak va AES shifrlash algoritmi tanlab olindi. Tanlangan texnologiyalar modulga asoslangan arxitektura yaratish, xavfsizlikni ta'minlash, foydalanuvchi interfeysini qulay qilish va ma'lumotlar bazasini barqaror yuritish imkonini berdi.

Arxitektura va tuzilma ishlab chiqish bosqichi dastur funksiyalarini aniq modullarga ajratishni o'z ichiga oldi. Backend va frontend qismlari o'zaro RESTful API orqali bog'landi. Fayl va papkalarni tanlash, fayldagi o'zgarishlarni aniqlash, har bir versiyani alohida saqlash va kerakli holatda tiklash kabi funksiyalar mustaqil servislarda amalga oshirildi. Shuningdek, autentifikatsiya jarayoni Keycloak orqali xavfsiz tarzda boshqarildi.

Dasturiy yechimni yaratish va kod yozish bosqichi esa real funksiyalarni hayotga tadbiq qilish bilan bog‘liq bo‘ldi. Bu jarayonda foydalanuvchi interfeysi JavaFX yordamida ishlab chiqildi, backend esa Spring Boot asosida yozildi. Hash funksiyalar yordamida fayllarning o‘zgarishlari aniqlandi, har bir yangi versiya fayl tizimida saqlanib, metadata esa PostgreSQL ma’lumotlar bazasida qayd etildi.

Yakuniy xulosa sifatida ta’kidlash mumkinki, “AutoBackup” dasturini ishlab chiqish bosqichlari muallifga real texnik muammolarni aniqlash va ularni yechishga doir to‘liq tajriba taqdim etdi. Har bir bosqichda mustaqil qaror qabul qilish, texnologik yechimni tanlash, tizimli yondashuv orqali dasturiy mahsulot yaratish va foydalanuvchi ehtiyojiga mos yechimni ishlab chiqish ko‘nikmalari shakllandi. Bu loyiha nafaqat ma’lumotlar xavfsizligini ta’minlovchi muhim dasturiy ta’minot sifatida, balki muallifning professional rivojlanishida ham muhim qadam bo‘lib xizmat qildi.

III-BOB. HAYOT FAOLIYATI XAVFSIZLIGI VA EKOLOGIYA

3.1. Havo muhitining changlanganligi va uning odamga ta'siri

Zamonaviy sanoat va urbanizatsiya sharoitida atmosfera havosining sifati inson salomatligi uchun muhim omillardan biriga aylangan. Ayniqsa, havoning changlanishi sog'liq uchun xavfli holatlardan biridir. Chang zarralari, odatda, ko'z bilan ko'rinmas bo'lib, ular turli manbalardan sanoat chiqindilari, avtomobil gazlari, qurilish ishlari, yo'l-transport harakati va hatto tabiiy hodisalardan kelib chiqadi.

Chang - bu havoda osilgan holda uchib yuruvchi, odatda juda mayda zarrachalardan iborat bo'lgan modda bo'lib, u ko'zga ko'rinmas darajada bo'lishi mumkin. U inson salomatligiga, ekologik holatga va texnik qurilmalarning ishlashiga sezilarli ta'sir ko'rsatadigan omillardan biri hisoblanadi. Chang – oddiygina tuproq yoki qum emas, balki u turli xil moddalarning parchalanishi, yo'qolishi yoki yoqilishi natijasida yuzaga keladigan fizik-kimyoviy holatdir. U atmosferada muallaq holda uzoq vaqt saqlanib qolishi mumkin va aynan shu xususiyati uni xavfli qiladi. Changning kelib chiqish manbalari bir nechta asosiy guruhga ajratiladi. Birinchi guruh - bu tabiiy manbalar. Masalan, shamol ta'sirida yuzaga keladigan qum bo'ronlari, vulqon otilishlari natijasida havoga chiqadigan kul va chang aralashmalari, o'rmon yong'inlari, cho'llarda quruq yer yuzasining ko'chishi, daraxtlarning changchi donalari (polenlar) va hatto dengiz sathidan bug'lanayotgan tuz zarralari ham tabiiy chang manbalaridir. Bu hodisalar ko'p hollarda mavsumiy xarakterga ega bo'lib, tabiatning o'z qonuniyatlariga asoslangan holda sodir bo'ladi.

Ikkinchi guruh – sun'iy yoki antropogen manbalar bo'lib, ular inson faoliyati natijasida yuzaga keladi. Eng ko'p uchraydiganlari sanoat korxonalari faoliyati, qurilish ishlari, yo'l qurilishi, konchilik, metallurgiya, to'qimachilik va qishloq xo'jaligidir. Ushbu sohalarda faoliyat yuritilayotgan vaqtda katta miqdorda mayda zarrachalar ajralib chiqadi va atrof-muhitga tarqaladi. Avtomobillar harakati, ayniqsa yirik shaharlarda, yo'l qoplamalarining yeyilishi, shinalarning ishqalanishi va ichki yonuv

dvigatellaridan chiqayotgan gazlar orqali havoga ko‘plab zararli chang zarralari ajraladi. Uy xo‘jaligida ham changning shakllanishi kuzatiladi: gilamlar, mebellar, kiyim-kechaklar, inson tanasidan to‘kiladigan teri zarralari va hayvonlarning tuklari ichki makondagi changni tashkil qiladi. Bundan tashqari, ko‘plab kimyoviy birikmalar bilan boyitilgan changlar ham mavjud bo‘lib, ular inson salomatligiga yanada katta xavf tug‘diradi. Masalan, asbest changi, metall zarralari, og‘ir metallar va radioaktiv moddalarning zarrachalari nafaqat nafas yo‘llarini, balki ichki organlarni ham jiddiy zararlashi mumkin. Bunday changlar o‘pka orqali qonga singib, butun organizm bo‘ylab tarqaladi va surunkali kasalliklarni keltirib chiqaradi.



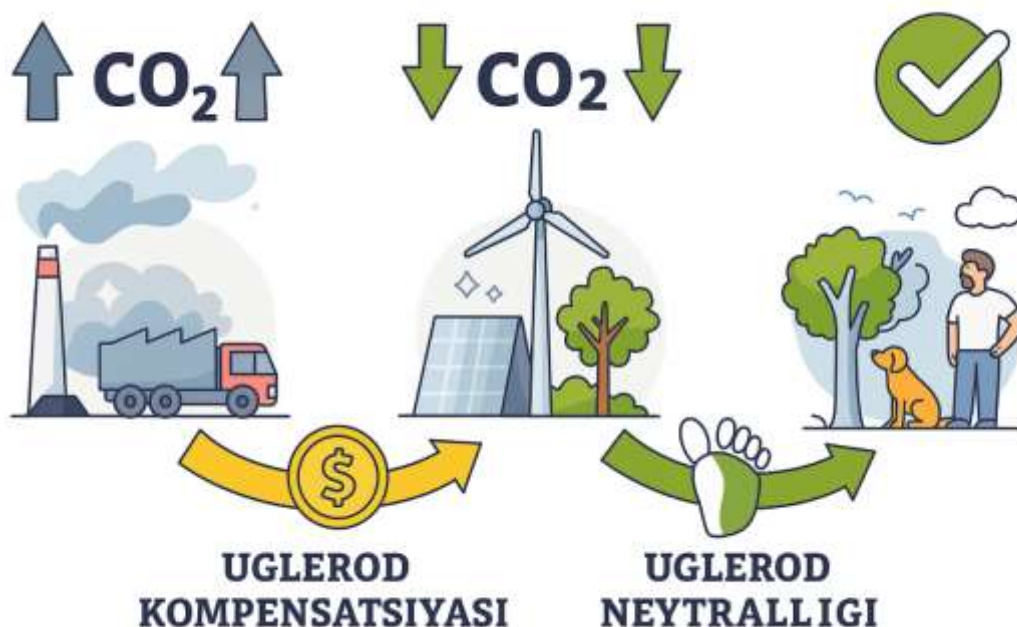
3.1.1-rasm. Havo ifloslanish asosiy sabablari

Shuni unutmaslik kerakki, chang faqatgina tashqi muhitni ifloslantiruvchi omil emas, balki inson hayoti va mehnat faoliyatiga ham bevosita ta'sir qiladi. Masalan,

chang ko'p bo'lgan ish joylarida xodimlarning ish unumdorligi kamayadi, tez charchash, bosh og'rig'i, yo'tal, ko'zlarning achishishi kabi holatlar ko'proq uchraydi. Bu esa sog'liq uchun xavfli bo'lishi bilan birga ishlab chiqarish samaradorligiga ham salbiy ta'sir ko'rsatadi. Shuning uchun chang manbalarini nazorat qilish, havoni tozalovchi vositalardan foydalanish, himoya niqoblari taqish, bino ichidagi shamollatish tizimlarini yangilab borish muhim profilaktik choralar hisoblanadi.

Chang tabiiy va sun'iy manbalar natijasida yuzaga keladi. Inson salomatligi, ekologik muhit va ishlab chiqarish faoliyati uchun changning zararli ta'siri yillar davomida ilmiy isbotlangan bo'lib, uning oldini olish har birimiz uchun muhim vazifadir. Toza havo – bu nafaqat sog'liq, balki sifatli hayot garovidir. Havodagi chang zarralarining inson sog'lig'iga ko'rsatadigan salbiy ta'siri juda keng va jiddiy bo'lib, u qisqa muddatli noqulayliklardan tortib, uzoq muddatli va surunkali kasalliklarga olib kelishi mumkin. Chang – bu oddiygina ko'rinmas zarracha emas, balki u ko'pincha turli kimyoviy, toksik va hatto allergik moddalarni o'z ichiga oladi. Ayniqsa, sanoat korxonalarida, qurilish ob'ektlarida yoki avtomobil harakati zich bo'lgan joylarda havo tarkibidagi chang miqdori yuqori bo'lib, bu bevosita inson organizmiga tahdid soladi.

Eng avvalo, chang nafas olish tizimiga salbiy ta'sir ko'rsatadi. Inson har kuni o'rtacha 20 ming martagacha nafas oladi va bu orqali taxminan 10–15 ming litr havo o'pkalardan o'tadi. Agar bu havo tarkibida chang bo'lsa, mayda zarralar burun, halqum, traxeya va bronxlar orqali to'g'ridan-to'g'ri o'pkaga yetib boradi. Bu holat o'pkada yallig'lanish jarayonlarini yuzaga keltiradi, surunkali bronxit, astma, pnevmoniya va hatto o'pka saratoni kabi xavfli kasalliklarni rivojlantiradi. Ayniqsa, 2.5 mikrometrdan kichik bo'lgan chang zarralari (PM2.5) juda xavfli hisoblanadi, chunki ular nafaqat o'pkaga kiradi, balki qon orqali butun organizmga tarqaladi [14].



3.1.2-rasm. Havo sifati almashinishi

Bundan tashqari, chang zarralari yurak-qon tomir tizimiga ham salbiy ta'sir ko'rsatadi. Kislorod yetishmovchiligi tufayli yurak ko'proq ishlashga majbur bo'ladi, bu esa yurak urishining tezlashishi, qon bosimining ko'tarilishi, yurakning ishemik kasalligi yoki insult xavfini oshiradi. Ayrim ilmiy tadqiqotlar chang miqdori yuqori bo'lgan shahar va sanoat hududlarida yurak-qon tomir kasalliklari tufayli o'lim hollari ko'proq uchrashini ko'rsatgan. Chang allergik reaksiyalarga sabab bo'lishi ham mumkin. Ayniqsa, o'simlik changlari (polen), uy changidagi mikroskopik mayda o'rgimchaklar (kleschlar), hayvon tuklari, zamburug' sporalariga allergiyasi bor odamlar bunday muhitda uzoq vaqt bo'lsa, ularda burun bitishi, ko'z yallig'lanishi, teri toshmalari va nafas olishda qiyinchiliklar kuzatiladi. Bunday allergiyalar esa keyinchalik bronxial astmaga olib kelishi mumkin.

Chang nafaqat ichki organlarga, balki tashqi a'zolarga ham zarar yetkazadi. Ko'zlar changli muhitda tezda achishadi, yosh oqadi, qizaradi va ko'rish qobiliyatida vaqtincha pasayish bo'lishi mumkin. Teri ustida esa chang yallig'lanishga, toshma va

qichishishga olib keladi. Bu holatlar gigiyena qoidalariga rioya qilinmasa, teri infeksiyalarining rivojlanishiga ham sabab bo‘ladi.

Alohida e‘tibor berish kerak bo‘lgan yana bir jihat chang insonning asab tizimiga ham bilvosita ta’sir qiladi. Kislorod yetishmasligi natijasida miya faoliyati sustlashadi, bosh og‘rig‘i, charchoq, e‘tibor yo‘qolishi, uyqu buzilishi, kayfiyatning tushishi kabi holatlar yuzaga keladi. Uzoq muddat changli muhitda ishlash yoki yashash, insonning umumiy ruhiy holatiga ham salbiy ta’sir qiladi.

Xulosa qilib aytganda, havodagi chang zarralari inson salomatligi uchun ko‘p qirrali xavf tug‘diradi. Ular nafas olish tizimidan tortib yurak, asab va immun tizimigacha ta’sir ko‘rsatadi. Ayniqsa, bolalar, keksa yoshdagilar, homilador ayollar va surunkali kasalliklarga ega insonlar bu xavf ostida bo‘ladi. Shuning uchun havo sifati ustidan nazorat o‘rnatish, havoni tozalash texnologiyalarini joriy etish, shaxsiy himoya vositalaridan foydalanish va ekologik madaniyatni yuksaltirish orqali changning sog‘liq uchun zararini kamaytirish mumkin. Toza havo – bu faqat qulaylik emas, balki sog‘lom hayot tarzining ajralmas tarkibiy qismidir.

Chang miqdorining oshishi ish unumdorligiga ko‘rsatadigan ta’siri bugungi kunda nafaqat tibbiy, balki iqtisodiy va texnologik muammo sifatida ham e’tirof etilmoqda. Har qanday ishlab chiqarish muhitida havo sifatining yomonlashuvi, ayniqsa chang miqdorining ortib borishi, bevosita insonning jismoniy holati, e‘tiborini jamlay olishi, ruhiy barqarorligi va umumiy ish unumdorligiga salbiy ta’sir ko‘rsatadi. Bu holat, odatda, sekinlik bilan kechadi va bir qarashda oddiy tuyulgan chang zarrachalari aslida katta miqyosdagi yo‘qotishlarga sabab bo‘lishi mumkin.

Avvalo, chang bilan to‘yingan ish muhitida nafas olish qiyinlashadi. Bu organizmga kislorod yetib borishini kamaytiradi. Natijada ishchilar tezda charchaydi, bosh og‘rig‘i, lohaslik, uyquchanlik yoki holsizlik holatlari paydo bo‘ladi. Bu holatlar insonning jismoniy quvvatini cheklab qo‘yadi, hattoki oddiy harakatlar yoki mexanik ishlarni bajarishning o‘zi qiyinchilik tug‘diradi. Ishchilarda tez-tez tanaffusga ehtiyoj

paydo bo‘ladi, bu esa ishlab chiqarish jarayonining sekinlashishiga olib keladi. Bundan tashqari, chang e’tibor va fikrni jamlash qobiliyatini pasaytiradi. Bu ayniqsa nozik, yuqori aniqlikni talab qiladigan sohalarda (masalan, elektronika, kimyo, dori ishlab chiqarish yoki tibbiyot asboblari yig‘ilishi) juda muhim hisoblanadi. Changli muhitda xatoliklar soni ortadi, bu esa mahsulot sifatining pasayishiga, qayta ishlov va chiqindilar hajmining ko‘payishiga olib keladi. Bularning barchasi korxona uchun vaqt va mablag‘ yo‘qotilishi demakdir.

Ruhiy holatga ham chang salbiy ta’sir ko‘rsatadi. Doimiy iflos va noqulay muhitda ishlash inson psixologiyasiga bosim o‘tkazadi. Ishchilarda qoniqarsizlik hissi paydo bo‘ladi, bu esa motivatsiyaning pasayishiga olib keladi. Ruhiy charchoq fonida jamoa ichidagi munosabatlar yomonlashadi, ishda intizom pasayadi va bu umumiy muhitga salbiy ta’sir ko‘rsatadi.

Yana bir muhim jihat chang uskunalarning ishlashiga ham ta’sir qiladi. Zarrachalar ventilyatsiya tizimlari, dvigatellar, kompyuterlar va boshqa mexanizmlarga kirib, ularning ishdan chiqish xavfini oshiradi. Bunday nosozliklar ortidan majburiy texnik to‘xtashlar, ta’mirlash ishlariga vaqt va mablag‘ sarfi ko‘payadi. Bu esa ishlab chiqarish sur’atining pasayishiga olib keladi. Ish joyida sog‘lom mehnat muhitini ta’minlash har bir ish beruvchining mas’uliyatidir. Shamollatish tizimlarini yangilash, havo filtrlarini o‘rnatish, changni yutuvchi qurilmalar bilan jihozlash, ishchilarga shaxsiy himoya vositalari (niqob, respirator) berish orqali ish unumdorligini saqlab qolish mumkin. Chunki toza havo nafaqat sog‘liq garovi, balki samarali mehnat uchun zarur asosdir.

Shu bois, changning ortishi nafaqat salomatlik, balki ishlab chiqarish samaradorligiga ham jiddiy xavf tug‘diradi. Bu holat vaqt o‘tgan sayin jiddiy oqibatlariga olib kelishi mumkin: mahsulot sifati tushadi, ishlab chiqarish xarajatlari ortadi, texnik xizmatlar ko‘payadi, ishchilar almashinuv tezligi kuchayadi. Shunday ekan, havoning tozaligi va chang miqdorini normada saqlash har qanday korxonaning

ustuvor strategik vazifasi bo'lishi kerak. Bu orqali nafaqat sog'lom ishchi kuchi, balki barqaror va raqobatbardosh ishlab chiqarish jarayoni yo'lga qo'yiladi. Havodagi changni kamaytirish bu faqat sanoat korxonalari yoki davlat organlari zimmasidagi vazifa emas, balki har bir tashkilot, har bir mahalla va hattoki har bir fuqaroning salomatlik va yashash muhitini yaxshilashga qaratilgan umumiy mas'uliyatidir. Changning havoda haddan tashqari ko'payib ketishi sog'liq uchun xavfli bo'lishi bilan birga, ishlab chiqarish jarayonlariga, ekologik holatga va turmush sifati darajasiga ham salbiy ta'sir ko'rsatadi. Shuning uchun bu muammoni bartaraf etish bo'yicha chora-tadbirlar tizimli, ko'p bosqichli va samarali bo'lishi zarur.

Eng avvalo, sanoat va ishlab chiqarish sohasida changga qarshi kurashda texnologik choralar asosiy o'rinni egallaydi. Ishlab chiqarish sexlari, omborlar, konlar yoki qurilish maydonlarida chang hosil bo'lishining oldini olish yoki uni kamaytirish maqsadida maxsus filtrli shamollatish tizimlari o'rnatilishi lozim. Bu tizimlar changni havoga chiqmasdan avval tutib qoladi yoki uni maxsus qoplarda yig'ib, utilizatsiya qilishga imkon beradi. Ayrim korxonalarda esa elektrostatik filtrlar, gidrofiltrlar yoki impulsli tozalovchi qurilmalar qo'llaniladi. Bu texnologiyalar changni sezilarli darajada kamaytiribgina qolmay, atrof-muhitni ham asraydi.

Shuningdek, ishchilarni himoya qilish choralarini ko'rish ham muhim ahamiyatga ega. Chang bilan ishlanadigan joylarda respiratorlar, niqoblar, ko'zoynaklar, yopiq maxsus kiyimlar taqdim etilishi kerak. Ular orqali inson organizmiga chang zarralarining kirish ehtimoli sezilarli darajada kamayadi. Bundan tashqari, bunday muhitda ishlovchilarga muntazam tibbiy ko'riklar o'tkazilishi, sog'lig'ida o'zgarish aniqlangan xodimlar esa vaqtincha boshqa ishga o'tkazilishi lozim. Bino va ofis ichidagi changga qarshi kurash uchun esa muntazam va sifatli tozalash ishlari amalga oshirilishi kerak. Chang yutuvchi changyutkichlar, nam lattalar, mikroto'qima matolar va havo tozalagichlar yordamida ish muhiti sog'lomlashtiriladi. Qurilish va ta'mirlash ishlari olib borilayotgan paytda esa xonalar ajratilishi, ajratuvchi

parda va polietilen qoplamalar orqali changning boshqa joylarga tarqalishi oldi olinadi. Xona ichidagi shamollatish tizimlari toza saqlanishi, ularni muntazam tozalab turish esa muhim hisoblanadi.

Shaharsozlik va ekologik chora-tadbirlar ham chang miqdorini kamaytirishda katta rol o'ynaydi. Daraxt ekish, yashil zonalarini kengaytirish, maysazorlar va gulzorlarni ko'paytirish havodagi changni o'zlashtirishda tabiiy filtr vazifasini bajaradi. Ayniqsa, avtomobil harakati zich bo'lgan joylarda shunday yashil to'siqlar chang tarqalishini kamaytiradi. Shahar va qishloqlarda yo'llarni asfaltlash, chang bosadigan moddalar bilan sug'orish, eski chang ko'tariluvchan yo'llarni betonga almashtirish orqali chang ko'tarilishi kamaytiriladi. Transport vositalaridan chiqadigan chiqindilarni nazorat qilish ham ekologik tozalikni saqlashda muhim o'rin tutadi. Buning uchun benzin sifati, texnik ko'rikning sifatli o'tkazilishi, elektrotransport vositalariga o'tish kabi chora-tadbirlar muhimdir. Aholi darajasida amalga oshiriladigan oddiy, lekin samarali choralarga tozalikka e'tiborli bo'lish, chang ko'tarilishini kamaytiradigan yer qoplamalaridan foydalanish, tomorqalarda yer yuzasini yalang'och qoldirmaslik, suv sepish va daraxt parvarishi kiradi. Har bir fuqaro o'z yashash hududida tozalikni saqlasa, umumiy ekologik muhit ham barqarorlashadi [15].

Xulosa qilib aytganda, havodagi changni kamaytirish bo'yicha chora-tadbirlar tizimli, doimiy va har tomonlama bo'lishi kerak. Bu borada davlat idoralari, sanoat korxonalari, mahalliy hokimiyatlar, atrof-muhit muhofazasi tashkilotlari va eng asosiysi – oddiy fuqarolarning birgalikdagi harakati orqali natijaga erishish mumkin.

Havodagi chang muammosiga qarshi kurashishda ijtimoiy va ekologik yondashuv masalaning eng muhim, kompleks va barqaror yechimlarini taklif etadi. Bu faqat texnik uskunalar yoki sanoat filtrlarining o'rnatilishi bilan cheklanmaydi, balki odamlarning fikrlash tarzini, yashash madaniyatini, atrof-muhitga bo'lgan munosabatini tubdan o'zgartirishni talab qiladi. Chunki chang muammosi – bu faqat bir korxona yoki bir hududga xos masala emas, balki jamiyatning barcha qatlamlariga

bevosita ta'sir ko'rsatadigan ekologik xavf hisoblanadi. Ijtimoiy ongda ushbu muammoning jiddiyligini tushunish va uni hal etishga har kim o'z hissasini qo'shishi bugungi davr talabi hisoblanadi.

Ekologik yondashuv deganda, avvalo, atrof-muhit bilan uyg'un yashash, inson faoliyatining tabiatga zarar yetkazmasligini ta'minlash tushuniladi. Masalan, yirik sanoat zonalarida atrofida yashil hududlar tashkil qilish, daraxtzor va maysazorlarni ko'paytirish orqali chang miqdorini tabiiy ravishda kamaytirish mumkin. Daraxtlar, ayniqsa ignabargli o'simliklar, havodagi chang va zararli moddalarni o'ziga singdiradi va tabiiy filtr vazifasini bajaradi. Shu bois har bir sanoat korxonasi ekologik mas'uliyatni o'z zimmasiga olib, o'z hududida yoki atrofida yashil zona yaratishi kerak. Shaharsozlikda ham ekologik yondashuv muhim: ko'p qavatli binolar orasida ochiq maydonlar, yashil bog'lar, havo oqimini to'g'ri yo'naltiradigan joylashuvga ahamiyat berilishi lozim.

Ijtimoiy yondashuv esa, odamlarning ongini o'zgartirish, ekologik savodxonlikni oshirish orqali chang muammosiga faol munosabat bildirishni nazarda tutadi. Bu yerda maktab, kollej va oliy ta'lim muassasalarida ekologik tarbiyani kuchaytirish, ommaviy axborot vositalarida havo sifati, sog'liq va chang muammosi haqida muntazam ma'lumotlar berish muhim ahamiyatga ega. Shuningdek, ekologik tadbirlar daraxt ekish aksiyalari, ko'ngillilar guruhlarining chiqindilarni tozalash ishlari, havo sifati bo'yicha ochiq monitoring tizimlari aholining befarq bo'lmasligini ta'minlaydi. Davlat va jamiyat o'rtasidagi hamkorlikni rivojlantirish ham bu boradagi chora-tadbirlarning samaradorligini oshiradi. Masalan, hukumat tomonidan ishlab chiqilgan ekologik siyosat va qonunchilik asnosida fuqarolarning havo sifati monitoringida ishtirok etishi, ekologik loyihalarni qo'llab-quvvatlashi, davlat tomonidan ekologik talablarni buzgan korxonalarga nisbatan jazo choralarining kuchaytirilishi bu jarayonni jadallashtiradi. Ekologik normativlar asosida ishlab chiqarish texnologiyalarini takomillashtirish, zararli moddalarning havoga chiqishini

kamaytiruvchi innovatsion texnologiyalarni joriy etish ijtimoiy mas'uliyatli yondashuvning asosiy ko'rinishlaridan biridir.

Ko'p hollarda oddiy fuqarolar o'zlari yashayotgan hududda chang yoki ifloslanish muammosiga befarq bo'lishadi. Bunday holatlarni bartaraf etish uchun ekologik tashabbuslarning moliyalashtirilishi, ijtimoiy rag'bat tizimlarining joriy etilishi (masalan, ekologik jihatdan qulay hududlarda yashayotganlar uchun imtiyozlar) juda katta ahamiyatga ega. Zero, ijtimoiy yondashuvning asosiy tamoyili har bir fuqaro o'z muhitiga befarq bo'lmasligi, uni toza saqlashda ishtirok etishi va boshqalarga o'rnak bo'lishidir.

Havodagi changga qarshi kurashda ijtimoiy va ekologik yondashuv eng samarali, uzoq muddatli va barqaror yechimlar beradi. Bu yondashuvlar jamiyatni, texnologiyalarni va tabiatni uyg'unlashtirishga xizmat qiladi. Har bir inson o'zining kichik harakati bilan katta o'zgarishlarga zamin yaratishi mumkin – bu daraxt ekish, axlatni to'g'ri ajratish, changli hududda niqob taqish, bolalarga toza havoning ahamiyatini o'rgatish bo'lishi mumkin. Atrof-muhitni muhofaza qilish – bu faqat ekologlarning emas, balki har bir insonning hayotiy burchidir.

3.2. Ishlab chiqarish binolari va uskunalarga yong'in xavfsizligi talablari

Ishlab chiqarish korxonalarida yong'in xavfsizligini ta'minlash – inson hayoti, mulk va atrof-muhitni muhofaza qilishning eng muhim shartlaridan biridir. Bunday binolarda har xil yonuvchan moddalar, elektr uskunalari, kimyoviy reagentlar, gaz ballonlari kabi xavf manbalari mavjud bo'lishi sababli yong'in chiqish xavfi yuqori darajada bo'ladi. Shu bois, ishlab chiqarish binolari va unda joylashgan uskunalalar uchun maxsus yong'in xavfsizligi talablari belgilangan va ularni qat'iy rioya qilish shart.

Yong'in xavfsizligi - bu har qanday ishlab chiqarish jarayonida asosiy ustuvor yo'nalishlardan biri bo'lib, inson hayoti, sog'lig'i, ishlab chiqarish uskunalari, moddiy boyliklar va umumiy mulk xavfsizligini ta'minlashga xizmat qiladi. Ayniqsa, sanoat

korxonalari, omborxonalar, ishlab chiqarish sexlari va texnologik obyektlar faoliyat yuritayotgan joylarda yong'in xavfi har doim mavjud bo'lib, bu xavfni oldindan aniqlash, baholash va unga qarshi kurashish dolzarb masala sifatida doimo kun tartibida turadi. So'nggi yillarda texnika va texnologiyalar rivojlanib borayotgan bir paytda ishlab chiqarish jarayonlari yanada murakkablashdi, yuqori harorat, bosim, yonuvchan moddalar, elektr jihozlari bilan ishlovchi tizimlar ko'paydi. Bu esa yong'in chiqish xavfini ham yuqori darajaga olib chiqdi. Ko'pgina korxonalarda yong'in faqat moddiy zarar bilan cheklanmay, balki inson hayotiga zomin bo'lishi mumkin bo'lgan og'ir oqibatlarga olib keladi. Ba'zan esa bitta kichik ehtiyotsizlik butun korxonaning ishdan chiqishiga, ishlab chiqarish zanjirining to'xtab qolishiga, minglab odamlarning ishsiz qolishiga, atrof-muhitga zarar yetishiga olib keladi. Aynan shu jihat yong'in xavfsizligini bugungi kunda dolzarb muammolardan biriga aylantirgan. Ishlab chiqarish obyektlarida, ayniqsa yonuvchan moddalar bilan ishlanadigan joylarda yong'in xavfsizligini e'tiborsiz qoldirish salbiy oqibatlarning oldini olishni imkonsiz qiladi.

Yong'in xavfsizligiga oid masalalarning dolzarbligi shundaki, bu soha nafaqat sanoat ishlab chiqarishida, balki har qanday iqtisodiy faoliyat turi – transport, qishloq xo'jaligi, energetika, savdo, xizmat ko'rsatish tizimlarida ham eng muhim xavfsizlik talabi sifatida e'tirof etiladi. Misol uchun, kichik elektr qisqa tutashuvi o'z vaqtida bartaraf etilmasa, butun bir sexni, ba'zan esa shaharni ishdan chiqarishi mumkin. Bu holatda texnik bilim, xavfsizlik me'yorlariga rioya qilish, zamonaviy texnologiyalar va inson omilining uyg'unligi hal qiluvchi rol o'ynaydi. Shuningdek, global iqlim o'zgarishlari va ekologik holatning yomonlashuvi fonida ham yong'in xavfsizligining ahamiyati ortib bormoqda. Harorat oshishi, quruq ob-havo, havo aylanishining o'zgarishi, ob'ektlarda saqlanayotgan yonuvchan materiallarning haddan tashqari ko'pligi yong'in xavfini kuchaytirmoqda. Bundan tashqari, texnik xavfsizlikka nisbatan beparvolik, korxonalarda zarur choralarni ko'rmaslik, noto'g'ri montaj ishlari

yoki yong'in o'chirish vositalarining yetarli emasligi vaziyatni yanada murakkablashtiradi.



3.2.1-rasm. Yong'in xavfsizligining talab va qoidalari

Ishlab chiqarish binolari va uskunalarda yong'in xavfsizligini ta'minlash masalasi har qanday iqtisodiy tizimda, ishlab chiqarish va texnologik jarayonda doimo dolzarb hisoblanadi. Bu faqat texnik ehtiyot chorasigina emas, balki inson hayoti va ekologik xavfsizlikning kafolatidir. Shu sababli ushbu mavzuni chuqur o'rganish, mavjud muammolarni tahlil qilish va ularning samarali yechimlarini ishlab chiqish bugungi kunda nihoyatda muhim va vaqtida hal qilinishi lozim bo'lgan masalalardan biridir. Ishlab chiqarish binolarini loyihalashda yong'in xavfsizligi talablarini inobatga olish - bu nafaqat texnik zarurat, balki inson hayoti, mulk, ishlab chiqarish jarayoni va atrof-muhit xavfsizligini ta'minlashga qaratilgan muhim ehtiyojdir. Bino qurilishining ilk bosqichidan boshlab barcha yondashuvlar yong'in xavfi ehtimolini kamaytirish, yong'in paytida tezkor evakuatsiyani tashkil etish va yong'inni oldini olishga

yo‘naltirilishi lozim. Chunki eng ilg‘or texnologiyalardan foydalangan holda ishlab chiqilgan binoning o‘zi yong‘inga nisbatan bardoshli bo‘lmasa, u yerda sodir bo‘ladigan yong‘in qisqa vaqt ichida jiddiy oqibatlarga olib keladi.

Avvalo, binoning konstruktiv tuzilmasi yong‘inga chidamli materiallardan tashkil topgan bo‘lishi shart. Qurilishda ishlatiladigan devor, pol, shift va tom qoplamalari yonuvchan bo‘lmagan yoki sekin yongan materiallardan tayyorlanishi kerak. Ayniqsa, bino ichidagi asosiy yuk ko‘taruvchi elementlar (karkaslar, ustunlar, rigellar) yong‘in ta‘sirida deformatsiyalanib, butun binoning qulashiga sabab bo‘lmasligi kerak. Shu maqsadda metall konstruksiyalar maxsus yong‘inga chidamli bo‘yoqlar bilan qoplanadi, beton elementlar esa issiqqa chidamli qatlamlar bilan mustahkamlanadi. Shuningdek, binoda yong‘in tarqalishining oldini olish uchun alohida sektorlarga bo‘lingan holda loyiha tuzish kerak. Har bir sektor orasida yong‘inga bardoshli to‘siqlar o‘rnatiladi. Bu devorlar va eshiklar yong‘in paytida bir hududdagi olovni boshqa hududga o‘tishiga to‘sqinlik qiladi. Yana bir muhim talab – har bir bino evakuatsiya yo‘laklari bilan jihozlangan bo‘lishi shart. Bu yo‘laklar to‘g‘ridan-to‘g‘ri chiqish eshiklariga olib boradigan, yetarli kenglikka ega bo‘lgan, to‘siqsiz, har qanday vaqtda ochilishi mumkin bo‘lgan va belgilar bilan to‘liq ta‘minlangan yo‘nalish bo‘lishi kerak. Evakuatsiya eshiklari hech qachon qulflanmasligi, ularning oldi bo‘sh va ochiq saqlanishi zarur.

Binoda havo aylanishini ta‘minlovchi shamollatish tizimi ham yong‘in xavfsizligida muhim rol o‘ynaydi. Tabiiy yoki sun‘iy yo‘llar bilan tashkil etilgan shamollatish tizimi orqali binoda tutun to‘planib qolishi, olov tez tarqalishi oldi olinadi. Shuningdek, tutunni chiqarish tizimlari, avtonom havo tozalovchi qurilmalar va signalizatsiya tizimlari loyihaning ajralmas qismi sifatida belgilanishi lozim.

Bino ichida elektr tizimlari ham yong‘inga sabab bo‘ladigan asosiy manba sifatida alohida e‘tiborga olinadi. Elektr kabellari maxsus yong‘inga chidamli quvurlar ichida o‘tkazilishi, ortiqcha yuklamalarni bartaraf etish uchun avtomatik uzgichlar o‘rnatilishi,

elektr qurilmalar joylashuvi esa xavfsizlik nuqtayi nazaridan baholanib, ularning yonida yonuvchan materiallar bo'lmashligi kerak.

Binoning tashqi maydoni ham xavfsizlik talablariga mos bo'lishi kerak. Yong'in bo'lgan taqdirda yong'in mashinalarining erkin kirib kelishini ta'minlaydigan yo'llar, bino atrofida suv gidrantlari yoki yong'inga qarshi suv omborlari mavjud bo'lishi kerak. Bundan tashqari, har qanday ishlab chiqarish binosining yonida maxsus tayyorlangan evakuatsiya maydonchasi bo'lishi, xodimlar yong'in paytida xavfsiz masofada to'planishi uchun sharoit yaratilishi shart.

Ishlab chiqarish binosining loyihasi yong'in xavfsizligiga to'liq mos bo'lishi uchun har bir texnik detaldan tortib, inson harakatlanish yo'nalishigacha puxta hisob-kitob qilingan bo'lishi kerak. Bu nafaqat loyiha sifati, balki korxona faoliyatining barqarorligi, inson hayotining asosiligi va atrof-muhitning muhofazasi uchun juda katta ahamiyatga ega. Yong'inning oldini olish – uni o'chirishdan ko'ra osonroq va arzonroq turadigan chora ekanini unutmaslik lozim. Shu bois, har qanday bino qurilishi yong'inga qarshi barqarorlik tamoyiliga asoslangan bo'lishi kerak. Ishlab chiqarish jarayonining ajralmas qismi bo'lgan texnologik uskuna va agregatlar o'zlarining konstruktiv xususiyatlari, ish prinsipi, energiya manbalari va ishlab chiqarilayotgan mahsulotlar tabiatiga ko'ra yong'in xavfi yuqori bo'lgan obyektlar hisoblanadi. Shu sababli ularga nisbatan yong'in xavfsizligi bo'yicha qo'yiladigan talablar nihoyatda qat'iy, batafsil va har tomonlama ehtiyotkorlikni talab etadi. Uskunalar bevosita elektr energiyasi, gaz, issiqlik, portlovchi yoki yonuvchan materiallar bilan ishlaydi, bu esa nosozlik yoki noto'g'ri foydalanish holatlarida yong'in xavfini kuchaytiradi.

Avvalo, har qanday uskuna ishlab chiqaruvchining texnik talqiniga mos ravishda o'rnatilishi va ekspluatatsiya qilinishi shart. Ishlab chiqaruvchi tomonidan belgilangan maksimal yuklama, kuchlanish, harorat chegarasi kabi parametrlar qat'iy rioya qilinmasa, bu ichki isish, qisqa tutashuv yoki portlashlarga olib kelishi mumkin.

Xususan, elektr uskunalarda kabellar va kontaktlarning ortiqcha qizishi, izolyatsiyaning yemirilishi yong'in manbai sifatida ko'p hollarda qayd etiladi.

Yong'in xavfsizligini ta'minlash uchun uskunaning barcha qismlari va bog'lovchi elementlari muntazam texnik ko'rikdan o'tkazilishi zarur. Ishlab chiqarish jarayonida intensiv ishlayotgan mexanizmlar, masalan, pechlar, presslar, motorlar, kompressorlar va konveyerlar kabi uskunarlar har kuni yoki haftalik jadval asosida tekshirib borilishi kerak. Nosozlik aniqlangan hollarda esa u darhol ishlatishdan chiqarilib, muammo bartaraf etilgachgina qayta ishga tushirilishi lozim.

Bundan tashqari, yonuvchan suyuqliklar, gazlar yoki portlovchi aralashmalar bilan ishlaydigan uskunarlar alohida xavf ostida bo'ladi. Bunday uskunarlar joylashgan xonalarda ventilyatsiya tizimi mukammal bo'lishi, gaz sizishi yoki suyuqlik to'kilishining oldini oladigan avtomatik sensorlar o'rnatilishi shart. Harorat, bosim, gaz miqdorini nazorat qiluvchi ko'rsatkichlar to'g'ri ishlashini doimiy tekshirib borish yong'inning oldini olishda muhim ahamiyatga ega.

Uskunarlar joylashtirilayotgan maydon ham yong'in xavfsizligi tamoyillariga javob berishi kerak. Ular orasida minimal masofa saqlanishi, havo aylanishi erkin bo'lishi, elektr manбайдan to'sqinliksiz foydalanish mumkin bo'lishi kerak. Uskuna atrofida yonuvchan materiallar (qog'oz, yog', kimyoviy moddalar) saqlanmasligi lozim. Agar bu uskunarlar bilan ishlovchi xodimlar doimiy harakatda bo'lsa, yong'in sodir bo'lishi mumkin bo'lgan holatlarda ularning evakuatsiyasi uchun ochiq yo'laklar mavjud bo'lishi zarur.

Ko'plab sanoat korxonalarida uskunarlar avtomatlashtirilgan bo'lib, yong'in xavfsizligi tizimlari bilan integratsiyalashgan bo'lishi mumkin. Masalan, harorat sensori yuqori darajani ko'rsatsa, uskuna avtomatik ravishda o'zini o'chiradi. Yoki gaz sizishini aniqlovchi detektor ishga tushganda darhol signalizatsiya ishlaydi va evakuatsiya tizimi faollashadi. Bunday xavfsizlik mexanizmlari ayniqsa portlovchi va toksik materiallar bilan ishlovchi uskunalarda albatta bo'lishi kerak. Uskunarlar bilan

ishlovchi xodimlarning bilim darajasi ham yong'in xavfsizligi tizimining muhim qismidir. Har bir xodim o'zi ishlayotgan uskuna qanday sharoitda yong'in xavfini tug'dirishi mumkinligini bilishi, favqulodda holatlarda qanday harakat qilishni o'zlashtirgan bo'lishi lozim. Ishga kirayotgan har bir ishchi yong'in xavfsizligi bo'yicha o'qitilishi, amaliy mashg'ulotlar orqali o'zini himoya qilish, uskunani xavfsiz to'xtatish yoki yordam chaqirish qobiliyatiga ega bo'lishi kerak.

Shu bilan birga, har bir uskuna joylashgan joyda yong'in qarsi kurash vositalari – o't o'chirgichlar, ko'pik purkagichlar, qum qutilari va signal tugmalari o'z o'rnida, foydalanishga tayyor holatda turishi shart. Ular alohida belgilar bilan ajratilgan, ko'zga ko'rinadigan va yetib borishga qulay bo'lgan joylarda saqlanishi lozim. Texnika texnologiyalar qanday rivojlanmasin, inson omili, ya'ni hushyorlik, mas'uliyat, ehtiyotkorlik va bilim asosiy rol o'ynaydi.

Xulosa qilib aytganda, ishlab chiqarishdagi har bir uskuna yong'in xavfini tug'dirish ehtimoliga ega. Buni faqat texnik jihatdan emas, balki inson omili, xulq-atvor, ishlash madaniyati va mas'uliyatli yondashuv bilan nazorat qilish mumkin. Uskunaga qo'yiladigan yong'in xavfsizligi talablari bu borada qonuniy, texnik va axloqiy mas'uliyatning uyg'unlashgan ko'rinishidir. Bu talablarning to'liq bajarilishi ishlab chiqarishning barqarorligini, ishchilarning hayoti va sog'lig'ini, mulkning saqlanishini kafolatlaydi.

Ishlab chiqarish binolarida yong'in xavfsizligini ta'minlashda yong'in o'chirish vositalari va maxsus jihozlarning mavjudligi hamda ularning to'g'ri ishlash holatida bo'lishi hayotiy ahamiyatga ega. Bu vositalar har qanday yong'inning ilk bosqichidayoq uni nazorat ostiga olish, tarqalishining oldini olish va insonlar hamda mulkni himoya qilish imkonini beradi. Ko'p hollarda yong'in sodir bo'lishi vaqt masalasiga bog'liq bo'lib, voqea joyida birinchi bo'lib qanday harakat qilinishi muammoning qanchalik jiddiy tus olishini belgilab beradi. Aynan shunday daqiqalarda

oddiy ko‘ringan, ammo to‘g‘ri joylashtirilgan va ishlashga tayyor yong‘in o‘chirish vositalari katta fojialarning oldini oladi.

Birinchi navbatda, eng ko‘p uchraydigan va universal vosita bu o‘t o‘chirgich balonlardir. Ular tarkibiga ko‘ra turlicha bo‘lishi mumkin: suv asosidagi, ko‘pik chiqaruvchi, karbonat angidrid gazli yoki quruq kimyoviy kukunli balonlar. Har bir turi ma’lum bir turdagi yong‘inlar uchun mo‘ljallangan bo‘lib, noto‘g‘ri ishlatilganda hatto yong‘inni yanada kuchaytirib yuborishi mumkin. Shuning uchun har bir ishlab chiqarish korxonasi o‘zining xavf darajasiga mos o‘t o‘chirgich turini tanlashi va ularni to‘g‘ri joylashtirishi zarur. Yong‘in chiqish ehtimoli yuqori bo‘lgan hududlarda, masalan, omborlar, yog‘ bilan ishlovchi uskunalar atrofida yoki gaz ballonlar saqlanadigan joylarda maxsus ko‘pik purkagichlar yoki chang purkaydigan moslamalar o‘rnatiladi. Shuningdek, gidrant tizimlari orqali binoning har qanday nuqtasiga suv yetkazish imkoniyati yaratiladi. Ular tashqarida yoki bino ichida joylashgan bo‘lib, maxsus suv liniyalari orqali yong‘in vaqtida kuchli bosimda suv bilan ta’minlaydi. Bu vositalar yirik ishlab chiqarish obyektlari, sexlar yoki ko‘p qavatli korxonalar uchun alohida ahamiyatga ega [18].

Yana bir zarur vosita bu tutun va olovni erta bosqichda aniqlovchi signalizatsiya tizimlari. Ular orqali yong‘in hali ko‘zga ko‘rinmasdan, tutun chiqishi boshlanganda ogohlantirish signali beriladi. Bu signal orqali xodimlar darhol evakuatsiya qilinadi, xavfli zona ajratiladi va tezkor chora ko‘rishga imkon yaratiladi. Ba’zi ilg‘or tizimlarda signal berilgach, avtomatik suv purkagich (sprinkler) tizimi ishga tushadi va yong‘inni o‘z-o‘zidan o‘chirish harakati boshlanadi. Bu texnologiya ayniqsa ommaviy ob’yektlarda keng qo‘llanilmoqda. Yong‘in o‘chirish vositalari doimo ko‘rinadigan, belgilangan joylarda, yo‘laklarga yaqin, odamlar tez yetib bora oladigan holatda saqlanishi shart. Ularning ustiga hech qanday buyumlar qo‘yilmasligi, eshiklari berk qilinmasligi kerak. Har bir vositaning yonida foydalanish bo‘yicha qisqa ko‘rsatmalar yozilgan bo‘lishi va bu ko‘rsatmalar oddiy, tushunarli tilda berilishi talab qilinadi.

Ayniqsa, kechayu kunduz faoliyat yurituvchi korxonalarda bu vositalarning holati muntazam tekshirib borilishi zarur, chunki nosoz yoki bosimi yo‘q balon voqea sodir bo‘lganda umuman foyda bermaydi.

Shuningdek, qum qutilari, yong‘in chelaklari, o‘tkir belkuraklar, issiqlikka chidamli asboblari, maxsus yong‘in o‘chirish kiyimlari, niqoblar va fonarlar ham korxonada saqlanishi kerak bo‘lgan jihozlari qatoriga kiradi. Ular xavfli zonalarda ishlovchi xodimlar yoki birinchi yordam ko‘rsatuvchi brigadalar uchun ajratilgan bo‘lishi mumkin. Qulaylik yaratish uchun ayrim joylarda yong‘inga qarshi to‘plam – “o‘t o‘chirish stendi” ko‘rinishida umumlashtirilgan holda saqlanishi ham amalda keng tarqalgan.

O‘qitish va amaliy mashg‘ulotlar orqali xodimlar bu vositalardan qanday foydalanishni, qaysi turdagi yong‘inga qanday vosita ishlatilishini, yong‘in vaqtida o‘zini va boshqalarni qanday himoya qilishni puxta bilishi kerak. Chunki eng zamonaviy vosita ham agar undan qanday foydalanishni bilmasa, foydasiz bo‘lib qoladi. Shu bois yong‘in o‘chirish vositalari va jihozlari faqatgina mavjud bo‘lishi bilan emas, balki to‘g‘ri joylashtirilganligi, ishlashga tayyorligi va ular bilan ishlay oladigan xodimlarning tayyorgarligi bilan ham yong‘in xavfsizligini ta‘minlashda asosiy omillardan biri bo‘lib xizmat qiladi.

XULOSA

Mazkur bitiruv malakaviy ish doirasida “AutoBackup” nomli avtomatik zaxiralash dasturi ishlab chiqildi. Ushbu dastur foydalanuvchi kompyuterida saqlanayotgan muhim fayllarni avtomatik tarzda zaxiralab borish, ularning versiyalarini yuritish va zarur holatda tiklash imkoniyatini beruvchi ishonchli tizim sifatida yaratilgan. Ish jarayonida zamonaviy texnologiyalar asosida foydalanuvchi uchun qulay, xavfsiz va funksional jihatdan boy bo‘lgan desktop dastur ishlab chiqildi. Dastur foydalanuvchining tanlangan fayllarini kuzatib boradi, ular o‘zgartirilgan taqdirda avtomatik ravishda yangi versiyasini yaratadi va ma’lumotlar serverga shifrlangan holda uzatiladi.

Loyiha davomida dasturiy ta’minotni ishlab chiqishning barcha asosiy bosqichlari: texnologik yechim tanlash, dastur arxitekturasini loyihalash, foydalanuvchi interfeysini ishlab chiqish, ma’lumotlar bazasi bilan ishlash, fayl versiyalash algoritmini joriy etish va xavfsiz autentifikatsiya mexanizmlarini amalga oshirish kabi jarayonlar bosqichma-bosqich bajarildi. Dastur Spring Boot asosida backend, JavaFX asosida interfeys, PostgreSQL ma’lumotlar bazasi va Keycloak orqali autentifikatsiya tizimi bilan ishlaydi. Loyihada foydalanuvchi tajribasini soddalashtirish va interfeysni tushunarli qilishga alohida e’tibor qaratildi.

AutoBackup dasturi foydalanuvchiga o‘z ma’lumotlarini yo‘qotish xavfisiz saqlash imkonini beruvchi vosita sifatida xizmat qiladi. Tizim fayllarning faqat o‘zgargan qismlarini zaxiralash orqali tizim resurslarini tejaydi va foydalanuvchiga istalgan vaqtda oldingi versiyani tiklash imkonini beradi. Shuningdek, foydalanilgan texnologiyalar muallifga real loyihalarda qo‘llaniladigan murakkab funksiyalarni amalda tatbiq etish, xavfsizlikka doir ilg‘or yondashuvlar bilan tanishish va to‘liq funksional tizim yaratish borasida mustahkam amaliy bilim va tajriba berdi.

Ushbu dastur nafaqat oddiy foydalanuvchilar, balki tashkilotlar, ofis xodimlari va IT infratuzilmasiga ega tashkilotlar uchun ham zarur vosita sifatida xizmat qilishi

mumkin. Kelajakda dastur imkoniyatlarini yanada kengaytirish - masalan, bulutli saqlash tizimlari bilan integratsiya, mobil qurilmalarga moslash, yoki veb-interfeys orqali boshqaruv imkoniyatlarini yaratish orqali tizimni keng qamrovli va moslashuvchan platformaga aylantirish mumkin.

FOYDALANILGAN ADABIYOTLAR VA MANBALAR.

1. Velte T., Velte A , Elsenpeter R. Cloud Computing: A Practical Approach. McGraw-Hill, 2010.
2. Ghemawat S., Gobioff H., Leung S. T. The Google File System ACM SIGOPS Operating Systems Review. – 2003. – Vol. 37, No 5.
3. De Loof N. Managing Files and Folders with Java. – Oracle Developer Guides, 2019.
4. Kumar S., Thakur R. S. Beginning PostgreSQL on the Cloud: Simplifying Database as a Service on Cloud Platforms. – Apress, 2021.
5. Fisher M.T. Modern PostgreSQL: Introduction and Practical Guide. – Apress, 2020.
6. Bryant D., Marin-Perez A. Securing Spring Applications. – O'Reilly Media, 2020.
7. Merkel D. Spring Boot Essentials. – O'Reilly Media, 2018.
8. Soto A., Porter J. Keycloak – Identity and Access Management for Modern Applications. – Packt Publishing, 2022.
9. Harold E. R. Java I/O. – 2nd ed. – O'Reilly Media, 2006.
10. Stallings W. Operating Systems: Internals and Design Principles. – 9th ed. – Pearson, 2018.
11. Walls C. Spring in Action. – Manning Publications, 2022.
12. Jackson W. JavaFX for Dummies. – Wiley, 2014.
13. Schneier B. Applied Cryptography: Protocols, Algorithms, and Source Code in C. – Wiley, 2015.
14. Hayot faoliyati xavfsizligi. Ekologiya. Sapaev M.S., Qodirov F.M. O'quv qo'llanma, Toshkent – “Aloqachi” – 2019

15. Hayot faoliyati xavfsizligi va ekologiya menejmenti (chizmalar, tushunchalar, faktlar va raqamlarda): darslik / A. Nigmatov, Sh. Muxamedov, N. Xasanova. – T.: Navro‘z, 2014.
16. Oracle Corporation. OpenJFX Documentation. – [Elektron resurs]: <https://openjfx.io> – Murojaat qilingan sana: 2024.
17. Apple Inc. iCloud Security Overview. – Apple White Paper, 2022. URL: <https://support.apple.com/en-us/HT202303>
18. O‘zbekiston Respublikasi Vazirlar Mahkamasining “Yong‘in xavfsizligi qoidalarini tasdiqlash” to‘g‘risida qarori <https://lex.uz/uz/docs/-5056473?ONDATE=05.04.2023>

Ilova

```
public class LoginController implements Initializable
    @FXML
    private TextField usernameField;
    @FXML
    private PasswordField passwordField;
    @FXML
    private Label errorLabel;
    @FXML
    private Button loginButton;
    @FXML
    private Hyperlink signupLink;
    @Override
    public void initialize(URL url, ResourceBundle resourceBundle) {
        errorLabel.setText("");
    }

    @FXML
    void handleLogin(ActionEvent event) {
        loginButton.setDisable(true);

        errorLabel.setText("");

        String username = usernameField.getText().trim();
        String password = passwordField.getText();

        if (username.isEmpty() || password.isEmpty()) {
            showError("Kirib bo‘lmadi", "Xatolik", "Username yoki parol bo‘sh bo‘lmasligi kerak.");
            loginButton.setDisable(false);
            return;
        }

        new Thread(() -> {
            try {
                User user = ApiClient.login(username, password);
                LocalFileService.saveUser(user.getId());

                Platform.runLater(() -> {
                    try {
```

```

        EncryptionService.getInstance().initialize(password);
    } catch (Exception ex) {
        showError("Xatolik", "Shifrlash kalitini yuklashda muammo",
ex.getMessage());
        loginButton.setDisable(false);
        return;
    }
    BackupService.getInstance().startDaemonService();
    AutoBackupApplication.switchToMainView();
});

} catch (Exception e) {
    Platform.runLater(() -> {
        showError("Kirib bo'lmadi", "Xatolik", e.getMessage());
        loginButton.setDisable(false);
    });
}
}).start();
}

```

@FXML

```

void handleSignUp(ActionEvent event) {
    Dialog<Map<String, String>> dialog = new Dialog<>();
    dialog.setTitle("Ro'yxatdan o'tish");
    dialog.setHeaderText("Yangi hisob yaratish");

    ButtonType registerButtonType = new ButtonType("Account Yaratish",
ButtonBar.ButtonData.OK_DONE);
    dialog.getDialogPane().getButtonTypes().addAll(registerButtonType,
ButtonType.CANCEL);

    GridPane grid = new GridPane();
    grid.setHgap(10);
    grid.setVgap(10);
    grid.setPadding(new Insets(20, 150, 10, 10));

    TextField usernameField = new TextField();
    usernameField.setPromptText("Username");
    TextField emailField = new TextField();
    emailField.setPromptText("Email");
}

```

```

PasswordField passwordField = new PasswordField();
passwordField.setPromptText("Password");
grid.add(new Label("Username:"), 0, 0);
grid.add(usernameField, 1, 0);
grid.add(new Label("Email:"), 0, 1);
grid.add(emailField, 1, 1);
grid.add(new Label("Password:"), 0, 2);
grid.add(passwordField, 1, 2);

Node registerButton =
dialog.getDialogPane().lookupButton(registerButtonType);
registerButton.setDisable(true);

Runnable validateFields = () -> {
    boolean isValid = !usernameField.getText().trim().isEmpty() &&
        !emailField.getText().trim().isEmpty() &&
        !passwordField.getText().trim().isEmpty();
    registerButton.setDisable(!isValid);
};

usernameField.textProperty().addListener((observable, oldValue, newValue) ->
validateFields.run());
emailField.textProperty().addListener((observable, oldValue, newValue) ->
validateFields.run());
passwordField.textProperty().addListener((observable, oldValue, newValue) ->
validateFields.run());

dialog.getDialogPane().setContent(grid);

Platform.runLater(usernameField::requestFocus);

dialog.setResultConverter(dialogButton -> {
    if (dialogButton == registerButtonType) {
        return Map.of(
            "username", usernameField.getText().trim(),
            "email", emailField.getText().trim(),
            "password", passwordField.getText().trim()
        );
    }
    return null;
});

```

```

Optional<Map<String, String>> result = dialog.showAndWait();

result.ifPresent(formData -> {
    dialog.getDialogPane().setDisable(true);

    new Thread() -> {
        try {
            ApiClient.register(
                formData.get("username"),
                formData.get("email"),
                formData.get("password"),
                "", ""
            );

            Platform.runLater() -> {
                Alert alert = new Alert(Alert.AlertType.INFORMATION);
                alert.setTitle("Hisobingiz Yaratildi");
                alert.setHeaderText("Hisobingiz Yaratildi");
                alert.setContentText("Hisobingiz muvaffaqiyatli yaratildi.\nTizimga kirishingiz mumkin.");
                alert.showAndWait();
            }
        }
    }

    LoginController.this.usernameField.setText(formData.get("username"));

    LoginController.this.passwordField.setText(formData.get("password"));
});
} catch (Exception e) {
    Platform.runLater() -> {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setTitle("Hisob yaratishda xatolik");
        alert.setHeaderText("Hisob yaratishda xatolik");
        alert.setContentText("Hisob yaratib bo'lmadi: " + e.getMessage());
        alert.showAndWait();
    }
}
}).start();
});
}
}
}

```

```

public class LocalFileService {

    private static final String FILE_PATH = "src/main/resources/user.json";
    private static final Gson gson = new Gson();

    public static void saveUser(Integer userId) {
        try (FileWriter writer = new FileWriter(FILE_PATH)) {
            gson.toJson(Map.of("currentUserId", userId), writer);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static Long getUserId() {
        try (FileReader reader = new FileReader(FILE_PATH)) {
            Map<?, ?> data = gson.fromJson(reader, Map.class);
            if (data != null && data.get("currentUserId") != null) {
                return ((Number) data.get("currentUserId")).longValue(); // Safe conversion
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        return null;
    }

    public static void clear(){
        File file = new File(FILE_PATH);
        if (file.exists()) {
            file.delete(); // Delete the file
        }
        try {file.createNewFile(); // Create a new empty file
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

@RestController
@RequestMapping("/api/auth")
public class AuthController {

```

```

private final UserService userService;

@Autowired
public AuthController(UserService userService) {
    this.userService = userService;
}

@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody Map<String, String> request) {
    String username = request.get("username");
    String email = request.get("email");
    String password = request.get("password");

    if (username == null || email == null || password == null) {
        return ResponseEntity.badRequest().body("Username, email, and password
are required");
    }

    try {
        User user = userService.register(username, email, password);

        Map<String, Object> response = new HashMap<>();
        response.put("id", user.getId());
        response.put("username", user.getUsername());
        response.put("email", user.getEmail());

        return ResponseEntity.ok(response);
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}

@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody Map<String, String> request) {
    String username = request.get("username");
    String password = request.get("password");

    if (username == null || password == null) {
        return ResponseEntity.badRequest().body("Username and password are
required");
    }
}

```



```

    }

    try {
        User user = userService.login(username, password);

        Map<String, Object> response = new HashMap<>();
        response.put("id", user.getId());
        response.put("username", user.getUsername());
        response.put("email", user.getEmail());

        return ResponseEntity.ok(response);
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}

@GetMapping("/user/{id}")
public ResponseEntity<?> getUserById(@PathVariable Long id) {
    try {
        User user = userService.findById(id);

        Map<String, Object> response = new HashMap<>();
        response.put("id", user.getId());
        response.put("username", user.getUsername());
        response.put("email", user.getEmail());

        return ResponseEntity.ok(response);
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}
}

```

```

@RestController
@RequestMapping("/api")
public class BackupController {

```

```

    private static final Logger logger =
Logger.getLogger(BackupController.class.getName());
    private final BackupService backupService;
    private final UserService userService;

    @Autowired
    public BackupController(BackupService backupService, UserService userService)
    {
        this.backupService = backupService;
        this.userService = userService;
    }

    @GetMapping("/files/user/{userId}")
    public ResponseEntity<?> getUserFiles(@PathVariable Long userId) {
        try {
            User user = userService.findById(userId);

            List<BackupFile> files = backupService.getUserFiles(user);

            List<Map<String, Object>> fileDtos = files.stream().map(file -> {
                Map<String, Object> dto = new HashMap<>();
                dto.put("id", file.getId());
                dto.put("userId", file.getUser().getId());
                dto.put("fileName", file.getFileName());
                dto.put("originalPath", file.getRealFilePath());
                dto.put("backupFrequency", file.getBackupFrequency());
                dto.put("versionCount", file.getVersionCount());
                dto.put("lastBackupTime", file.getLastBackupTime());
                dto.put("nextBackupTime", file.getNextBackupTime());
                dto.put("createdAt", file.getCreatedAt());
                dto.put("status", file.getStatus());
                return dto;
            }).collect(Collectors.toList());

            return ResponseEntity.ok(fileDtos);
        } catch (Exception e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
}

```

```

@PostMapping(value = "/files", consumes =
MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<?> createBackupFile(
    @RequestPart("file") MultipartFile file,
    @RequestParam("userId") Long userId,
    @RequestParam("backupFrequency") String backupFrequency,
    @RequestParam("realFilePath") String realFilePath,
    @RequestParam("fileHash") String fileHash
) {
    try {
        System.out.println("Received userId: " + userId);
        System.out.println("Received backupFrequency: " + backupFrequency);
        System.out.println("Received file: " + file.getOriginalFilename());

        User user = userService.findById(userId);
        String fileName = file.getOriginalFilename();
        String directoryPath = "uploads";
        String filePath = directoryPath + "/" + fileName;

        File directory = new File(directoryPath);
        if (!directory.exists()) {
            directory.mkdirs();
        }

        Files.copy(file.getInputStream(), Paths.get(filePath),
StandardCopyOption.REPLACE_EXISTING);

        BackupFile backupFile = backupService.createBackupFile(user, fileName,
filePath, Integer.valueOf(backupFrequency) , realFilePath , fileHash);

        Map<String, Object> dto = new HashMap<>();
        dto.put("id", backupFile.getId());
        dto.put("userId", backupFile.getUser().getId());
        dto.put("fileName", backupFile.getFileName());
        dto.put("filePath", backupFile.getFilePath());
        dto.put("backupFrequency", backupFile.getBackupFrequency());
        dto.put("versionCount", backupFile.getVersionCount());
        dto.put("lastBackupTime", backupFile.getLastBackupTime());
        dto.put("nextBackupTime", backupFile.getNextBackupTime());
        dto.put("realFilePath", backupFile.getRealFilePath());
    }
}

```

```

        dto.put("status", backupFile.getStatus());

        return ResponseEntity.ok(dto);
    } catch (Exception e) {
        e.printStackTrace();
        return ResponseEntity.badRequest().body("Error: " + e.getMessage());
    }
}

@PutMapping("/files/{fileId}")
public ResponseEntity<?> updateBackupFile(@PathVariable Long fileId,
@RequestBody Map<String, Object> request) {
    try {
        Integer backupFrequency = request.containsKey("backupFrequency") ?
Integer.valueOf(request.get("backupFrequency").toString()) : null;
        String status = request.containsKey("status") ? request.get("status").toString()
: null;

        BackupFile updatedFile = backupService.updateBackupFile(fileId,
backupFrequency, status);

        Map<String, Object> dto = new HashMap<>();
        dto.put("id", updatedFile.getId());
        dto.put("userId", updatedFile.getUser().getId());
        dto.put("fileName", updatedFile.getFileName());
        dto.put("filePath", updatedFile.getFilePath());
        dto.put("backupFrequency", updatedFile.getBackupFrequency());
        dto.put("versionCount", updatedFile.getVersionCount());
        dto.put("lastBackupTime", updatedFile.getLastBackupTime());
        dto.put("status", updatedFile.getStatus());

        return ResponseEntity.ok(dto);
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}

@GetMapping("/files/{fileId}")
public ResponseEntity<?> getBackupFile(@PathVariable Long fileId) {
    try {

```

```

BackupFile updatedFile = backupService.getBackupFile(fileId);

Map<String, Object> dto = new HashMap<>();
dto.put("id", updatedFile.getId());
dto.put("userId", updatedFile.getUser().getId());
dto.put("fileName", updatedFile.getFileName());
dto.put("filePath", updatedFile.getFilePath());
dto.put("backupFrequency", updatedFile.getBackupFrequency());
dto.put("versionCount", updatedFile.getVersionCount());
dto.put("lastBackupTime", updatedFile.getLastBackupTime());
dto.put("status", updatedFile.getStatus());
dto.put("fileHash", updatedFile.getFileHash());
dto.put("realFilePath", updatedFile.getRealFilePath());

return ResponseEntity.ok(dto);
} catch (Exception e) {
    return ResponseEntity.badRequest().body(e.getMessage());
}
}

@DeleteMapping("/files/{fileId}")
public ResponseEntity<?> deleteBackupFile(@PathVariable Long fileId) {
    try {
        backupService.deleteBackupFile(fileId);

        return ResponseEntity.ok("File deleted successfully");
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}

@GetMapping("/versions/file/{fileId}")
public ResponseEntity<?> getFileVersions(@PathVariable Long fileId) {
    try {
        List<BackupVersion> versions = backupService.getFileVersions(fileId);

        List<Map<String, Object>> versionDtos = versions.stream().map(version -> {
            Map<String, Object> dto = new HashMap<>();
            dto.put("id", version.getId());
            dto.put("fileId", version.getBackupFile().getId());

```

```

        dto.put("versionNumber", version.getVersionNumber());
        dto.put("fileUrl", version.getFileUrl());
        dto.put("fileSize", version.getFileSize());
        dto.put("creationTime", version.getCreationTime());
        return dto;
    }).collect(Collectors.toList());

    return ResponseEntity.ok(versionDtos);
} catch (Exception e) {
    return ResponseEntity.badRequest().body(e.getMessage());
}
}

@PostMapping("/files/{fileId}/upload")
public ResponseEntity<?> uploadBackupVersion(@PathVariable Long fileId,
@RequestParam("file") MultipartFile file , @RequestParam String fileHash) {
    try {
        BackupVersion version = backupService.createBackupVersion(fileId, file ,
fileHash);

        Map<String, Object> dto = new HashMap<>();
        dto.put("id", version.getId());
        dto.put("fileId", version.getBackupFile().getId());
        dto.put("versionNumber", version.getVersionNumber());
        dto.put("fileUrl", version.getFileUrl());
        dto.put("fileSize", version.getFileSize());
        dto.put("creationTime", version.getCreationTime());

        return ResponseEntity.ok(dto);
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}

@GetMapping("/versions/{versionId}/download")
public ResponseEntity<Resource> downloadBackupVersion(@PathVariable Long
versionId) {
    try {
        Resource resource = backupService.getBackupVersion(versionId);

        String contentType = "application/octet-stream";

```

```

        try {
            contentType =
Files.probeContentType(Paths.get(resource.getFile().getAbsolutePath()));
            if (contentType == null) {
                contentType = "application/octet-stream";
            }
        } catch (IOException ex) {
            logger.warning("Could not determine file type: " + ex.getMessage());
        }

        String filename = resource.getFilename();
        if (filename.matches("^\\d+_.*")) {
            filename = filename.substring(filename.indexOf('_') + 1);
        }

        return ResponseEntity.ok()
            .contentType(MediaType.parseMediaType(contentType))
            .header(HttpHeaders.CONTENT_DISPOSITION, "attachment;
filename=\"" + filename + "\"")
            .header(HttpHeaders.CACHE_CONTROL, "no-cache, no-store")
            .header(HttpHeaders.PRAGMA, "no-cache")
            .header(HttpHeaders.EXPIRES, "0")
            .body(resource);
    } catch (Exception e) {
        e.printStackTrace();
        return ResponseEntity.badRequest().body(null);
    }
}

@GetMapping("/files/{fileId}/rollback/{versionId}")
public ResponseEntity<?> rollbackToVersion(@PathVariable Long fileId,
@PathVariable Long versionId) {
    try {
        Map<String, Object> result = backupService.rollbackToVersion(fileId,
versionId);
        return ResponseEntity.ok(result);
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}
}

```

```

@Service
public class BackupService {

    private final BackupFileRepository backupFileRepository;
    private final BackupVersionRepository backupVersionRepository;
    private final FileStorageService fileStorageService;

    @Autowired
    public BackupService(BackupFileRepository backupFileRepository,
BackupVersionRepository backupVersionRepository, FileStorageService
fileStorageService) {
        this.backupFileRepository = backupFileRepository;
        this.backupVersionRepository = backupVersionRepository;
        this.fileStorageService = fileStorageService;
    }

    public List<BackupFile> getUserFiles(User user) {
        System.out.println(LocalDateTime.now() + " -- >> ");
        return backupFileRepository.findByUserId(user.getId());
    }

    public BackupFile createBackupFile(User user, String fileName, String filePath,
Integer backupFrequency, String realFilePath , String fileHash) {
        BackupFile backupFile = new BackupFile();
        backupFile.setUser(user);
        backupFile.setFileName(fileName);
        backupFile.setFilePath(filePath);
        backupFile.setBackupFrequency(backupFrequency);
        backupFile.setRealFilePath(realFilePath);

        backupFile.setNextBackupTime(LocalDateTime.now().plusMinutes(backupFrequenc
y));
        backupFile.setFileHash(fileHash);

        backupFile.setStatus("ACTIVE");

        return backupFileRepository.save(backupFile);
    }
}

```



```

    public BackupFile updateBackupFile(Long fileId, Integer backupFrequency, String
status) {
        BackupFile backupFile = backupFileRepository.findById(fileId)
            .orElseThrow(() -> new RuntimeException("Backup file not found"));

        if (backupFrequency != null) {
            backupFile.setBackupFrequency(backupFrequency);
        }

        if (status != null) {
            backupFile.setStatus(status);
        }

        return backupFileRepository.save(backupFile);
    }

    public void deleteBackupFile(Long fileId) {
        BackupFile backupFile = backupFileRepository.findById(fileId)
            .orElseThrow(() -> new RuntimeException("Backup file not found"));

        List<BackupVersion> versions =
backupVersionRepository.findByBackupFileOrderByVersionNumberDesc(backupFil
e);
        for (BackupVersion version : versions) {
            fileStorageService.deleteFile(version);
            backupVersionRepository.delete(version);
        }

        backupFileRepository.delete(backupFile);
    }

    public List<BackupVersion> getFileVersions(Long fileId) {
        BackupFile backupFile = backupFileRepository.findById(fileId)
            .orElseThrow(() -> new RuntimeException("Backup file not found"));

        return
backupVersionRepository.findByBackupFileOrderByVersionNumberDesc(backupFil
e);
    }

```

```

    public BackupVersion createBackupVersion(Long fileId, MultipartFile file , String
fileHash) {
        BackupFile backupFile = backupFileRepository.findById(fileId)
            .orElseThrow(() -> new RuntimeException("Backup file not found"));

        BackupVersion version = fileStorageService.storeFile(file, backupFile);

        backupFile.setLastBackupTime(LocalDateTime.now());
        backupFile.setFileHash(fileHash);
        backupFileRepository.save(backupFile);

        return version;
    }

    public Resource getBackupVersion(Long versionId) {
        BackupVersion version = backupVersionRepository.findById(versionId)
            .orElseThrow(() -> new RuntimeException("Version not found"));

        return fileStorageService.loadFileAsResource(version);
    }

    public Map<String, Object> rollbackToVersion(Long fileId, Long versionId) {
        BackupFile backupFile = backupFileRepository.findById(fileId)
            .orElseThrow(() -> new RuntimeException("Backup file not found"));

        BackupVersion version = backupVersionRepository.findById(versionId)
            .orElseThrow(() -> new RuntimeException("Version not found"));

        if (!version.getBackupFile().getId().equals(fileId)) {
            throw new RuntimeException("Version does not belong to this file");
        }

        try {
            Resource resource = fileStorageService.loadFileAsResource(version);

            File originalFile = new File(backupFile.getFilePath());
            Files.copy(resource.getInputStream(), Paths.get(backupFile.getFilePath()),
StandardCopyOption.REPLACE_EXISTING);
            File versionFile = new File(resource.getURI());
            Map<String , Object> res = new HashMap<>();
            res.put("file", versionFile);

```

```

        res.put("filePath", backupFile.getRealFilePath());

        Map<String, Object> dto = new HashMap<>();
        dto.put("id", version.getId());
        dto.put("fileId", version.getBackupFile().getId());
        dto.put("versionNumber", version.getVersionNumber());
        dto.put("fileUrl", version.getFileUrl());
        dto.put("fileSize", version.getFileSize());
        dto.put("creationTime", version.getCreationTime());

        res.put("version" , dto);
        return res;
    } catch (Exception e) {
        throw new RuntimeException("Failed to rollback: " + e.getMessage());
    }
}

public BackupFile getBackupFile(Long fileId) {
    return backupFileRepository.findById(fileId).orElse(null);
}
}

```